Enterprise AI Platform Playbook

A unified, implementation-ready blueprint for MLOps, SLMOps and LLMOps

Version 1.0 — 25 June 2026
Scope: design, build, deploy, operate and scale enterprise AI systems across classical machine learning, small and large language models, retrieval-augmented generation and bounded agents.

> **The platform is not a collection of notebooks and endpoints. It is the enterprise control system that turns data, code, models, prompts and policies into governed, observable, repeatable business services.**

This playbook is technology-neutral. Product names are representative options, not mandatory selections. Regulatory statements are implementation guidance, not legal advice; applicability must be verified for each jurisdiction, industry and use case.

Table of contents

1. Executive blueprint

1.1 Target outcome

An enterprise AI platform must create a controlled path from an approved business objective to a measurable production outcome. It must support multiple workload classes without duplicating enterprise controls:

- **Classical ML:** tabular prediction, ranking, recommendation, anomaly detection, forecasting, optimization, computer vision and natural-language classifiers.
- **SLM/LLM:** extraction, classification, summarization, generation, search, reasoning, translation and multimodal processing.
- **RAG:** generation grounded in enterprise or licensed knowledge with authorization-aware retrieval and source attribution.
- **Agents:** bounded workflows in which models select tools or actions under explicit policy, budgets, approvals and transaction controls.
- **Edge/on-device AI:** compact or quantized models deployed where privacy, latency, connectivity or unit economics require local inference.

The target state is a **shared control plane** plus **workload-fit data planes**. The control plane standardizes identity, policy, metadata, lineage, registries, release gates, evidence, telemetry, cost allocation and lifecycle state. Data planes may differ by region, risk tier, data residency, latency target, accelerator type or managed-service choice.

1.2 The first-principles model

Every production AI service is a versioned function:

> **Outcome = f(data, features or context, model, prompt, tools, policy, runtime, infrastructure, human process, time).**

A model artifact alone is therefore not deployable evidence. Reproducibility and accountability require the platform to bind all material inputs to one release identity.

The platform must optimize four coupled objectives:

1. **Value:** measurable improvement in revenue, cost, risk, speed, quality or experience.
2. **Trust:** acceptable safety, security, privacy, fairness, transparency and legal posture for the use context.
3. **Flow:** low-friction movement from idea to validated production service.
4. **Efficiency:** reliable performance at an economically and environmentally defensible cost.

Optimizing one while ignoring the others creates failure: fast delivery without trust creates unmanaged risk; maximal controls without self-service creates shadow AI; high benchmark scores without value create expensive demonstrations; low unit cost without reliability creates unusable services.

1.3 Recommended enterprise target state

- A global or enterprise management account/project/subscription hosts the identity integrations, policy engine, service catalog, registries, audit evidence, orchestration metadata and cost controls.
- Development, validation and production are separate trust zones. Production data and credentials do not flow backward into development.
- Domain teams consume versioned platform services through APIs, templates and Git-based workflows rather than requesting bespoke infrastructure.
- Artifacts are built once, signed, evaluated and promoted. Production does not rebuild from mutable source or download unpinned models at startup.
- Every service has an owner, risk tier, business KPI, technical SLO, evaluation policy, data classification, runbook, rollback path and retirement condition.
- Every material change to data, features, model, prompt, retrieval corpus, tool permissions, runtime or policy creates a new release candidate and executes the relevant regression suite.

- High-impact systems require independent validation and separation of duties. Low-risk systems use automated controls to preserve speed.
- All model calls, retrievals and agent actions cross policy enforcement points and generate privacy-aware, end-to-end traces.
- Cost is allocated to product, tenant, model and environment. The primary efficiency metric is **cost per successful business outcome**, not only infrastructure utilization.

1.4 Minimum viable platform versus target platform

Capability area	Minimum viable enterprise platform	Scaled target platform
Identity and tenancy	SSO, RBAC, separate environments, service identities	Attribute-based policy, workload identity, just-in-time access, delegated domain administration
Data	Versioned object storage, catalog, basic quality checks	Data contracts, automated lineage, privacy policy enforcement, streaming, feature and knowledge services
Development	Managed workspaces, source control, reproducible environments	Self-service templates, ephemeral workspaces, remote compute, policy-aware SDKs
Experimentation	Experiment tracking and artifact storage	Reproducible pipelines, dataset lineage, distributed HPO, shared evaluation service
Registry	Model registry and approval state	Unified registry for models, adapters, prompts, datasets, embeddings, policies and deployment bundles
Serving	Batch jobs and managed online endpoint	Multi-runtime inference, model routing, cache, RAG, agents, edge packaging and multi-region failover
Evaluation	Task metrics and manual review	Risk-tiered automated suites, adversarial testing, human evaluation operations, online experiments
Operations	Logs, metrics, alerts and on-call	Full tracing, quality/safety SLOs, automated rollback, error budgets, chaos and recovery testing
Governance	Inventory, ownership, risk questionnaire	Controls as code, evidence automation, recertification, regulatory profiles and exception workflows
Economics	Tags and monthly showback	Unit economics, quotas, forecasting, model routing, capacity portfolio and chargeback

1.5 Critical success conditions

- Executive sponsorship defines decision rights and accepts that AI governance is a delivery mechanism, not an after-the-fact committee.
- The platform team operates as a product organization with users, roadmaps, service levels and adoption metrics.
- Business, data, model and service ownership are explicit; “the AI team” is not a valid owner.
- Golden paths cover the dominant 70–80% of workloads while documented exception paths handle legitimate edge cases.
- Quality and risk thresholds are specified before model selection, preventing benchmark-driven architecture.
- Domain data products are contractually reliable enough for automation; no platform can compensate for unknown semantics and unstable sources.
- Platform telemetry connects technical behavior to business outcomes and user impact.

2. Scope, definitions and workload taxonomy

2.1 System boundary

The governed unit is the **AI system**, not only the model. The system includes:

- source data, labels, features, documents and embedding indexes;
- training and evaluation code, dependencies and environments;
- model weights, adapters, tokenizers and calibration artifacts;
- prompts, system instructions, examples, output schemas and policies;
- retrieval, ranking, memory and caching components;
- tools, APIs, identities, permissions and human approval steps;
- serving runtime, infrastructure, network and third-party providers;
- monitoring, feedback, support and business process integration.

A change outside the model can materially change behavior. Prompt edits, corpus refreshes, feature logic, model-provider upgrades, tool schemas and access-policy changes therefore require versioning, testing and traceability.

2.2 Core terms

- **AI platform:** shared enterprise capabilities and operating mechanisms used to create and run AI systems.
- **Control plane:** identity, policy, catalog, metadata, orchestration, registries, release, evidence, telemetry and cost management.
- **Data plane:** workload execution and state: data processing, training, inference, retrieval, tools and domain storage.
- **Model package:** immutable model artifact plus runtime contract, dependencies, signatures and metadata.
- **System release:** deployable bundle that binds model, prompt, data/feature/context versions, policies, runtime image and configuration.
- **Evaluation policy:** versioned metrics, test datasets, thresholds, segment requirements and approval rules.
- **Golden path:** supported, automated path for a common workload with secure defaults and operational ownership.
- **Exception path:** time-bound, risk-accepted deviation with named owner and remediation or exit date.
- **Continuous training (CT):** automated retraining or tuning triggered by schedule, data, drift, quality or approved demand.
- **TEVV:** test, evaluation, verification and validation across functional, safety, security, privacy and operational dimensions.

2.3 Workload archetypes

Archetype	Typical latency	State/context	Primary failure concern	Preferred delivery pattern
Offline analytics or scoring	Minutes to hours	Large bounded dataset	stale or incorrect batch, leakage	scheduled immutable pipeline with reconciled outputs
Online predictive ML	5–200 ms	request plus online features	feature skew, drift, tail latency	stateless endpoint plus low-latency feature service
Streaming inference	milliseconds to seconds	ordered event state	replay, duplication, backpressure	stream processor with idempotent sinks and checkpoints
Interactive LLM	sub-second TTFT, seconds total	prompt and session context	unsafe or ungrounded output, cost	gateway, policy, routing, inference and tracing
Enterprise RAG	seconds	authorized documents and citations	retrieval miss, stale index, ACL leak	retrieval service plus grounded generation
Agentic workflow	seconds to hours	durable workflow and tool state	unauthorized or irreversible action	state machine, sandboxed tools, approvals and compensation

Archetype	Typical latency	State/context	Primary failure concern	Preferred delivery pattern
Edge/on-device SLM	milliseconds to seconds	local context	update integrity, device heterogeneity	signed package, staged rollout and local rollback

2.4 Risk-tiering dimensions

Score each use case on a defined scale and retain the evidence. Material dimensions are:

- impact on rights, safety, employment, credit, health, access to essential services or legal status;
- autonomy and whether actions are advisory, reversible, financially material or irreversible;
- data sensitivity, children or vulnerable populations, biometrics and regulated data;
- number of affected people, transaction volume, geographic reach and external exposure;
- model opacity, novelty, non-determinism and ability to explain or contest outcomes;
- susceptibility to manipulation, fraud, prompt injection, poisoning or adversarial inputs;
- third-party dependency, model/data provenance, residency and supply-chain exposure;
- ability to detect failure, contain harm, restore service and provide human recourse.

A practical tiering scheme:

- **Tier 0 — Sandbox:** synthetic/public data, no production decisions, no external users. Controls: identity, isolation, cost quota, code and artifact scanning.
- **Tier 1 — Low:** assistive, reversible, non-sensitive, limited scope. Controls: owner, baseline evaluation, logging, standard release gates.
- **Tier 2 — Moderate:** customer or employee exposure, internal sensitive data, material workflow influence. Controls: impact assessment, privacy review, segmented evaluation, monitoring and human escalation.
- **Tier 3 — High:** consequential decisions, regulated contexts, broad external scale or autonomous actions. Controls: independent validation, formal human oversight, red team, enhanced audit, strict change control and recertification.
- **Tier 4 — Prohibited or executive exception:** unacceptable practices, uncontrollable harm or legal prohibition. Default action: reject, redesign or obtain explicit legal and executive authorization where lawful.

2.5 Service tiers

Risk tier describes harm potential; service tier describes operational criticality. Keep them separate.

Service tier	Example	Availability objective	Recovery objective	Support
Bronze	experimentation, noncritical batch	best effort	restore from source	business hours
Silver	internal production assistance	99.5% monthly	RTO 8 h / RPO 24 h	extended business hours
Gold	customer-facing or material operations	99.9% monthly	RTO 1 h / RPO 15 min	24×7 on-call
Platinum	safety, revenue or mission critical	workload-specific, often ≥99.95%	near-zero or bounded by business process	24×7, multi-region and tested failover

Targets above are examples. Set them from business-loss curves, dependency reliability and achievable architecture—not from convention.

3. Design principles and non-negotiable invariants

3.1 Architectural principles

1. **One governance model, multiple runtimes.** Unify controls and metadata; do not force tabular batch jobs, low-latency predictors, GPU LLM serving and edge models onto one execution engine.

2. **The simplest sufficient intelligence wins.** Compare rules, search, classical ML, SLM, LLM, RAG and agents against the same outcome thresholds. Complexity must buy measurable value.
3. **Artifacts are immutable; aliases move.** Promote signed versions through environments. Never mutate a production model, prompt, image or dataset in place.
4. **Everything material is versioned.** Data snapshots, feature definitions, labels, code, dependencies, model weights, adapters, prompts, indexes, policies, tool schemas and infrastructure.
5. **Evaluation is a release dependency.** A deployment may reference only an evaluation report produced against the exact release bundle.
6. **Risk-proportionate automation.** Automate low-risk approvals; require independent challenge for high-impact systems. Uniform manual governance is both slow and weak.
7. **Identity precedes network location.** Authenticate humans and workloads, authorize every resource and action, and use short-lived credentials.
8. **Production is private by default.** Private endpoints, controlled egress, deny-by-default network policy and no direct notebook access to production.
9. **State is externalized and recoverable.** Compute is replaceable; registries, metadata, artifacts, audit logs, feature state and indexes have explicit durability and restore designs.
10. **Observability is part of the contract.** No service reaches production without traces, metrics, logs, quality signals, cost attribution and synthetic probes.
11. **Business value closes the loop.** Technical metrics are necessary but insufficient. Every service maps predictions or generations to measurable outcomes and guardrail metrics.
12. **Exceptions expire.** Deviations have an owner, rationale, compensating controls, expiry date and tracked remediation.

3.2 Required invariants

- Every production request is attributable to a tenant, user or workload identity, system release and policy version.
- Every deployed artifact has a cryptographic digest, provenance record and approved source.
- Every production model has a named service owner and a named data/model owner; high-risk models have an independent validator.
- No sensitive prompt or training data is sent to an external provider without an approved data-flow record and contract controls.
- Retrieval enforces source authorization at indexing and query time; vector similarity never bypasses access control.
- Agents cannot acquire permissions dynamically from model output. Tool access is allowlisted, scoped and mediated by deterministic policy.
- Human approval is required before high-impact, irreversible or legally significant actions unless an explicitly approved control design provides equivalent protection.
- Monitoring and audit pipelines are themselves monitored. Missing telemetry is an incident, not “no issue detected.”
- Production rollback does not depend on retraining or internet access.
- Data deletion and retention policies propagate to derived datasets, feature materializations, vector indexes, logs and backups according to defined procedures.

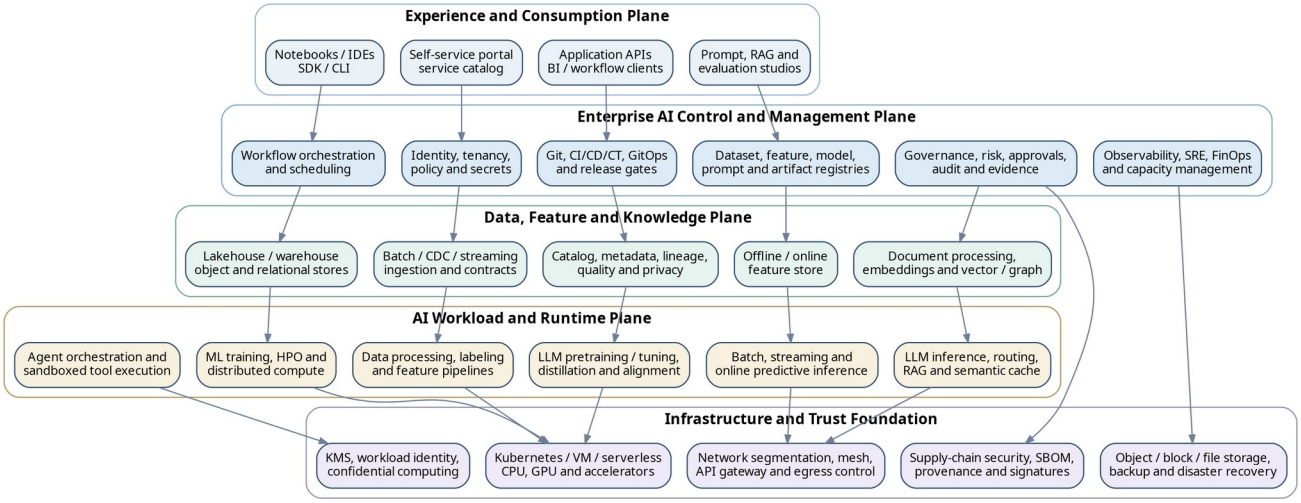
3.3 Anti-principles

Avoid these design assumptions:

- “A single vendor suite eliminates architecture.” It relocates integration and concentration risk; it does not remove them.
- “A foundation model benchmark predicts our use case.” Only representative, segment-aware evaluation predicts fitness.
- “RAG solves hallucination.” Retrieval adds evidence but also introduces retrieval misses, stale content, injection and authorization failure modes.
- “Agents are prompts with tools.” Production agents are distributed systems with identity, state, transactions, policy, failure recovery and audit.
- “More GPUs equals faster delivery.” Data readiness, evaluation throughput, queueing, networking, storage and approvals often dominate lead time.

- "Governance is documentation." Effective governance is executable policy, evidence generation, decision rights and monitored controls.

4. Enterprise reference architecture



Unified logical architecture. Shared enterprise controls sit above workload-fit data and runtime planes.

4.1 Architecture planes

Experience and consumption plane

Purpose. Provide secure, consistent entry points for builders, operators, reviewers and consuming applications.

Capabilities.

- self-service portal and service catalog with entitlement-aware requests;
- managed notebooks, browser IDEs, remote development, SDKs and CLI;
- experiment, prompt, RAG, evaluation and trace-inspection interfaces;
- APIs for training jobs, registry operations, deployments, batch scoring and inference;
- reference applications, templates, documentation, sample datasets and test harnesses;
- role-specific views for product owners, validators, SRE, security, audit and FinOps.

Design rules. Use one identity plane, stable APIs and opinionated templates. Interfaces may vary, but they must create the same underlying resource definitions and evidence. A GUI-only action that cannot be reproduced through an API or declarative record is an audit and automation gap.

Enterprise control and management plane

Purpose. Maintain desired state, enforce enterprise policy and provide traceability across all environments.

Core services.

- **Identity and tenancy:** SSO, federation, groups, workload identities, RBAC/ABAC, project/namespace/account provisioning, quotas and delegated administration.
- **Policy decision and enforcement:** admission policy, data-use policy, release policy, provider routing, network/egress policy, agent-tool policy and exceptions.
- **Catalog and metadata:** AI-system inventory, data catalog, ownership, classification, schemas, lineage, glossary and lifecycle state.

- **Registries:** datasets, features, labels, models, adapters, tokenizers, prompts, evaluation sets, policies, containers, deployment bundles and software/model bills of materials.
- **Orchestration:** data, training, evaluation, deployment, retraining, index refresh, recertification and retirement workflows.
- **Release management:** source control, CI, artifact build/sign, policy gates, GitOps, progressive delivery, change records and rollback.
- **Governance and evidence:** risk tier, impact assessment, review decisions, approvals, control evidence, audit trail, recertification and exceptions.
- **Operations:** telemetry pipeline, SLOs, alerting, incident/problem records, capacity, usage allocation, chargeback/showback and support.

Availability. The control plane should degrade safely. Existing production services should continue for a bounded period if the management UI is unavailable; new deployments and policy-changing actions should fail closed. Protect registry, identity, policy and audit metadata as Tier-0 platform services.

Data, feature and knowledge plane

Purpose. Convert governed source data into reproducible, authorized inputs for training and inference.

Capabilities.

- batch, change-data-capture and streaming ingestion;
- object storage, lakehouse tables, warehouses, relational stores and event logs;
- schema registry, contracts, catalog, lineage, quality, classification, retention and consent;
- label schemas, annotation operations, weak supervision and human-review queues;
- offline and online feature storage with point-in-time-correct retrieval;
- document parsing, deduplication, chunking, metadata enrichment and language detection;
- embedding generation, vector and lexical indexes, knowledge graphs and reranking;
- row/document-level access-control propagation, deletion and freshness management.

Topology. Centralize standards and metadata while federating domain ownership. Highly sensitive or residency-bound data can remain in domain or regional data planes; move compute to data and publish governed interfaces rather than creating uncontrolled copies.

AI workload and runtime plane

Purpose. Execute data processing, training, evaluation and inference with workload-appropriate isolation and acceleration.

Runtime classes.

- CPU batch and distributed data processing;
- classical ML training, cross-validation and hyperparameter search;
- GPU/accelerator training, continued pretraining, SFT, preference optimization, distillation and quantization;
- scheduled and event-driven batch inference;
- low-latency online predictive serving;
- streaming feature computation and inference;
- LLM inference with continuous batching, KV cache, model parallelism and streaming output;
- RAG retrieval/reranking and semantic or prefix caching;
- durable agent workflows and sandboxed tool execution;
- edge packaging and fleet rollout.

Scheduling. Separate interactive, batch, inference and training pools. Use priority classes, fair-share queues, quotas and preemption rules. Do not let experimental training starve production inference or platform control services.

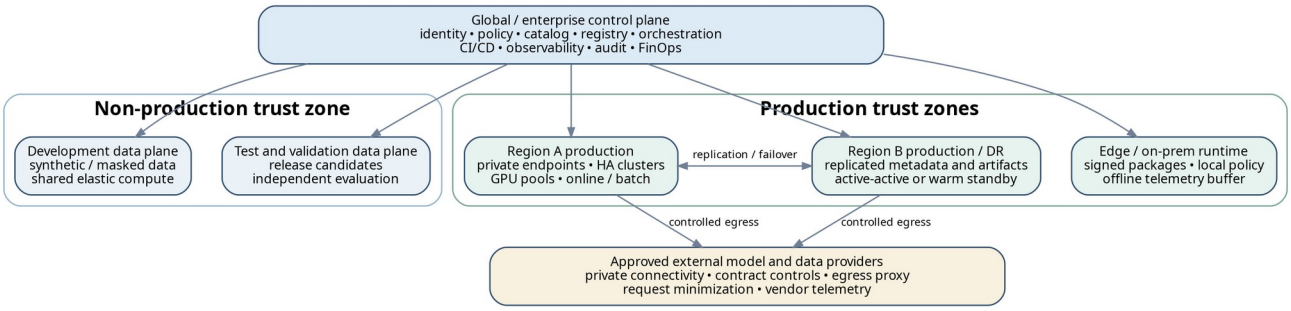
Infrastructure and trust foundation

Purpose. Supply reliable, isolated and observable compute, storage, network and security primitives.

Components.

- Kubernetes, managed training/serving platforms, VMs, serverless and edge runtimes;
- CPU, GPU, TPU/NPU or other accelerators; device plugins and topology-aware scheduling;
- object, block, file, relational, cache and vector storage;
- private networking, service mesh where justified, load balancing, DNS, API gateways and egress proxies;
- key management, secret management, certificate automation and workload identity;
- image/artifact registries, SBOM/model-BOM, signatures, provenance and vulnerability scanning;
- backup, replication, immutable audit storage and disaster recovery;
- infrastructure as code, configuration policy, patching and fleet lifecycle.

4.2 Control-plane and data-plane topology



Recommended enterprise topology: centralized policy and metadata with isolated regional and environment data planes.

A scalable topology uses three separation axes:

1. **Environment:** development, validation/staging and production have distinct credentials, data access and policy. Promotion crosses a controlled interface; direct lateral access is denied.
2. **Region or jurisdiction:** data, inference and logs remain in approved locations. Global metadata contains only what policy permits.
3. **Domain or risk:** high-sensitivity workloads may use dedicated accounts, clusters, keys, network zones or hardware; lower-risk workloads share capacity under logical isolation.

Recommended control-plane deployment:

- run enterprise services in a dedicated management boundary with no customer workload execution;
- replicate critical metadata databases and object stores across failure domains;
- expose private APIs to data planes using workload identity and mTLS;
- keep policy decision points highly available and cache signed policy bundles for bounded disconnected operation;
- write append-only audit events to a separate security account and retention domain;
- use outbound gateways for approved external model providers, package repositories and telemetry sinks;
- prevent production clusters from downloading arbitrary dependencies or model weights.

4.3 Multi-tenancy patterns

Pattern	Isolation	Cost efficiency	Operational burden	Use when
Shared cluster, namespace/project isolation	logical	high	low to moderate	trusted internal teams, low/moderate risk, mature policy controls
Shared control plane, dedicated node pools	workload and compute	medium-high	moderate	GPU isolation, predictable performance, differentiated patch windows

Pattern	Isolation	Cost efficiency	Operational burden	Use when
Dedicated cluster/account per domain	strong	medium-low	high	regulated domains, residency, independent release cadence, blast-radius limits
Dedicated stack per use case	strongest	low	very high	exceptional criticality or legal separation; avoid as a default
Managed external service	provider-defined	variable	low internal, higher vendor dependency	rapid delivery where contracts, data flow and exit strategy are acceptable

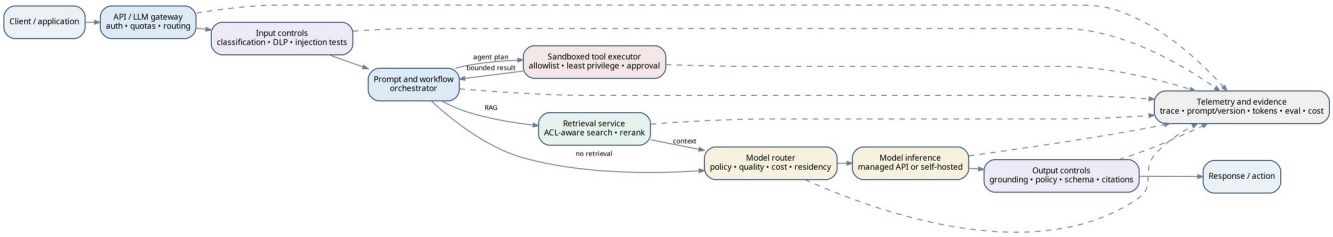
Isolation must cover more than Kubernetes namespaces. Assess identity, keys, network, storage, caches, logs, vector indexes, GPUs, queues, metadata, backup, support access and billing. Shared GPU memory, prompt caches or trace stores can become cross-tenant leakage paths.

4.4 Reference data flows

Classical online prediction

1. Client authenticates through API gateway.
2. Request schema and business authorization are validated.
3. Feature service retrieves point-in-time-consistent online features using entity keys and freshness limits.
4. Model server performs deterministic preprocessing, prediction, calibration and response formatting.
5. Prediction ID links request features, model release, outcome and later feedback.
6. Telemetry records latency, errors, feature freshness, prediction distribution and cost without retaining prohibited data.
7. Delayed outcomes join to prediction IDs for performance and drift monitoring.

LLM, RAG and agent request



Controlled LLM/RAG/agent request path. Every branch emits a trace and passes through deterministic policy points.

Key controls:

- classify request and data before provider routing;
- enforce tenant quotas, context limits and permitted models;
- retrieve only sources the current identity may access;
- treat retrieved content and tool output as untrusted data, not instructions;
- use structured tool schemas and deterministic authorization outside the model;
- validate output schema, policy, citations and action scope;
- require approval for high-impact side effects;
- capture release, prompt, retrieval, tool, model, token, latency, safety and cost evidence in one trace.

4.5 Architecture decision records

Create a versioned architecture decision record for choices that affect portability, risk or operating cost. At minimum document:

- context and constraints;

- considered options, including “do nothing” and simpler non-AI approaches;
- decision criteria and measured evidence;
- selected option and consequences;
- security, privacy, regulatory and residency implications;
- estimated three-year total cost and exit cost;
- migration or rollback approach;
- review trigger and decision owner.

Mandatory ADR topics typically include model source, managed versus self-hosted inference, Kubernetes versus managed runtime, vector store, feature store, multi-region strategy, tenant isolation, telemetry retention, external provider routing and agent autonomy.

4.6 Reference technology map

Capability	Open or portable examples	Managed categories
Source, CI and artifact build	Git, GitHub/GitLab, Jenkins, Tekton, BuildKit	managed source and pipeline services
GitOps and deployment	Argo CD, Flux, Helm, Kustomize	managed deployment services
Workflow orchestration	Airflow, Dagster, Argo Workflows, Kubeflow Pipelines, Temporal	managed workflow and ML pipeline services
Data processing	Spark, Flink, Ray, dbt	managed lakehouse, warehouse and stream processing
Lakehouse/table format	Apache Iceberg, Delta Lake, Apache Hudi	managed lakehouse services
Catalog and lineage	OpenLineage/Marquez, DataHub, OpenMetadata	cloud data catalogs and governance suites
Data quality	Great Expectations, Deequ, Soda	managed data observability
Feature management	Feast, Hopsworks	managed feature stores
Experiment tracking/registry	MLflow, Weights & Biases, Comet	managed ML lifecycle services
Training	scikit-learn, XGBoost, LightGBM, PyTorch, TensorFlow, Ray Train	managed training and accelerator platforms
LLM tuning	Transformers, PEFT, TRL, PyTorch FSDP, DeepSpeed	managed fine-tuning services
Predictive serving	KServe, Seldon, BentoML, Ray Serve, Triton	managed model endpoints
LLM serving	vLLM, Hugging Face TGI, Triton, llama.cpp	managed foundation-model APIs/endpoints
Vector/search	pgvector, Milvus, OpenSearch/Elasticsearch, Weaviate	managed vector databases and search
Policy	Open Policy Agent, Gatekeeper, Kyverno	cloud policy and GRC services
Telemetry	OpenTelemetry, Prometheus, Grafana, Loki, Tempo, OpenSearch	managed observability and model-monitoring services
Secrets/identity	Vault, SPIFFE/SPIRE, cert-manager	cloud KMS, secrets and workload identity
Supply chain	SLSA, Sigstore/Cosign, in-toto, SPDX, CycloneDX	managed signing and artifact-security services

Select products only after defining contracts and operating requirements. A reference stack is not an architecture until ownership, failure behavior, data flow, SLOs and lifecycle are specified.

5. Platform product and operating model

5.1 Platform mission

The platform team's product is **safe, reliable delivery speed**. Its users are data scientists, ML/LLM engineers, application teams, data teams, validators, operators and governance functions. Treating the platform as an infrastructure project produces low adoption and duplicated shadow stacks.

The platform product manager owns:

- customer research and workload segmentation;
- service catalog, roadmap and deprecation policy;
- adoption, lead time, reliability and unit-cost outcomes;
- prioritization across common capabilities and domain exceptions;
- documentation, education and feedback loops;
- vendor portfolio and total-cost transparency.

5.2 Team topology

Enterprise AI platform team

Owns shared APIs, templates, control-plane services, paved roads, integration, platform SLOs, upgrades and support. Typical skills: platform engineering, MLOps/LLMOps, data infrastructure, SRE, security engineering and developer experience.

Domain AI product teams

Own use-case outcomes, domain logic, model/system behavior, application integration, business process, operational acceptance and first-line diagnosis. They select from approved platform services and contribute reusable components.

Data product teams

Own source semantics, contracts, quality, lineage, access, retention and change communication. They do not merely "provide tables"; they provide dependable, versioned data products.

Independent assurance functions

Security, privacy, legal, compliance, responsible AI and model risk define policies, review high-risk cases, independently validate where required and audit control effectiveness. They should expose machine-readable policies and reusable test packs, not only ticket queues.

SRE and operations

Define reliability standards, service-tier requirements, incident/change/problem processes, on-call readiness, capacity and disaster-recovery tests. Ownership may be embedded or shared, but each production service has one accountable operational owner.

FinOps and procurement

Provide cost allocation, forecasting, commitment strategy, vendor terms, rate-card governance and value measurement. They work with engineering before architecture is locked.

5.3 Decision rights

Decision	Accountable role	Required consultation
Business objective and value threshold	AI product owner	finance, operations, affected business owners
Data-use authorization	data owner	privacy, security, legal, domain steward
Risk tier and applicable controls	AI risk owner / governance	product, security, privacy, legal, model risk
Model and architecture choice	service technical owner	platform, data, security, SRE, FinOps
Evaluation policy and release thresholds	product owner plus independent validator for high risk	domain experts, governance, SRE
Production release	service owner; separate approver for high risk	validator, change authority, SRE
Incident severity and containment	incident commander	service owner, security/privacy/legal as applicable
Exception acceptance	designated risk executive	control owner, service owner, legal/compliance
Retirement	product owner and data/model owner	records, legal, platform, operations

5.4 Service catalog

Offer capabilities as products with explicit contracts. A useful initial catalog:

- governed AI workspace;
- dataset/data-product onboarding;
- feature definition and online feature service;
- document/knowledge ingestion and retrieval index;
- managed experiment tracking;
- CPU/GPU training jobs and distributed tuning;
- evaluation and red-team service;
- model/prompt/system registry;
- scheduled batch inference;
- online predictive endpoint;
- LLM gateway and managed/self-hosted model access;
- RAG runtime;
- bounded agent workflow runtime;
- observability, quality monitoring and cost dashboards;
- edge model packaging and rollout;
- compliance evidence export and recertification workflow.

Every catalog item specifies:

- intended use and prohibited use;
- API or declarative resource schema;
- ownership and support channel;
- availability, durability, latency and support SLO;
- security boundary, data classes and residency;
- quotas, rate limits, pricing/showback and default retention;
- version policy, maintenance windows and deprecation notice;
- evidence generated automatically;
- customer responsibilities and escalation path.

5.5 Federated governance model

Use a hub-and-spoke model:

- the enterprise hub sets minimum controls, risk taxonomy, shared policy, registry schema, audit evidence and approved services;
- domain spokes own local data semantics, domain evaluation, business outcomes and additional controls;
- automated conformance checks prevent fragmentation;
- exceptions are visible centrally and expire;
- reusable domain patterns are promoted into enterprise golden paths.

A central team should not become the throughput bottleneck for every model. A fully decentralized model should not permit inconsistent risk classification, provider contracts, telemetry or security. Federation separates **standard setting** from **workload delivery** while preserving evidence.

5.6 Support and operational engagement

Use a tiered support model:

- **L0 self-service:** docs, templates, diagnostics, status pages and known-error database.
- **L1 platform support:** access, quota, standard pipeline and service issues.
- **L2 specialist:** data, training, serving, evaluation, security or accelerator expertise.
- **L3 engineering/vendor:** defects, performance engineering and product fixes.

Production readiness requires a service onboarding review, named on-call, dashboards, alerts, runbook, dependency map, rollback test, data-quality response, security contacts and business communication plan.

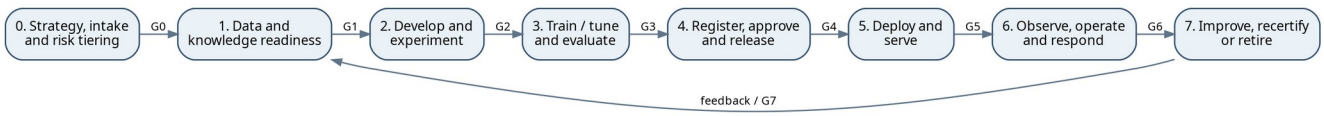
5.7 Platform KPIs

Measure the platform as a value-delivery system:

- median and p90 lead time from approved use case to first production value;
- environment provisioning time and pipeline success rate;
- percentage of production services using golden paths;
- deployment frequency and change failure rate;
- mean time to detect and recover;
- evaluation suite duration and false-block rate;
- percentage of releases with complete lineage, signed provenance and current approval;
- platform availability and customer workload SLO attainment;
- active users, retained teams, self-service completion and support ticket deflection;
- cost per successful training run, million predictions, million tokens and successful task;
- duplicate-tool retirement and avoided spend;
- number and age of exceptions, vulnerabilities and overdue recertifications.

Avoid vanity metrics such as number of notebooks, models registered or GPUs installed. They measure inventory, not value or control.

6. Lifecycle gates and artifact system



End-to-end lifecycle with explicit gates. A failed gate returns work to the owning stage; it does not create an undocumented waiver.

6.1 Gate model

Gate	Decision	Minimum evidence	Primary approver
G0 — Intake	Is AI justified and permissible?	business case, baseline, owner, risk tier, initial data/provider assessment	product sponsor and risk owner
G1 — Data readiness	Are data and knowledge fit and authorized?	contracts, lineage, quality baseline, access, privacy and retention decisions	data owner; privacy/security as required
G2 — Experiment readiness	Can work proceed reproducibly and safely?	approved workspace, source repo, environment lock, experiment plan, evaluation plan	technical owner
G3 — Validation	Does the exact release meet outcome and risk thresholds?	evaluation report, segment results, security tests, model/system card, residual-risk statement	product owner; independent validator for high risk
G4 — Release	Is the bundle deployable and supportable?	signed artifacts, provenance, deployment manifest, SLOs, runbook, rollback, approvals	service owner/change authority
G5 — Launch	Is production behavior acceptable under controlled exposure?	canary/shadow evidence, smoke tests, telemetry, capacity and cost checks	service owner and SRE
G6 — Operate/recertify	Does the service remain within approved bounds?	monitoring, incidents, drift, value, access review, provider/version review	service owner and risk owner
G7 — Improve or retire	Should the service change, continue or stop?	retraining trigger, updated business case, archival/deletion plan	product owner and data/model owner

Gates are policy objects. Their required tests vary by risk and service tier. Automate evidence collection and approval routing; do not automate accountability.

6.2 Artifact identity model

Assign a globally unique, immutable **release ID** to each candidate. The release manifest should reference digests for:

- source commit and build recipe;
- container and dependency lock;
- dataset snapshot and label set;
- feature definitions or document corpus/index version;
- model weights, adapter, tokenizer and calibration artifacts;
- prompt templates, examples, output schemas and guardrail policies;
- evaluation datasets, harness, thresholds and report;
- infrastructure and deployment configuration;
- tool schemas, permissions and workflow definition;
- SBOM/model-BOM, signatures and provenance;
- owners, risk tier, approvals, expiration and rollback target.

A release manifest is the system's bill of behavior. It enables replay, comparison, incident reconstruction and rollback.

6.3 Required artifact inventory

Business and governance

- use-case record and business case;
- intended use, affected populations and prohibited use;

- risk classification and impact assessment;
- legal/privacy/security review and data-flow record;
- ownership, human-oversight design and escalation path;
- vendor/model due diligence;
- approval, exception and recertification records.

Data and knowledge

- source registration, owner and contract;
- schema, semantic definitions and classification;
- lineage graph and processing code;
- dataset snapshot, data statement and quality report;
- label taxonomy, instructions, annotator quality and adjudication record;
- feature definitions and point-in-time validation;
- document manifest, ACL metadata, chunking/embedding configuration and index version;
- consent, retention, deletion and licensing evidence.

Engineering and model

- source commit, tests and environment lock;
- experiment runs, parameters, metrics and artifacts;
- model weights, adapter, tokenizer, calibration and export format;
- prompt, tool, workflow and policy versions;
- performance profile, hardware target and dependency manifest;
- model card and system card;
- SBOM/model-BOM, vulnerability scan, signatures and provenance.

Evaluation and operations

- evaluation plan, datasets, rubrics and test harness;
- validation report, confidence intervals and segment results;
- red-team/security report and remediation;
- deployment manifest and change record;
- SLOs, dashboards, alerts, runbook and rollback plan;
- capacity and cost forecast;
- production traces, feedback, incident and problem records;
- recertification, archival and destruction evidence.

6.4 Artifact retention

Retention is policy-driven. Keep enough to reproduce decisions and investigate incidents while minimizing sensitive data. Separate:

- **long-lived evidence:** release manifests, approvals, digests, evaluation summaries, cards, incidents and audit records;
- **reproducibility artifacts:** code, environment locks, model packages, dataset references and transformation logic;
- **sensitive operational telemetry:** prompts, features, retrieved text and outputs, retained only when justified and redacted/tokenized where possible;
- **ephemeral computation:** caches, intermediate checkpoints and scratch data, automatically expired unless promoted.

A deletion request must traverse lineage from source to derived datasets, embeddings, indexes, caches, traces and backups. Where immediate deletion from immutable backup is infeasible, document cryptographic erasure or bounded expiry and prevent restoration into active systems without reapplying deletion records.

7. Stage 0 — Strategy, portfolio intake and risk tiering

7.1 Purpose and objectives

Convert an idea into an approved, bounded problem with accountable ownership, measurable value and known control requirements. The stage prevents three expensive mistakes: applying AI where deterministic automation is sufficient, beginning before data and process owners are committed, and discovering regulatory or operational blockers after model investment.

Objectives:

- define the decision or workflow being improved and the counterfactual baseline;
- specify measurable benefit, guardrails, affected users and time to value;
- choose the least-complex solution class capable of meeting the target;
- classify impact, autonomy, data sensitivity, criticality and regulatory scope;
- establish budget, owner, delivery team, data sources and evaluation plan;
- decide whether to proceed, redesign, defer, buy or stop.

7.2 Core concepts and design principles

- **Problem before model.** Express the use case as an operational decision, user job or process outcome.
- **Baseline before benchmark.** Compare against the current process, rules, search or human workflow, not only against other models.
- **Value hypothesis with falsification.** Define what evidence would show the initiative should stop.
- **Risk is contextual.** The same model may be low risk for drafting and high risk for eligibility decisions.
- **Autonomy is an architecture choice.** A model's ability to propose an action does not grant authority to execute it.
- **Portfolio sequencing matters.** Favor use cases that build reusable data, evaluation and platform capabilities while delivering value.

7.3 Fine-grained platform capabilities

- intake portal with standardized use-case schema and workflow;
- enterprise AI-system inventory, ownership and lifecycle state;
- risk-tiering questionnaire with policy-driven control profile;
- value model covering benefit, implementation cost, operating cost and change adoption;
- model/provider catalog with approved uses, data restrictions and rate cards;
- architecture pattern selector: rules, classical ML, SLM, LLM, RAG, agent or hybrid;
- data-source discovery, owner lookup and preliminary quality/access assessment;
- regulatory and contractual applicability checklist;
- dependency and integration inventory;
- portfolio scoring, capacity reservation and budget approval;
- decision record, exception workflow and expiration;
- initial evaluation-plan template and acceptance criteria.

7.4 Inputs, outputs and artifacts

Inputs: business problem, current process metrics, user research, candidate data, expected volume, latency and criticality, existing controls, vendor proposals and strategic constraints.

Outputs: approved or rejected use-case record, accountable product owner, technical owner and risk owner; workload/risk/service tier; target KPI and guardrails; initial architecture; data and provider list; budget envelope; evaluation plan; delivery milestone; decision log.

Required artifacts:

- problem statement and intended/prohibited use;
- baseline process map and quantified baseline;
- benefit hypothesis and value-realization method;
- affected-population and harm analysis;
- preliminary data-flow diagram and data classifications;
- risk-tier result with rationale;
- build/buy/model-source decision record;
- human-oversight and recourse concept;
- initial success, safety, reliability and cost thresholds;
- G0 approval or rejection.

7.5 System architecture and reference workflow

1. Product owner submits an intake record.
2. The platform enriches it with data ownership, existing reusable assets, approved providers and similar systems.
3. A cross-functional triage performs problem decomposition and selects candidate solution classes.
4. A small feasibility test uses synthetic, public or already-authorized data; it must not become an ungoverned production prototype.
5. Risk engine maps the use case to a control profile and required reviewers.
6. Finance and platform teams estimate full-lifecycle cost, including evaluation, human review, inference, support and exit.
7. Decision authority approves, requests redesign or stops the initiative.
8. Approved records create a project, repository, workspace, budget/quota and lifecycle workflow.

7.6 Technologies, frameworks and tooling

- service-management or product-portfolio workflow platform;
- enterprise architecture repository and data catalog;
- GRC workflow and policy engine;
- cost estimator using internal rate cards and provider prices;
- model/provider catalog and contract metadata;
- issue tracker and architecture decision record repository;
- dashboards for portfolio value, risk, stage aging and capacity.

Do not begin with a specialized AI-governance product if the enterprise cannot define owners, tiers and decisions. A simple, integrated workflow with reliable evidence is superior to a complex inventory that teams bypass.

7.7 Roles and responsibilities

- **Executive sponsor:** strategic alignment, funding and risk appetite.
- **AI product owner:** problem, value, adoption, business process and outcome measurement.
- **Domain expert:** valid operating constraints and harm scenarios.
- **Technical owner/architect:** solution options, integration and feasibility.
- **Data owner/steward:** source authorization, semantics and quality expectations.
- **AI risk/governance:** tiering and control profile.
- **Security/privacy/legal/compliance:** data flow, provider and regulatory assessment.
- **Platform/SRE:** platform fit, nonfunctional requirements and capacity.
- **FinOps/procurement:** TCO, contract and commitment exposure.
- **Independent validator:** early challenge for high-impact systems.

7.8 Decision points and trade-offs

Rules versus ML versus language model

- Choose rules when logic is stable, explicit, auditable and cheap to maintain.
- Choose classical ML when structured signals predict a bounded target and calibrated probabilities or ranking are valuable.
- Choose an SLM when a narrow language task can be met with lower latency, privacy exposure and cost.
- Choose an LLM when broad language capability, few-shot adaptability or complex generation justifies non-determinism and cost.
- Add RAG when authoritative, changing or private knowledge must ground outputs.
- Add agentic action only when dynamic tool selection creates material value beyond a deterministic workflow.

Build versus buy

Buy when the capability is commodity, provider controls are acceptable and speed outweighs portability. Build when data or workflow differentiation is strategic, latency/residency requires control, volume creates favorable economics, or provider behavior cannot meet assurance requirements. Preserve an exit path for any critical managed service.

Human review

Human review reduces some risks but adds delay, cost and automation bias. Specify what the reviewer sees, what they may change, required competence, sampling/coverage and escalation. “A human is involved” is not a control design.

7.9 KPIs and operational metrics

- percentage of use cases with quantified baseline and named outcome owner;
- intake-to-decision lead time, segmented by risk tier;
- percentage rejected or redesigned before engineering spend;
- forecast versus realized value and total cost;
- portfolio concentration by provider, domain, model and risk tier;
- percentage with complete data ownership and evaluation plan at G0;
- stage aging and blocked-work reasons;
- reuse rate of platform patterns, datasets and evaluation assets.

7.10 Security, governance, compliance and risk controls

- screen prohibited or unacceptable uses before experimentation;
- record affected populations, rights, recourse and human oversight;
- prohibit production or sensitive data in feasibility work without G1 authorization;
- assess external-provider data retention, training use, location, subprocessors and security terms;
- require legal/license review for model weights and training data;
- enforce conflict-of-interest and separation-of-duty rules for high risk;
- assign review cadence and expiry to the use-case approval;
- make the inventory authoritative: unregistered production AI is a policy violation.

7.11 Common failure modes and troubleshooting

Failure mode	Diagnostic	Corrective action
“Use AI” is the objective	no measurable decision or workflow outcome	rewrite as process metric and compare non-AI options
Prototype becomes production	no owner, release ID or support model	stop exposure, register system, perform retrospective gates

Failure mode	Diagnostic	Corrective action
Benefits are double-counted	value not reconciled to finance/process baseline	define one owner, counterfactual and measurement window
Risk discovered late	data flow/provider/autonomy omitted at intake	expand intake schema and make missing fields blocking
Portfolio overload	many pilots, no production capacity	apply WIP limits and fund platforms plus outcome teams
Human review is nominal	high override latency or rubber-stamping	redesign interface, sampling, training and escalation

7.12 Scalability, reliability, performance and cost optimization

- automate low-risk triage using deterministic policy, not model-generated approval;
- use reusable control profiles to avoid reviewing every use case from zero;
- cap work in progress by platform capacity and change-management bandwidth;
- estimate steady-state inference, evaluation, human review and support cost—not only build cost;
- identify demand shape early: peak RPS, batch windows, token distribution, concurrency and geographic load;
- include provider outage and rate-limit behavior in the architecture decision;
- reserve scarce accelerators only after feasibility and risk approval.

7.13 Production best practices and anti-patterns

Best practices: set stop criteria; use a small, representative proof; fund adoption and process change; include operational and control teams at intake; treat reusable assets as portfolio value.

Anti-patterns: executive solution mandates without problem definition; model demos as business cases; pilots using data that cannot be used in production; ignoring human workflow; assuming a managed API transfers accountability; using one risk checklist for all systems.

7.14 Exit criteria

G0 passes only when the use case has a named owner, measurable baseline and target, accepted risk tier, preliminary data/provider authorization, selected architecture class, budget, evaluation plan and explicit decision authority. Otherwise stop or return for redesign.

8. Stage 1 — Data source onboarding and ingestion

8.1 Purpose and objectives

Establish authorized, reliable and observable movement of source data into governed analytical, training, retrieval and inference zones. The goal is not merely transport; it is preserving semantics, identity, lineage, quality, ordering, privacy and replayability.

Objectives:

- register source, owner, contract, classification and lawful/approved use;
- select batch, CDC, event-stream or request-time access based on freshness and consistency needs;
- validate schema and quality at boundaries;
- create replayable, idempotent and monitored ingestion;
- quarantine invalid data without silently losing it;
- protect credentials and sensitive fields;
- produce a versioned raw/landing dataset and lineage events.

8.2 Core concepts and design principles

- **Data contract:** explicit schema, semantics, quality, freshness, ownership, compatibility and change-notification agreement.
- **Immutable landing:** preserve source truth and ingestion metadata before transformations, subject to privacy and retention policy.
- **At-least-once is normal.** Design idempotent processing and deduplication rather than assuming exactly-once across heterogeneous systems.
- **Event time differs from processing time.** Capture both and define late-arrival behavior.
- **Schema compatibility is a policy.** Backward, forward or full compatibility must be selected per consumer and enforced.
- **Quarantine over coercion.** Invalid values should not be silently cast, dropped or defaulted.
- **Replay is a product capability.** Retain sufficient source history, offsets and code versions to rebuild derived state.

8.3 Fine-grained platform capabilities

- source registration and owner approval;
- connectors for databases, SaaS, files, object stores, message buses, APIs, devices and external datasets;
- CDC with snapshot/bootstrap and log-position management;
- batch transfer with manifests, checksums and atomic publication;
- event streaming with schema registry, partitions, ordering keys and dead-letter handling;
- API ingestion with rate limiting, retry, idempotency keys and backoff;
- file malware scanning, format validation and content-type verification;
- classification, DLP, tokenization/redaction and field-level encryption;
- schema and contract validation at producer and consumer boundaries;
- data-quality checks, reconciliation counts and source-to-target controls;
- lineage emission, source timestamps and ingestion metadata;
- quarantine, replay, backfill and correction workflows;
- freshness, lag, throughput and failure telemetry;
- retention, legal hold and deletion integration.

8.4 Inputs, outputs and artifacts

Inputs: approved data-source record, credentials/identity, source schema, semantics, volume and change rate, classification, residency, retention, freshness target, historical backfill requirements and consumer use cases.

Outputs: versioned raw dataset or governed stream, ingestion metadata, schema registry entry, quality baseline, lineage, quarantine queue, dashboards and operating runbook.

Artifacts:

- data contract and compatibility policy;
- source-to-target mapping and data-flow diagram;
- connector configuration as code;
- credential and network-access policy;
- schema and classification record;
- validation and reconciliation rules;
- retention/deletion schedule;
- backfill/replay plan and checkpoint state;
- SLO, dashboard, alert rules and owner;
- G1 ingestion evidence.

8.5 System architecture and reference workflow

A reference ingestion path:

1. Source is registered in the catalog with owner, classification and approved purposes.
2. Workload identity obtains least-privilege source access; static shared credentials are prohibited.
3. Connector writes to an isolated landing zone or governed stream using encryption and checksums.
4. Boundary validation checks schema, file integrity, malware, expected volume and critical quality rules.
5. Valid data is atomically committed to a versioned table or topic; invalid data is quarantined with reason and source pointer.
6. Catalog and lineage services receive dataset version, job/run and source metadata.
7. Reconciliation compares source counts, totals or control fields to landed output.
8. Monitoring enforces freshness, lag, error-rate and quarantine thresholds.
9. Downstream publication occurs only after contract checks pass.

For streaming, separate durable ingestion from downstream feature/model logic. Consumer groups should checkpoint offsets and support replay into versioned sinks. For batch, use manifests and marker files or transactional table commits so consumers never read partial output.

8.6 Technologies, frameworks and tooling

- event buses and logs: Apache Kafka, Pulsar or managed equivalents;
- stream processing: Flink, Spark Structured Streaming, Kafka Streams or managed services;
- CDC: Debezium and managed database replication services;
- batch/orchestration: Airflow, Dagster, Argo Workflows, managed schedulers;
- table formats: Iceberg, Delta Lake or Hudi for snapshots, schema evolution and time travel;
- schema registries using Avro, Protobuf or JSON Schema;
- validation: Great Expectations, Deequ, Soda or custom domain checks;
- catalog/lineage: OpenLineage-compatible orchestration, DataHub, OpenMetadata or managed catalog;
- secret and identity services: Vault, cloud secret managers, workload identity;
- DLP and tokenization services appropriate to data classes.

8.7 Roles and responsibilities

- **Source/data owner:** authorizes access, semantics, change policy and quality commitments.
- **Data engineer:** connector, schema, replay, reconciliation and operational reliability.
- **Data steward:** classification, glossary and quality rules.
- **Security engineer:** identity, network, encryption and credential controls.
- **Privacy/legal:** purpose, consent, retention, cross-border and deletion requirements.
- **Platform team:** connector framework, orchestration, catalog integration and observability.
- **Consumer/model owner:** declares freshness, fields, historical window and tolerance for late/missing data.
- **SRE:** SLOs, alerting, capacity and incident readiness for critical pipelines.

8.8 Decision points and trade-offs

Batch versus CDC versus streaming

- Batch is simpler and cost-effective when freshness permits.
- CDC provides near-real-time database changes but inherits source log semantics, schema evolution and delete handling.
- Event streaming is appropriate when producers can publish domain events and consumers need low latency or independent replay.
- Request-time federation avoids copies but couples inference reliability and latency to source systems; use sparingly and cache safely.

Centralized landing versus domain-local processing

Central landing improves reuse and governance but may violate residency or create bottlenecks. Domain-local landing with federated catalog and policy reduces movement but requires consistent standards. Choose based on legal boundary, data gravity, network cost and operating maturity.

Raw retention

Long raw retention improves replay and investigation but increases exposure and cost. Retain only what is authorized; tokenize or encrypt sensitive fields; use short-lived landing when source can reliably replay.

8.9 KPIs and operational metrics

- ingestion freshness and end-to-end lag by source;
- successful records/bytes/events per interval;
- pipeline success rate and retry volume;
- source-to-target reconciliation variance;
- schema-contract pass/fail and incompatible-change count;
- quarantine rate, age and disposition;
- duplicate, late and out-of-order event rates;
- backfill duration and replay success;
- lineage coverage and catalog completeness;
- cost per GB/event/source and network egress;
- mean time to detect and resolve source incidents.

8.10 Security, governance, compliance and risk controls

- least-privilege workload identity and private connectivity;
- field and dataset classification before broad access;
- encryption in transit and at rest with managed key rotation;
- DLP/malware/content validation at ingress;
- purpose-based access and data-use policy tied to the use-case record;
- residency and cross-border enforcement;
- immutable audit of access, schema and configuration changes;
- retention, legal hold and deletion workflows;
- contract/license verification for external data;
- separation of production and development data; masked/synthetic subsets for lower environments;
- approval before new fields or sources become model inputs.

8.11 Common failure modes and troubleshooting

Symptom	Likely causes	Diagnosis and response
Sudden freshness breach	source outage, credential expiry, partition stall, quota	inspect source health, offsets, auth and backpressure; fail over or pause downstream publication
Count mismatch	dropped files, duplicate retries, filter change, late events	reconcile by partition/control totals; replay from last verified checkpoint
Schema break	producer deployed incompatible change	quarantine new version, notify owner, use compatibility adapter or coordinated rollback
Silent null/default growth	coercive parser or upstream semantic change	compare field distributions and parser logs; remove silent defaults and restore explicit validation
Duplicate events	at-least-once retry without idempotency	deduplicate on stable event ID/version; fix sink transaction/idempotency logic

Symptom	Likely causes	Diagnosis and response
PII appears unexpectedly	source added field or classifier missed content	contain access, classify, purge unauthorized copies, update contract and detection rules
Replay corrupts downstream state	non-idempotent transforms or changed code	replay into isolated versioned target; compare, then atomically promote

8.12 Scalability, reliability, performance and cost optimization

- partition by stable, high-cardinality keys that preserve required ordering;
- size consumers for peak plus replay demand, not average only;
- apply backpressure and bounded queues; never solve overload with unbounded memory;
- separate hot ingestion from expensive enrichment;
- compact small files and tune table partitioning to query patterns;
- use incremental loads and predicate pushdown;
- compress in transit/storage and avoid repeated cross-region copies;
- autoscale stateless workers, but protect source systems with rate limits;
- maintain capacity for urgent backfill without violating production SLOs;
- test connector failover, checkpoint restore and schema rollback.

8.13 Production best practices and anti-patterns

Best practices: contracts owned by producers and consumers; immutable raw/versioned tables; explicit event IDs and timestamps; reconciliation; quarantine with operational ownership; replay drills; lineage emitted at runtime.

Anti-patterns: copying production databases nightly without owner or contract; CSV as an unversioned interface; shared static credentials; dropping malformed rows; overwriting partitions in place; assuming topic ordering globally; granting broad raw-zone access to notebooks.

8.14 Exit criteria

G1 ingestion readiness requires approved data use, operational owner, schema/contract, versioned landing or stream, quality and reconciliation baseline, lineage, security controls, retention/deletion policy, replay plan and SLO telemetry.

9. Stage 2 — Data, feature and knowledge engineering

9.1 Purpose and objectives

Transform governed source data into reproducible training/evaluation datasets, production features and retrieval-ready knowledge. This stage creates the semantic contract between raw data and model behavior.

Objectives:

- clean and transform data without losing lineage or point-in-time correctness;
- create representative labels and evaluation sets;
- maintain offline/online feature parity where online inference is required;
- build authorized, fresh and injection-resistant knowledge indexes for RAG;
- quantify quality, bias, coverage, duplication and contamination;
- version every material data and knowledge artifact;
- provide reusable domain assets rather than one-off pipeline outputs.

9.2 Core concepts and design principles

- **Point-in-time correctness:** training examples may use only information available at prediction time.
- **Training-serving parity:** the same feature definition and transformation logic should drive offline and online values.
- **Data quality is contextual:** completeness may be acceptable globally yet fail for a protected or high-value segment.
- **Labels are measurements, not truth.** Record guidelines, disagreement, uncertainty and provenance.
- **Knowledge retrieval is authorization-sensitive.** Similarity is not permission.
- **Embedding indexes are derived data.** They inherit source classification, retention, deletion and residency.
- **Evaluate retrieval separately from generation.** A generator cannot recover evidence that retrieval omitted.

9.3 Fine-grained platform capabilities

Dataset engineering

- declarative transformations with unit, integration and data tests;
- snapshot/time-travel datasets and reproducible sampling;
- entity resolution, deduplication and leakage checks;
- missingness, outlier and distribution analysis;
- point-in-time joins and temporal validation;
- train/validation/test split service by entity/time/group;
- dataset documentation, licensing and representativeness profiles;
- PII minimization, tokenization, de-identification and deletion propagation.

Labeling

- label taxonomy and annotation guideline versioning;
- task assignment, workforce qualification and access control;
- double annotation, gold questions and adjudication;
- active learning, uncertainty sampling and weak supervision;
- annotator agreement, per-class quality and bias analysis;
- secure handling of sensitive content and reviewer wellbeing controls;
- label lineage back to source and guideline version.

Feature engineering

- centralized feature definitions and ownership;
- offline historical retrieval with point-in-time joins;
- materialization to online stores with freshness SLOs;
- streaming feature computation and late-event policies;
- feature validation, statistics and drift baselines;
- entity-key, TTL and default-value contracts;
- feature reuse, discovery and deprecation;
- consistency checks between offline and online values.

Knowledge and retrieval engineering

- document connectors, parsing, OCR when necessary, language detection and structural extraction;
- duplicate and near-duplicate detection;
- chunking by document structure and task, with stable source offsets;
- metadata enrichment: owner, dates, jurisdiction, sensitivity, ACL, version and source URI;
- PII/IP/content filtering and malicious-instruction detection;
- embedding generation with model/version tracking;
- lexical and vector indexing, hybrid retrieval, filters and reranking;
- authorization-aware query and post-filtering;
- index freshness, deletion, re-embedding and rollback;

- citation payloads and source-snippet provenance.

9.4 Inputs, outputs and artifacts

Inputs: versioned source datasets/streams, semantic contracts, approved purposes, feature/knowledge requirements, label definitions, evaluation plan, data classes and service freshness targets.

Outputs: training/validation/test datasets, label sets, feature views, online materializations, document/chunk manifests, embedding/index versions, quality reports and lineage.

Artifacts:

- transformation code and environment;
- dataset manifest with snapshot IDs and hashes;
- data statement describing population, collection, exclusions and limitations;
- split logic and leakage report;
- label schema, instructions, agreement and adjudication metrics;
- feature definitions, entity/TTL/default/freshness contracts;
- offline-online consistency report;
- document manifest, parser/chunker/embedding versions and ACL mapping;
- retrieval benchmark and index quality report;
- deletion and refresh runbook;
- G1/G2 data readiness approval.

9.5 System architecture and reference workflow

Feature path

1. Transform raw source into curated, versioned domain tables.
2. Define features in a registry using code or declarative specifications.
3. Generate historical training sets with event-time and point-in-time joins.
4. Validate statistics, leakage, segment coverage and null/default behavior.
5. Materialize approved features to an online store or compute them in streaming paths.
6. Serve features through an authenticated service with freshness and entity validation.
7. Join prediction IDs to feature versions for traceability and delayed outcome analysis.

Knowledge path

1. Connector retrieves only approved sources and captures source version and ACL.
2. Parser produces structural elements; failed or suspicious files enter quarantine.
3. Normalization, deduplication and content policy checks run before chunking.
4. Chunker creates stable chunk IDs tied to source offsets and versions.
5. Embedding service generates vectors with a pinned model and normalization policy.
6. Index stores vector, lexical representation, metadata and authorization attributes.
7. Retrieval service applies identity filters, hybrid search and optional reranking.
8. Index release is evaluated, signed and promoted; previous index remains rollback-ready.
9. Source changes and deletions trigger incremental update or bounded full rebuild.

9.6 Technologies, frameworks and tooling

- SQL/dbt, Spark, Flink, Ray Data and lakehouse engines;
- versioned table formats and object storage;
- Feast, Hopsworks or managed feature stores;
- Label Studio and managed annotation services;
- text/document parsers, layout-aware extraction and content scanners;

- embedding runtimes using Transformers or approved APIs;
- pgvector, Milvus, Weaviate, OpenSearch/Elasticsearch or managed vector services;
- lexical retrieval such as BM25 plus cross-encoder or model-based rerankers;
- data quality/observability and lineage platforms;
- privacy-enhancing tools for tokenization, de-identification or synthetic data where appropriate.

9.7 Roles and responsibilities

- **Data product owner:** semantics, quality and source roadmap.
- **Data/analytics engineer:** transformation and dataset reliability.
- **Data scientist/ML engineer:** feature intent, leakage prevention and representativeness.
- **Knowledge engineer:** document pipeline, metadata, retrieval and index lifecycle.
- **Domain annotator/adjudicator:** label validity and edge cases.
- **Privacy/security:** sensitive content, access propagation and deletion.
- **Platform team:** feature/index services, compute, catalogs and templates.
- **Independent validator:** evaluation-set independence and segment adequacy for high risk.

9.8 Decision points and trade-offs

Feature store or ordinary tables

Use a feature store when multiple models reuse features, online serving needs low-latency materialization, point-in-time joins are complex or governance of definitions is valuable. For a single batch model, versioned tables and tested transformation code may be simpler.

Precompute versus request-time features

Precomputation improves latency and isolates source systems but can be stale and costly. Request-time computation is fresh but adds dependency latency and failure modes. Hybrid designs precompute stable features and compute small request-specific features synchronously.

Chunking strategy

Fixed token chunks are simple but can split meaning. Structure-aware chunks preserve sections but vary in size. Parent-child retrieval improves context while controlling search granularity. Select via retrieval benchmarks, not intuition; retain source offsets and overlap explicitly.

Embedding and vector-store choice

Evaluate domain recall, multilingual behavior, vector dimension, latency, hardware, update rate, licensing and data residency. A larger embedding model is not automatically better. Choose the database based on filtering, consistency, tenancy, backup, reindexing and operational competence—not only nearest-neighbor benchmark speed.

Synthetic data

Synthetic data can expand rare cases and protect some sensitive attributes, but it may reproduce source bias, leak memorized records or create unrealistic distributions. Validate against real held-out data and retain provenance.

9.9 KPIs and operational metrics

Dataset: contract pass rate; missingness/outlier/duplicate rates; label coverage; leakage findings; segment representation; freshness; reproducible build success; data-deletion completion time.

Labels: inter-annotator agreement; adjudication rate; gold-task accuracy; class balance; time and cost per accepted label; reviewer escalation and wellbeing indicators for harmful content.

Features: offline-online skew; materialization lag; stale feature rate; online read p95/p99; default-value rate; feature reuse; failed entity lookups; cost per million reads.

Retrieval: corpus coverage; parse success; index lag; retrieval recall@k, precision@k, MRR/nDCG and hit rate; ACL-filter correctness; rerank latency; zero-result rate; stale citation rate; cost per indexed document and query.

9.10 Security, governance, compliance and risk controls

- data minimization and approved-purpose enforcement;
- access controls inherited from source through features, chunks and indexes;
- separate encryption keys or stores for high-sensitivity tenants;
- content-origin and license metadata;
- PII, secrets, malware and malicious-instruction scanning;
- label-workforce confidentiality, need-to-know access and secure tooling;
- protected evaluation sets with limited builder access where independence matters;
- deletion propagation to materialized features, embeddings, caches and snapshots;
- audit of feature/index publication and access-policy changes;
- review of proxy variables, representativeness and potential disparate impact;
- no raw sensitive text in observability systems unless explicitly approved.

9.11 Common failure modes and troubleshooting

Symptom	Likely causes	Corrective action
Excellent offline score, weak production	leakage, temporal split error, label mismatch, feature skew	reconstruct point-in-time dataset, compare online features, redefine target
Online defaults spike	materialization lag, entity mismatch, TTL expiry	inspect entity keys, freshness, store health; fail safe or route to fallback
RAG answers miss known content	parse/chunk failure, embedding mismatch, poor query, filter	test corpus coverage, retrieval stages and ACL separately; tune chunk/search/rerank
Cross-user document appears	missing ACL metadata, stale cache, post-filter bug	contain service, invalidate cache/index, audit queries, enforce pre-filter and tests
Index grows without quality	duplicates, obsolete versions, poor retention	deduplicate, version sources, tombstone deletes, rebuild and benchmark
Labels disagree heavily	ambiguous taxonomy or insufficient expertise	refine guidance, add examples, retrain annotators and adjudicate
Training-serving skew	duplicated feature code or inconsistent defaults	centralize definitions, parity tests and release coupling

9.12 Scalability, reliability, performance and cost optimization

- incremental transformations and index updates rather than full rebuilds where safe;
- partition training datasets by common access paths and compact small files;
- cache feature reads and embeddings only with correct tenant/version keys;
- use approximate nearest-neighbor indexes after measuring recall impact;
- separate ingestion, embedding and indexing queues to absorb bursts;
- batch embedding requests and exploit accelerator throughput;
- apply tiered storage to old snapshots while preserving reproducibility metadata;
- shard online feature/vector stores by tenant/entity and plan rebalancing;
- keep a reproducible full-rebuild path even when incremental updates are primary;
- maintain previous feature/index release for rapid rollback;
- set corpus and feature freshness SLOs from business need, not “real time” by default.

9.13 Production best practices and anti-patterns

Best practices: stable entity/chunk IDs; versioned definitions; point-in-time tests; retrieval test sets from real questions; ACL tests as release blockers; human-reviewed label guidelines; explicit defaults and TTL; incremental plus full rebuild paths.

Anti-patterns: copy-pasted feature logic; random splits for temporal problems; embedding everything without source/license metadata; using vector similarity as authorization; tuning generation before retrieval; treating labels as unquestioned truth; retaining all raw prompts/documents indefinitely.

9.14 Exit criteria

Proceed when datasets, labels, features and knowledge indexes are versioned, authorized, reproducible and evaluated; lineage and deletion paths exist; point-in-time/online parity is demonstrated where relevant; retrieval and ACL thresholds pass; limitations are recorded.

10. Stage 3 — Development environments and experimentation

10.1 Purpose and objectives

Provide fast, isolated and reproducible development while preventing credentials, sensitive data, untracked artifacts and environment drift from leaking into production. The stage turns exploratory work into reviewable engineering assets.

Objectives:

- provision policy-compliant workspaces in minutes;
- separate interactive exploration from durable pipelines;
- version code, environment, data references, prompts and runs;
- enable local or remote CPU/GPU execution without copying sensitive data to laptops;
- track experiments and compare against baselines;
- establish tests and evaluation harnesses before scale training;
- control cost and idle resources;
- make successful experiments reproducible by another engineer.

10.2 Core concepts and design principles

- **Notebook is an interface, not the production artifact.** Production logic moves to tested modules and pipelines.
- **Reproducibility is bounded.** Capture seeds, libraries, hardware, data and nondeterministic settings; declare remaining variance.
- **Experiment identity links all inputs and outputs.** A metric without code/data/environment lineage is not evidence.
- **Ephemeral by default.** Workspaces and compute expire; durable artifacts live in approved stores.
- **Remote data, local ergonomics.** Provide IDE integration and remote execution instead of uncontrolled data downloads.
- **Baseline discipline.** Every experiment compares to a simple baseline and the current process.

10.3 Fine-grained platform capabilities

- self-service project, repository and workspace provisioning;
- preapproved CPU/GPU images with pinned packages and vulnerability scanning;
- browser notebooks, remote IDE and CLI/SDK;
- workload identity, scoped secrets and private package/model mirrors;
- read-only or policy-scoped dataset mounts and query access;

- experiment tracking for parameters, metrics, code commit, data version, artifacts and notes;
- prompt/version tracking and trace capture for LLM experiments;
- reusable pipeline/component templates;
- unit, data, schema, integration and evaluation test frameworks;
- remote jobs, distributed compute and hyperparameter search;
- reproducible environment locks and container builds;
- collaboration, review, branch protection and code ownership;
- quotas, idle shutdown, budget alerts and usage dashboards;
- secure sandbox for untrusted model weights or code;
- artifact promotion from scratch to governed registry.

10.4 Inputs, outputs and artifacts

Inputs: approved project and risk profile, versioned data/feature/index references, experiment and evaluation plan, baseline, workspace template, budget and access policy.

Outputs: reproducible experiment runs, candidate model/prompt/retrieval configurations, tested source code, dependency lock, preliminary evaluation results, performance/cost profile and decision record.

Artifacts:

- repository and commit history;
- environment file/container definition and digest;
- notebook plus extracted tested modules;
- pipeline/component specification;
- experiment run records and lineage;
- baseline implementation;
- prompt templates/tool schemas and trace samples;
- unit/data/evaluation tests;
- candidate selection rationale and discarded alternatives;
- known limitations and unresolved risks.

10.5 System architecture and reference workflow

1. Approved intake automatically provisions project boundaries, repository, workspace, service identities and budget.
2. Developer selects a versioned base image and accesses governed datasets through policy-aware connectors.
3. Exploratory code logs runs to a central tracker; large jobs execute on remote ephemeral compute.
4. Reusable logic moves from notebook cells into packages and pipeline components with tests.
5. CI validates formatting, unit tests, secret scanning, dependency licenses, image vulnerabilities and small-data pipeline execution.
6. Experiment compares baseline and candidate using the registered evaluation plan.
7. Candidate is exported as an immutable package or configuration and submitted to training/evaluation stages.
8. Workspace expires or is archived; no production service depends on its local state.

10.6 Technologies, frameworks and tooling

- JupyterLab, managed notebook services, VS Code remote development;
- containerized development environments and internal package/model mirrors;
- Git and pull-request workflows;
- MLflow, Weights & Biases, Comet or managed experiment tracking;
- Kubeflow Pipelines, Airflow, Dagster, Metaflow or Argo Workflows;
- Ray Tune, Optuna or managed HPO;
- pytest and property-based testing; data-quality frameworks;
- prompt/evaluation tracing systems for LLM applications;

- infrastructure templates and developer portals;
- static analysis, secret scanners, dependency/license and container scanners.

10.7 Roles and responsibilities

- **Data scientist/researcher:** hypothesis, experiments, analysis and model limitations.
- **ML/LLM engineer:** reproducible code, runtime, pipeline and performance profile.
- **Application/knowledge engineer:** integration, prompts, retrieval and tool contracts.
- **Platform team:** workspace images, access, remote compute, tracking and templates.
- **Security:** base-image, package, sandbox and secret controls.
- **Data owner:** approved data access and usage constraints.
- **Validator/domain expert:** evaluation design and representative cases.
- **FinOps:** quotas, rate cards and abnormal-use alerts.

10.8 Decision points and trade-offs

Notebook-first versus code-first

Notebooks accelerate exploration and communication. Modules and pipelines support testing, reuse and deployment. Use both with an explicit transition criterion: any logic affecting a release must exist in version-controlled, testable code.

Managed workspace versus local development

Local development has low latency and familiar tools but weakens data control and environment consistency. Managed remote workspaces are preferred for sensitive data and accelerators. Offer remote IDE ergonomics to reduce pressure for downloads.

Shared versus per-team environments

Shared environments reduce cost but increase dependency conflicts and blast radius. Standard immutable base images plus per-project dependencies provide balance. High-risk or untrusted code may need dedicated sandboxes.

Experiment tracking depth

Log enough to reproduce and compare, but avoid capturing prohibited raw data or secrets. Store references/hashes for sensitive inputs; make sampling and redaction explicit.

10.9 KPIs and operational metrics

- workspace provisioning time and failure rate;
- time from first commit to reproducible baseline;
- percentage of runs with code/data/environment lineage;
- experiment reproducibility pass rate;
- CI pass rate, test coverage for release-critical code and secret findings;
- notebook-to-pipeline conversion rate;
- GPU/CPU active utilization and idle shutdown savings;
- experiment cost per accepted candidate;
- candidate evaluation throughput and queue time;
- number/age of unpinned dependencies and vulnerable images;
- developer satisfaction and self-service completion.

10.10 Security, governance, compliance and risk controls

- SSO, MFA and role-based project access;
- short-lived workload credentials; no secrets in notebooks, code or environment files;

- private package/model mirrors and egress allowlists;
- signed, patched base images with unprivileged containers and restricted mounts;
- no production credentials or write access in development;
- data masking, row/column policy and download controls;
- audit of data/model access and external provider calls;
- malware/model-weight scanning and sandboxed deserialization;
- dependency license and vulnerability policy;
- automated workspace expiration, retention and secure deletion;
- prohibit sensitive run payloads in trackers unless approved and encrypted.

10.11 Common failure modes and troubleshooting

Symptom	Likely cause	Response
Run cannot be reproduced	unpinned data/library, hidden notebook state, random seed/hardware variance	rebuild from clean environment; log digests and extract code
“Works in notebook” only	implicit state, local files, manual steps	convert to component pipeline; add integration test
GPU spend with little progress	idle notebooks, oversized instances, queue misuse	auto-stop, right-size, use remote jobs and quotas
Package conflict	shared mutable environment	immutable base plus project lock; rebuild image
Secret leakage	pasted key or verbose logging	revoke, rotate, scan history, use secret injection and redaction
External model call blocked	data classification/provider policy mismatch	inspect policy decision; minimize/redact data or use approved route
Metric improvement is unstable	small sample, tuning on test set, nondeterminism	lock test set, use confidence intervals/repeated runs and nested validation

10.12 Scalability, reliability, performance and cost optimization

- separate interactive scheduler from batch training queues;
- use small samples and cheap models for pipeline validation before full scale;
- cache deterministic preprocessing with data/version keys;
- provide spot/preemptible pools for checkpointable experimentation;
- establish GPU quotas, maximum runtime and idle termination;
- make base images layered and pre-pull common images/models;
- use remote object storage and artifact streaming instead of local disks;
- monitor queue wait, startup time and environment build time as developer-flow SLOs;
- maintain capacity classes for urgent incidents, standard work and best effort;
- use experiment early stopping and search algorithms rather than exhaustive grids.

10.13 Production best practices and anti-patterns

Best practices: clean-room rerun before candidate selection; versioned baselines; small deterministic tests; code review; protected evaluation sets; remote jobs; cost visible inside experiment UI; research notes linked to runs.

Anti-patterns: mutable shared conda environment; notebook as deployment package; logging secrets/raw PII; tuning directly on production feedback without holdout; unlimited GPU notebooks; downloading models from the public internet during a run; selecting by one aggregate metric.

10.14 Exit criteria

G2 passes when another qualified engineer can reproduce the baseline and candidate from source, locked environment and versioned inputs; tests pass; experiment lineage is complete; cost/performance is profiled; the candidate and evaluation plan are ready for controlled training and validation.

11. Stage 4 — Training, fine-tuning and optimization

11.1 Purpose and objectives

Produce an immutable, reproducible candidate model or adapter that meets learning objectives and can be evaluated on the target runtime. Training includes classical model fitting, hyperparameter optimization, deep learning, continued pretraining, supervised fine-tuning, preference/alignment methods, distillation, pruning and quantization.

Objectives:

- execute versioned training code against approved data with deterministic lineage;
- select algorithms and scale proportional to the problem;
- use distributed compute efficiently and recover from interruption;
- prevent data contamination, unauthorized use and supply-chain compromise;
- generate checkpoints, metrics and runtime-compatible exports;
- characterize convergence, variance, memory, throughput and cost;
- produce a candidate without using protected test data for optimization.

11.2 Core concepts and design principles

- **Training is a pipeline, not a command.** Data validation, environment build, fit, checkpoint, evaluation, export and evidence are one controlled workflow.
- **Scale only after correctness.** Verify code and learning behavior on a small representative slice before adding nodes or GPUs.
- **Checkpoint for recovery and evidence.** Checkpoints are versioned artifacts with data/code/environment identity, not anonymous files.
- **Separate optimization sets.** Training and tuning may use train/validation data; final test and independent challenge sets remain protected.
- **Parameter efficiency is an architectural lever.** Prompting, RAG, adapters, distillation and quantization can outperform full fine-tuning on cost, control and portability.
- **Runtime target matters during training.** Export format, context length, tokenizer, precision and hardware constraints must be known before the final candidate.
- **Reproducibility has tolerances.** Distributed floating-point training may not be bit-identical; define acceptable metric variance and deterministic settings.

11.3 Fine-grained platform capabilities

Shared capabilities

- declarative training-job specification with code, data, resources and policy;
- preflight checks for data, schema, permissions, quota and artifact destination;
- reproducible container/environment build;
- CPU/GPU/accelerator scheduling, queues, priority, fair share and topology awareness;
- experiment tracking, logs, metrics, profiling and distributed traces;
- checkpoint save/resume, retention and promotion;
- hyperparameter search, early stopping and failure recovery;
- distributed data loading, sharding and caching;

- deterministic seeds and environment capture;
- artifact export, validation and signing;
- cost estimate, budget guard and termination threshold.

Classical ML

- scikit-learn/XGBoost/LightGBM/Spark pipelines;
- cross-validation appropriate to entity, group or time;
- class weighting, sampling and imbalance handling;
- preprocessing fitted only on training folds;
- calibration and threshold optimization;
- distributed HPO and ensemble construction;
- explainability artifacts and feature-importance diagnostics;
- export to portable formats where required.

SLM/LLM and deep learning

- tokenizer and vocabulary management;
- sequence packing, masking, deduplication and contamination checks;
- mixed precision and activation checkpointing;
- data, tensor, pipeline and expert parallelism;
- parameter/gradient/optimizer sharding and offload;
- continued pretraining, SFT, PEFT/LoRA/QLoRA and preference optimization;
- adapter registry and compatibility checks;
- distillation from a larger teacher under license and policy;
- structured-output and tool-use fine-tuning where justified;
- quantization-aware or post-training quantization;
- pruning, sparsity and export to serving/edge formats;
- training-time safety filtering and benchmark decontamination.

11.4 Inputs, outputs and artifacts

Inputs: approved training/validation data, feature or tokenization specification, source commit, environment lock, training configuration, base model and license, resource plan, evaluation hooks, budget and security policy.

Outputs: model checkpoint/package, adapter, tokenizer, calibration/thresholds, training metrics, resource profile, exported runtime artifact, lineage and training report.

Artifacts:

- training-job specification and orchestration run;
- exact dataset/feature/tokenizer/base-model references and digests;
- source commit, container digest, dependency lock and hardware topology;
- hyperparameters, seeds and distributed configuration;
- checkpoints with retention class;
- learning curves, profiler output and failure/retry history;
- HPO search space and selection logic;
- final model/adapter/tokenizer/calibration artifacts;
- export and load test;
- cost, energy or accelerator-hour report where tracked;
- training data/license and model-source attestations.

11.5 System architecture and reference workflow

1. CI builds and signs the training image from pinned dependencies.

2. Orchestrator resolves immutable data and base-model references, verifies authorization and estimates resources/cost.
3. A smoke run validates loading, forward/backward pass, checkpoint and metrics on minimal data.
4. Scheduler places the job using accelerator type, locality, topology, priority and quota.
5. Training workers read sharded data, emit metrics and write atomic checkpoints to durable object storage.
6. Watchdog detects stalls, NaNs, data-loader starvation, network failure and budget breach.
7. Failed preemptible jobs resume from a verified checkpoint; nonrecoverable failures preserve diagnostics.
8. Candidate selection follows predefined validation metrics and complexity/cost constraints.
9. Export pipeline converts to target format/precision, validates numerical tolerance and performs a serving smoke test.
10. Candidate artifacts, lineage, training report and signatures are submitted to independent evaluation.

11.6 Technologies, frameworks and tooling

- classical: scikit-learn, XGBoost, LightGBM, Spark ML;
- deep learning: PyTorch, TensorFlow/JAX where organizationally supported;
- distributed training: PyTorch FSDP/FSDP2, DeepSpeed, Ray Train, Horovod or managed equivalents;
- LLM: Hugging Face Transformers, PEFT, TRL and approved model-specific tooling;
- HPO: Ray Tune, Optuna, Katib or managed services;
- orchestration: Kubeflow Pipelines, Argo Workflows, Airflow, Dagster or managed training pipelines;
- profiling: framework profilers, accelerator telemetry and network/storage monitoring;
- export/optimization: ONNX, TensorRT, OpenVINO, llama.cpp formats or target-specific compilers;
- tracking/registry: MLflow or managed lifecycle systems;
- cluster scheduling: Kubernetes batch schedulers, queueing/fair-share systems or managed accelerator schedulers.

11.7 Roles and responsibilities

- **Data scientist/research scientist:** objective, algorithm, training design and interpretation.
- **ML/LLM engineer:** reproducible pipeline, distributed configuration, export and performance.
- **Data engineer:** training dataset reliability and efficient access.
- **Platform/accelerator engineer:** scheduler, images, drivers, networking, storage and quotas.
- **Security/privacy/legal:** training data rights, base-model/license, isolation and supply chain.
- **Validator:** protected tests and independence.
- **FinOps:** capacity option, budget guard, commitment and utilization analysis.
- **SRE:** critical training-service reliability and recovery standards.

11.8 Decision points and trade-offs

Prompt/RAG versus fine-tuning

Use prompt and retrieval first when knowledge changes frequently, traceable sources matter or task adaptation is mostly contextual. Fine-tune when behavior, style, classification boundaries, structured output or domain language cannot meet thresholds through context alone. Combine them when behavior and knowledge are distinct.

Full fine-tune versus PEFT

Full fine-tuning offers maximal parameter adaptation but requires more memory, compute and isolated model copies. PEFT is cheaper, faster and supports many tenant/domain adapters, but may underperform for deep behavioral shifts and adds adapter/base compatibility management.

Larger model versus specialized smaller model

A larger model may improve broad capability but increases latency, cost, exposure and operational complexity. A specialized SLM can be superior for bounded high-volume tasks and edge deployment. Compare quality at the same end-to-end constraint, including retrieval, review and cost.

On-demand, reserved or spot capacity

- On-demand minimizes commitment but may be scarce and expensive.
- Reserved capacity improves availability/economics for predictable utilization but creates idle risk.
- Spot/preemptible capacity is efficient for checkpointable jobs but increases elapsed time and engineering complexity.
- Hybrid portfolios allocate critical deadlines to reserved capacity and exploratory/recoverable jobs to spot.

Quantization

Lower precision reduces memory and may increase throughput, but can degrade rare classes, long-context behavior, tool use or numerical calibration. Validate task and segment quality after quantization; do not assume benchmark equivalence.

11.9 KPIs and operational metrics

Learning: train/validation loss; task metric; calibration; generalization gap; convergence steps/time; run-to-run variance; early-stop rate.

System: job success; queue wait; startup time; checkpoint duration/failure; data-loader utilization; GPU/accelerator utilization; memory headroom; tokens/examples per second; communication overhead; straggler rate; scaling efficiency.

Economics: accelerator-hours per accepted candidate; cost per million training tokens/examples; failed-run waste; spot interruption recovery; reserved-capacity utilization; storage/checkpoint cost.

Governance: percentage with complete data/base-model/license lineage; protected-test access violations; unsigned checkpoints; vulnerability or policy failures.

For large-model training, track model FLOP utilization or an equivalent useful-work metric, but interpret it with end-to-end time and cost. High device utilization can coexist with inefficient data, convergence or oversized models.

11.10 Security, governance, compliance and risk controls

- approved, licensed and purpose-compatible training data and model weights;
- malware, unsafe serialization and artifact scanning before loading external models;
- pinned hashes, signed containers and controlled package/model mirrors;
- isolated training networks with restricted egress;
- short-lived identities and scoped access to exact data/artifact paths;
- encryption for data, checkpoints and inter-node traffic where required;
- no secrets embedded in checkpoints, logs or tokenizer files;
- benchmark/test-set access control and contamination detection;
- retention and deletion policy for intermediate checkpoints and optimizer state;
- dual control for high-risk base-model or data changes;
- lineage and provenance conforming to supply-chain policy;
- content and worker protections for harmful training data.

11.11 Common failure modes and troubleshooting

Symptom	Likely cause	Diagnosis and corrective action
GPU out-of-memory	sequence/batch too large, fragmentation, unsharded states	profile memory; reduce/pack batch, checkpoint activations, shard/offload, change precision
Loss diverges/NaN	learning rate, bad data, precision overflow, initialization	inspect first bad step/data shard, gradients and scaler; lower LR, clip, clean data

Symptom	Likely cause	Diagnosis and corrective action
Poor multi-node scaling	network bottleneck, small batch, imbalance, data I/O	profile communication and loaders; increase work per step, topology-aware placement, shard data
Low GPU utilization	slow input, CPU transform, checkpointing, synchronization	precompute/tokenize, parallelize loaders, local cache, reduce sync frequency
Resume changes behavior	incomplete checkpoint, data order or RNG not restored	include optimizer/scheduler/RNG/dataloader state; validate resumed curve
Validation improves, test fails	over-tuning, leakage, contamination	lock test set, review split and benchmark overlap; nested validation
Fine-tune forgets general skills	narrow data, excessive LR/epochs	mix representative data, regularize, lower update magnitude, use adapter
Quantized model regresses	sensitive layers/tasks, calibration set mismatch	per-layer analysis, better calibration, mixed precision or less aggressive quantization
Cost runaway	search explosion, stalled job, retries, oversized nodes	budget watchdog, max trials/runtime, health termination, staged scaling

11.12 Scalability, reliability, performance and cost optimization

- validate a single process, single node and small data before distributed scale;
- use profiling to identify compute, memory, network or storage bottleneck;
- colocate/cache hot data, shard deterministically and avoid small-file storms;
- use mixed precision, fused kernels and efficient attention where supported and validated;
- right-size global batch and learning-rate schedule; large batch is not free if convergence worsens;
- use PEFT, distillation and curriculum/sampling to reduce compute;
- checkpoint at an interval derived from failure rate, checkpoint time and restart cost;
- retain fewer checkpoints by policy while preserving milestones and best candidate;
- schedule large jobs across topology-compatible nodes and avoid heterogeneous stragglers;
- preempt best-effort work for production incidents only under explicit priority policy;
- measure actual cost per accepted model, including failed experiments and idle reservation.

11.13 Production best practices and anti-patterns

Best practices: smoke test; pinned inputs; atomic checkpoints; protected test set; training watchdog; measured scaling; export validation; clear base/adaptor compatibility; budget kill switch.

Anti-patterns: scaling an unprofiled script; full fine-tuning by default; using the test set for HPO; untrusted pickle/model loading; downloading dependencies during training; checkpointing only to local disk; selecting the latest checkpoint without predefined criterion; measuring only GPU utilization.

11.14 Exit criteria

A candidate proceeds to G3 evaluation when training is reproducible within declared tolerance, lineage is complete, artifacts load in the target runtime, protected test data was not used for optimization, convergence and resource profiles are understood, scans pass and cost is within the approved envelope.

12. Stage 5 — Evaluation, validation and assurance

12.1 Purpose and objectives

Determine whether the exact AI-system release is fit for its intended context and whether residual risk is acceptable. Evaluation covers model quality, system behavior, robustness, security, privacy, fairness, human factors, reliability, latency and economics. It is the primary evidence for release decisions.

Objectives:

- measure against the current baseline and predefined thresholds;
- test representative segments, edge cases and foreseeable misuse;
- separate model, retrieval, workflow and system failures;
- quantify uncertainty and practical significance;
- evaluate safety, security, privacy and human oversight;
- test the production-equivalent runtime and configuration;
- provide independent challenge for high-impact systems;
- generate a signed, reproducible evaluation report linked to the release.

12.2 Core concepts and design principles

- **Fitness is multidimensional.** No single accuracy or benchmark score establishes production readiness.
- **Evaluate the deployed system.** Model, prompt, retrieval corpus, tools, policy, quantization and runtime all affect outcomes.
- **Thresholds precede results.** Post-hoc acceptance criteria invite bias.
- **Segments matter.** Aggregate performance can hide severe failures for rare, protected or high-value groups.
- **Confidence and effect size matter.** Small score differences may be noise or economically irrelevant.
- **Test independence matters.** High-risk validation must be organizationally and technically separated from optimization.
- **LLM-as-judge is an instrument, not authority.** Calibrate against humans, version the judge and monitor bias/instability.
- **Security tests assume hostile inputs and content.** Retrieved documents, prompts, model files and tool results are untrusted.

12.3 Fine-grained platform capabilities

Evaluation management

- versioned evaluation policies, datasets, rubrics, metrics and thresholds;
- protected golden sets and controlled builder access;
- stratified sampling and segment definitions;
- scalable batch execution against production-equivalent endpoints;
- paired comparison, confidence intervals and significance/equivalence tests;
- regression comparison to baseline and previous production release;
- human-evaluation queue, blind randomization and adjudication;
- automatic evidence generation, approval routing and exception handling;
- result drill-down to individual cases and trace replay;
- contamination, duplication and leakage checks.

Classical ML methods

- classification: precision, recall, F1, specificity, ROC-AUC, PR-AUC, log loss, calibration/Brier score and threshold curves;
- regression: MAE, RMSE, quantile/pinball loss, MAPE variants with denominator safeguards and interval coverage;
- ranking/recommendation: precision/recall@k, MAP, MRR, NDCG, coverage, diversity and exposure;

- forecasting: rolling-origin backtests, scale-aware error, bias, interval coverage and stability across horizons;
- anomaly/fraud: event-level recall, false positives per unit, time-to-detect and investigator yield;
- clustering/segmentation: stability, separation, usefulness and downstream outcome—not silhouette alone;
- probability decisions: calibration by segment, expected cost/value and decision-curve analysis;
- causal/uplift: treatment-effect calibration, policy value and randomized or quasi-experimental validation.

LLM and generative methods

- task success and rubric-based correctness;
- factual consistency, groundedness and citation support;
- instruction adherence and structured-output/schema validity;
- relevance, completeness, concision and style where business-relevant;
- refusal correctness: unsafe requests refused and valid requests not over-refused;
- toxicity, bias, privacy leakage, copyright/IP and prohibited-content tests;
- multilingual, long-context, ambiguous-input and adversarial robustness;
- consistency across repeated samples and temperature settings;
- prompt injection, jailbreak, indirect injection and data-exfiltration tests;
- memorization or canary leakage tests where applicable;
- latency, time to first token, inter-token latency, throughput and token use.

RAG methods

- corpus coverage and document parse success;
- retrieval recall@k, precision@k, hit rate, MRR and nDCG;
- filter/ACL correctness and cross-tenant isolation;
- context relevance, sufficiency and redundancy;
- answer groundedness, citation precision/recall and source freshness;
- robustness to malicious or conflicting documents;
- query rewrite and reranker contribution through ablation;
- end-to-end task success with and without retrieval.

Agent methods

- end-to-end task completion and partial-credit rubric;
- tool selection, argument validity and sequence correctness;
- permission/policy compliance and approval adherence;
- side-effect correctness, idempotency and compensation;
- loop rate, step count, timeout and budget adherence;
- recovery from tool errors, stale state and provider failure;
- human intervention/escalation and unsafe-action prevention;
- state/memory isolation and cross-session leakage;
- simulation/replay before live side effects.

Nonfunctional assurance

- load, stress, soak and concurrency testing;
- availability, retry, timeout, circuit-breaker and fallback behavior;
- degradation under missing features, stale index or provider outage;
- resource saturation and autoscaling behavior;
- accessibility and user-interface comprehension;
- human-review accuracy, workload and automation bias;
- disaster recovery and rollback verification for critical services.

12.4 Inputs, outputs and artifacts

Inputs: exact release manifest, intended/prohibited use, risk and service tier, evaluation policy, independent datasets, threat model, production-like environment, baseline and business thresholds.

Outputs: evaluation results, failure taxonomy, residual-risk statement, mitigation plan, release recommendation and signed report.

Artifacts:

- evaluation plan and versioned policy;
- dataset manifests, segment definitions and contamination checks;
- metric implementation and test harness;
- classical/LLM/RAG/agent test results as applicable;
- human-evaluation rubric, rater training, agreement and adjudication;
- robustness, fairness, privacy and security reports;
- latency/load/capacity/cost profile;
- comparison to baseline and prior release;
- known limitations and residual-risk acceptance;
- validator identity, independence statement and approval decision.

12.5 System architecture and reference workflow

1. Evaluator resolves the release manifest and verifies artifact digests.
2. Evaluation service provisions a production-equivalent isolated environment.
3. Static checks validate metadata, licenses, dependency/security scans and required artifacts.
4. Deterministic and statistical functional suites run on protected sets and segments.
5. For RAG/agents, component tests isolate retrieval, model, policy and tool behavior before end-to-end tests.
6. Adversarial/security/privacy/fairness suites run according to risk profile.
7. Performance/load tests execute with realistic request and token distributions.
8. Human evaluation samples uncertain, high-impact and representative cases using blinded rubrics.
9. Statistical service computes confidence intervals, paired deltas and threshold results.
10. Validator reviews failures and mitigations, then approves, rejects or requests bounded remediation.
11. The signed report and raw result references are registered against the exact release.

12.6 Technologies, frameworks and tooling

- general test frameworks and data-science statistics libraries;
- MLflow or lifecycle platform for evaluation records;
- domain benchmark harnesses and EleutherAI LM Evaluation Harness for supported language-model tasks;
- prompt/RAG evaluation frameworks, augmented with enterprise-owned metrics and datasets;
- load tools such as k6, Locust, JMeter or protocol-specific generators;
- security testing mapped to OWASP LLM risks and MITRE ATLAS techniques;
- fairness, explainability and privacy toolkits selected for task and regulation;
- human-evaluation systems with blinded queues and audit;
- observability/tracing to replay exact prompts, retrieval and tool calls under policy.

Tool-generated scores must be validated. Framework defaults rarely encode enterprise context, segment definitions or business loss.

12.7 Roles and responsibilities

- **Product owner/domain expert:** outcome threshold, rubric and acceptable trade-offs.
- **Data scientist/ML engineer:** test implementation and failure analysis.

- **Independent validator/model risk:** challenge design, data independence and release recommendation for high risk.
- **Security/red team:** abuse, adversarial, prompt injection, supply-chain and exfiltration tests.
- **Privacy/legal/compliance:** privacy, rights, transparency and regulatory requirements.
- **Human-factors/UX:** oversight effectiveness, user comprehension and accessibility.
- **SRE/performance engineer:** load, resilience, capacity and SLO evidence.
- **FinOps:** cost and capacity acceptance.
- **Data/knowledge owner:** dataset/corpus validity and freshness.

12.8 Decision points and trade-offs

Metric and threshold selection

Choose metrics from decision costs and user impact. For rare positive events, PR-AUC and precision/recall at operating thresholds are often more informative than accuracy. For generation, use task rubrics and grounded evidence rather than surface-text overlap alone.

Offline versus online evidence

Offline tests are controlled, repeatable and safe but may miss user adaptation and traffic shift. Online shadow/canary/A-B tests reveal real behavior but expose users and can confound causality. Use offline gates first, then bounded online evidence.

Automated versus human evaluation

Automation provides scale and regression speed. Human judgment is necessary for nuanced, high-impact or subjective tasks. Use calibrated automated metrics for broad coverage and human review for gold standards, disagreements and risk-critical samples.

Strict safety versus utility

Overly aggressive filters may deny legitimate use; permissive filters may allow harm. Evaluate both unsafe acceptance and valid-request refusal by segment. Product policy, not a generic vendor setting, defines the operating point.

Statistical superiority versus equivalence/non-inferiority

A cheaper/faster model may be acceptable if quality is statistically non-inferior within a predeclared margin. Do not require nominal superiority when the business objective is equivalent quality at lower cost or latency.

12.9 KPIs and operational metrics

- percentage of releases with complete, reproducible evaluation evidence;
- suite duration, queue time, infrastructure cost and test flakiness;
- regression escape rate: production defects not detected pre-release;
- false-block and waiver rate;
- metric coverage by segment, risk and failure mode;
- human inter-rater agreement and adjudication rate;
- security/red-team finding count, severity and remediation age;
- baseline-to-candidate delta with confidence interval;
- cost/latency/quality frontier and Pareto-dominant candidate rate;
- percentage of test cases sourced from real incidents and feedback;
- benchmark contamination or leakage findings.

12.10 Security, governance, compliance and risk controls

- protect test sets and high-risk scenarios from optimization leakage;

- require independent validation and separation of duties for defined tiers;
- version judges, rubrics, prompts and threshold policy;
- redact sensitive payloads in evaluation logs and restrict access;
- perform adversarial testing in isolated environments with no uncontrolled tool permissions;
- include privacy leakage, membership/model extraction and sensitive-information disclosure tests as applicable;
- document fairness rationale, segments, sample limits and mitigation;
- retain evidence and approval according to regulation and audit policy;
- require explicit residual-risk acceptance for failed nonblocking controls;
- reject releases with missing telemetry, rollback or incident readiness regardless of quality score.

12.11 Common failure modes and troubleshooting

Failure mode	Diagnostic	Corrective action
Aggregate score hides harm	inspect segment/confusion/error distributions	define blocking segment thresholds and collect more representative cases
Evaluation score unstable	sampling variance, stochastic generation, judge drift	repeated paired runs, fixed configs, confidence intervals, calibrated human checks
LLM judge favors a style/model	position/verbosity/self-preference bias	blind/randomize order, multiple judges, human calibration and anchored rubric
RAG metric blames generator	retrieval coverage not measured	evaluate parse, retrieval, rerank, context and answer separately
Test set leaked into tuning	shared access, benchmark memorization	replace/refresh protected set, audit access and contamination
Load test passes but production fails	unrealistic token/feature distribution or dependencies	replay sanitized traffic shape and include downstream services/queues
Safety filter over-refuses	broad keywords or provider default	segment false refusals, tune policy, use context-aware deterministic checks
Agent passes happy path only	no tool failure/permission/state tests	simulate faults, duplicate events, stale state and denied actions
Statistical significance without value	large sample, tiny effect	compare to minimum practical effect and full unit economics

12.12 Scalability, reliability, performance and cost optimization

- maintain layered suites: fast commit tests, candidate tests, full release tests and periodic deep assurance;
- cache deterministic results keyed by exact artifact digests while invalidating material changes;
- parallelize cases, not shared state; isolate stochastic seeds and tenant context;
- use representative sampling plus risk-based oversampling of rare critical cases;
- run expensive red-team and human evaluation on selected candidates, not every exploratory run;
- generate synthetic adversarial cases but validate them with domain experts and real incidents;
- keep a stable core benchmark plus rotating hidden sets to reduce overfitting;
- measure evaluation cost per release and defect prevented;
- profile model, retrieval and tools separately to locate performance bottlenecks;
- use non-inferiority gates to enable smaller/cheaper models when quality is sufficient.

12.13 Production best practices and anti-patterns

Best practices: predeclared policy; system-level evaluation; protected sets; segment thresholds; paired comparisons; incident-derived cases; human calibration; component ablation; load and failure testing; signed report.

Anti-patterns: one public benchmark; LLM-as-judge without calibration; accepting average accuracy; evaluating a different quantization/prompt/index than production; tuning on test results; equating toxicity filter pass with safety; no cost/latency thresholds; waiver by informal chat.

12.14 Exit criteria

G3 passes when the exact release meets blocking quality, safety, security, privacy, fairness, reliability, latency and cost thresholds; limitations and residual risk are documented; required independent validation is complete; rollback and observability are testable. A failed blocking threshold requires remediation or formal, authorized exception—not reinterpretation after results.

13. Stage 6 — Registration, provenance and promotion

13.1 Purpose and objectives

Create the authoritative inventory and immutable chain of custody for datasets, models, prompts, policies and deployable system releases. Registration turns a candidate into a governed asset; promotion changes approved lifecycle state without altering bytes.

Objectives:

- store immutable artifacts with content digests and signatures;
- link all components through a release manifest;
- record ownership, intended use, risk, evaluation and approval;
- enforce lifecycle states and allowed transitions;
- provide aliases for consumers without mutating versions;
- support reproducibility, discovery, comparison, rollback and audit;
- prevent unapproved or vulnerable artifacts from reaching production.

13.2 Core concepts and design principles

- **Registry metadata and artifact storage are distinct.** Metadata may live in a database; large immutable artifacts live in durable object/image stores.
- **Digest is identity.** Names and tags are mutable pointers; cryptographic digest identifies bytes.
- **Promotion, not copying by hand.** Automated workflows promote verified artifacts and preserve lineage.
- **System release over model version.** Production consumes a bundle that includes prompt, policy, data/feature/index and runtime references.
- **Aliases are controlled interfaces.** `champion`, `candidate` or environment aliases move through policy and audit.
- **Provenance is generated by build systems.** Self-attestation by a developer is insufficient for high assurance.

13.3 Fine-grained platform capabilities

- registries for datasets, features, labels, models, adapters, tokenizers, prompts, evaluation sets, policies and release bundles;
- immutable artifact and container repositories;
- semantic metadata, tags, owners, risk tier and approved uses;
- lineage graph from source through training/evaluation/deployment;
- model/system/data/prompt cards and searchable documentation;
- SBOM and model/AI BOM attachment;
- build provenance, signatures and verification policy;
- vulnerability, license and malware scan results;
- stage/alias transitions with policy and approvals;
- compatibility constraints among base model, adapter, tokenizer, runtime and hardware;
- deprecation, expiration, quarantine, recall and retirement states;
- cross-region replication and backup;
- event stream for registry changes and downstream automation.

13.4 Inputs, outputs and artifacts

Inputs: candidate artifacts and digests, release manifest, training and evaluation evidence, cards, scan results, owners, risk/service tier and approvals.

Outputs: registered immutable versions, signed provenance, lifecycle state, environment alias or rejection/quarantine record.

Artifacts:

- model/adapter/tokenizer packages;
- prompt/tool/workflow/policy package;
- container image and deployment chart/specification;
- release manifest with dependency graph;
- model and system cards;
- SBOM/model-BOM and vulnerability/license reports;
- signature, certificate/identity and provenance attestation;
- evaluation report and approval record;
- compatibility matrix and runtime profile;
- deprecation/retention/rollback metadata.

13.5 System architecture and reference workflow

1. Build pipeline writes artifacts to a staging repository using content-addressed paths.
2. Registry validates required metadata, schema and artifact existence.
3. Security service scans packages, images and external model formats in isolation.
4. Provenance verifier checks trusted builder identity, source commit, build recipe and signatures.
5. Evaluation service attaches a report whose release digest matches exactly.
6. Policy engine computes allowed transition from `draft` to `validated`, `approved`, `staged` or `production`.
7. Authorized approvers sign the transition; registry emits an immutable event.
8. GitOps/deployment systems consume only approved digest or environment alias.
9. Quarantine/recall revokes aliases and blocks new deployments while preserving evidence.

13.6 Technologies, frameworks and tooling

- MLflow Model Registry or managed model registries;
- OCI-compatible image/artifact registries;
- object storage with versioning, retention lock and replication;
- metadata/catalog and lineage platforms;
- Sigstore/Cosign or equivalent signing and verification;
- SLSA/in-toto-style provenance;
- SPDX or CycloneDX for software and AI/model component inventories;
- policy engines such as OPA and admission controls such as Gatekeeper/Kyverno;
- secrets/KMS and immutable audit logging.

13.7 Roles and responsibilities

- **Artifact owner/model owner:** completeness, intended use and technical compatibility.
- **Platform registry team:** schemas, availability, replication, APIs and lifecycle automation.
- **Security/supply-chain team:** trust roots, scanning, signing and verification policy.
- **Validator/risk owner:** evaluation and approval evidence.
- **Release manager/service owner:** promotion and rollback target.
- **Records/legal:** retention, legal hold and licensing.
- **Auditor:** evidence review and control testing without write authority.

13.8 Decision points and trade-offs

Separate registries versus unified metadata

Specialized stores may handle models, prompts and containers well. Use a federated registry architecture if needed, but expose one release manifest and search/inventory layer. Avoid forcing binary artifacts into a metadata database or maintaining disconnected lifecycle states.

Mutable tags versus immutable digests

Use human-readable aliases for operations, but resolve and record digests at deployment. Never rely on `latest` or a mutable container/model tag as production identity.

Retention versus storage cost

Retain production releases, rollback targets and required evidence long enough for audit and incident reconstruction. Expire intermediate checkpoints and failed candidates by policy. Preserve metadata/digests even when large artifacts are legally deleted, where permitted.

13.9 KPIs and operational metrics

- percentage of production deployments resolved to immutable digests;
- metadata completeness and lineage coverage;
- signature/provenance verification pass rate;
- artifact scan findings and remediation age;
- time from evaluation approval to promotable release;
- orphan artifact count and storage growth;
- rollback artifact availability and restore test success;
- registry API availability/latency;
- stale aliases, expired approvals and overdue deprecations;
- cross-region replication lag.

13.10 Security, governance, compliance and risk controls

- deny unsigned or untrusted-builder artifacts;
- verify artifact digest at every promotion and deployment;
- restrict lifecycle transitions by role and risk tier;
- separate builder, validator and promoter for high-risk releases;
- immutable audit of metadata and alias changes;
- retention lock for required evidence;
- vulnerability/license policy with severity and exception rules;
- quarantine compromised artifacts and revoke signing identities;
- restrict external model formats and unsafe deserialization;
- protect model weights as sensitive intellectual property where applicable;
- record provider/model license and permitted use/distribution;
- replicate only to approved regions.

13.11 Common failure modes and troubleshooting

Symptom	Likely cause	Corrective action
Cannot reproduce production	tag moved, missing data/prompt/runtime link	resolve digest from deployment/audit, require complete release manifest
Adapter fails after base upgrade	compatibility not enforced	declare base/tokenizer/runtime constraints; block incompatible alias move

Symptom	Likely cause	Corrective action
Registry says approved but bytes changed	mutable storage/tag or digest omitted	content-address, verify hash at read/deploy, investigate supply-chain incident
Rollback artifact missing	aggressive cleanup or regional replication gap	retention policy for prod/rollback; restore from immutable backup
Metadata incomplete	optional fields or manual entry	schema validation and automated population from pipelines
Scan blocks all releases	policy too broad or scanner false positive	triage severity/exploitability, calibrated exception with expiry
Orphan storage grows	runs write outside registry lifecycle	enforce staging namespace and garbage collection based on references

13.12 Scalability, reliability, performance and cost optimization

- store large blobs once by digest and deduplicate references;
- cache metadata reads while preserving authorization and lifecycle freshness;
- separate high-availability metadata service from scalable object storage;
- use asynchronous deep scans but block promotion until required results complete;
- replicate production artifacts and trust roots before deployment;
- test restore of metadata and blobs together;
- partition catalogs by tenant/domain while preserving global search and policy;
- tier old nonproduction artifacts to cheaper storage;
- emit registry events for decoupled deployment, monitoring and audit workflows;
- protect the registry from noisy experiment logging through staging buffers or separate tracking stores.

13.13 Production best practices and anti-patterns

Best practices: content digests; signed provenance; one system release manifest; automated metadata; explicit lifecycle; environment aliases with audit; quarantine/recall; tested restore.

Anti-patterns: `latest` tags; shared file-system model folders; manually uploading from laptops; model registry disconnected from prompts/index/runtime; deleting previous production immediately; treating a model card as approval; allowing runtime internet downloads.

13.14 Exit criteria

G4 registration readiness requires immutable signed artifacts, complete release manifest, evaluation attached to the same digest, required cards and scans, authorized lifecycle state, replicated rollback target and deployable runtime contract.

14. Stage 7 — CI/CD/CT and release engineering

14.1 Purpose and objectives

Automate safe, repeatable movement from source change to validated production release. CI validates code and artifacts; CD promotes approved bundles; CT retrains or retunes under explicit triggers and the same gates. The objective is fast flow with reproducible evidence and controlled blast radius.

14.2 Core concepts and design principles

- **Build once, promote many.** The exact signed artifact evaluated in staging is deployed to production.

- **Declarative desired state.** Git or another controlled repository records deployment configuration and policy-reviewed changes.
- **Progressive exposure.** Separate technical deployment from traffic or decision authority.
- **Every material input is a trigger.** Data, feature, corpus, prompt, policy, tool, base model and runtime changes may require different test scopes.
- **CT is not autonomous release.** Automated retraining produces a candidate; policy determines evaluation and approval before promotion.
- **Rollback is precomputed.** Previous known-good bundle and configuration are always resolvable.
- **Pipeline security is production security.** CI identities and artifact systems are high-value attack surfaces.

14.3 Fine-grained platform capabilities

- branch protection, code owners and signed commits where required;
- lint, unit, type, data, schema, prompt and policy tests;
- secret, dependency, license, IaC, container and model scans;
- reproducible build in isolated trusted workers;
- artifact signing, SBOM/model-BOM and provenance;
- small-data pipeline and model load/inference smoke tests;
- evaluation orchestration and threshold policy;
- environment-specific configuration validation without artifact mutation;
- GitOps reconciliation and drift detection;
- canary, blue-green, shadow and A/B traffic controls;
- automatic health/quality rollback and manual abort;
- change record, approval and release notes;
- retraining triggers, dataset freeze, candidate generation and recertification;
- provider/version change detection and regression pipeline;
- deprecation and compatibility testing across SDK/API versions.

14.4 Inputs, outputs and artifacts

Inputs: source/prompt/data/policy change, dependency update, retraining trigger, approved release policy, test suites, environment configuration and rollout strategy.

Outputs: signed release candidate, evaluation result, deployment change, canary evidence, production alias or rollback/quarantine.

Artifacts:

- CI logs and test reports;
- trusted build provenance, image/model digests and SBOM;
- evaluation report and policy decision;
- deployment manifest and Git commit;
- change/approval record and release notes;
- rollout metrics, comparison and decision;
- rollback event if executed;
- CT trigger, dataset window and retraining rationale;
- post-deployment verification report.

14.5 System architecture and reference workflow

Commit-to-candidate

1. Pull request runs static, unit, component, schema, policy and small evaluation tests.
2. Merge to protected branch invokes an isolated trusted builder.

3. Builder fetches pinned dependencies from approved mirrors, creates artifacts, SBOM and provenance, then signs by workload identity.
4. Candidate is registered in staging state.
5. Full evaluation runs against the exact release; policy produces pass/fail and required approvals.

Candidate-to-production

6. Approved digest is referenced in an environment configuration pull request.
7. GitOps controller reconciles staging, runs smoke/integration/load checks and records health.
8. Production deployment creates capacity with zero traffic or shadow traffic.
9. Progressive delivery shifts a bounded cohort while comparing technical, quality, safety, cost and business guardrails.
10. Automated rollback fires on hard SLO/safety thresholds; operator may abort on ambiguous signals.
11. Successful rollout moves production alias and closes change record.

Continuous training

12. A schedule, drift, performance, new labels, corpus update or provider deprecation creates a retraining request.
13. Policy checks trigger legitimacy, data window, cooldown and budget; duplicates are coalesced.
14. Training creates a candidate that re-enters the same evaluation, promotion and rollout workflow.

14.6 Technologies, frameworks and tooling

- source/CI: GitHub Actions, GitLab CI, Jenkins, Tekton or enterprise equivalents;
- build: BuildKit, Kaniko alternatives where supported, reproducible package tooling;
- GitOps: Argo CD or Flux;
- workflow: Argo Workflows, Kubeflow Pipelines, Airflow, Dagster, Temporal;
- progressive delivery: service mesh/gateway/load balancer or rollout controllers;
- policy: OPA, Gatekeeper, Kyverno and custom release-policy services;
- signing/provenance: Sigstore/Cosign, in-toto/SLSA-compatible tooling;
- registries and test/evaluation services;
- feature flags and experimentation platforms for business A/B tests.

14.7 Roles and responsibilities

- **Developer/ML engineer:** code, tests and change description.
- **Code owner:** technical review and maintainability.
- **Platform release engineering:** pipeline templates, trusted builders, GitOps and deployment automation.
- **Security:** supply-chain policy and high-risk scan findings.
- **Validator/risk approver:** evaluation and required release decision.
- **Service owner:** rollout strategy, rollback and business acceptance.
- **SRE:** production SLOs, capacity, alerting and launch oversight.
- **Data/model owner:** CT trigger, training window and candidate fitness.
- **Change authority:** formal approval where service/risk tier requires it.

14.8 Decision points and trade-offs

Trunk-based versus long-lived branches

Trunk-based development with small changes reduces merge risk and supports frequent evaluation. Regulated environments may require release branches, but they should be short-lived and automated. Long-lived model branches create untestable divergence.

GitOps versus imperative deployment

GitOps improves audit, drift detection and rollback for declarative infrastructure. Some managed model APIs require imperative calls; wrap them in a controller or pipeline that records desired state, idempotency and resulting resource IDs.

Automatic versus manual promotion

Low-risk changes with strong tests can auto-promote through nonproduction and possibly bounded production canaries. High-risk systems require independent approval. Manual approval without useful evidence is ceremony; automation without accountability is unsafe.

Retraining triggers

Schedule-based CT is predictable but may retrain unnecessarily. Drift/performance-based CT is responsive but can oscillate or use delayed/noisy labels. Use minimum data volume, cooldown, stability and business-trigger rules; retraining is not always the correct response to drift.

14.9 KPIs and operational metrics

- lead time for change and deployment frequency;
- CI duration, queue time and success/flakiness;
- evaluation duration and gate failure distribution;
- change failure rate and rollback rate;
- mean time from approved candidate to production;
- percentage of builds with trusted provenance/signature/SBOM;
- configuration drift and out-of-band change count;
- canary decision time and false rollback rate;
- CT trigger-to-candidate time and unnecessary retraining rate;
- release size, batch size and time between production versions;
- pipeline availability and mean time to restore.

14.10 Security, governance, compliance and risk controls

- protected branches, required reviews and least-privilege CI identities;
- ephemeral isolated runners; no shared long-lived credentials;
- approved dependency/model mirrors and restricted egress;
- signed artifacts and verification at admission/deployment;
- policy-as-code for risk-tier gates, regions, resource limits and providers;
- dual control for production alias or high-impact configuration changes;
- immutable change and approval logs;
- secrets injected at runtime, never stored in Git or artifacts;
- break-glass process with time-bound access and retrospective review;
- provider model-version pinning or change detection where pinning is unavailable;
- automatic block for expired approvals, critical vulnerabilities or recalled artifacts;
- evidence retention and release-to-incident traceability.

14.11 Common failure modes and troubleshooting

Symptom	Likely cause	Corrective action
Staging passes, production differs	rebuild or mutable dependency/config	build once, digest pin, environment overlays only, compare manifest
Pipeline intermittently fails	shared state, external dependency, nondeterministic tests	isolate runners, mock/stub where valid, retry only transient steps, remove flakiness

Symptom	Likely cause	Corrective action
Canary quality undetectable	low traffic or delayed labels	longer shadow, proxy metrics, stratified routing, human review before expansion
Rollback fails	schema/state incompatibility or missing artifact	backward-compatible migration, dual read/write, pretest rollback and retain bundle
CT loops repeatedly	sensitive drift trigger, no cooldown	hysteresis, minimum evidence, root-cause triage and trigger coalescing
Git says desired state but provider drifted	unmanaged imperative resource	controller/reconciliation, import resource state, alert on drift
CI credential compromised	long-lived broad token or runner persistence	revoke, rotate, investigate provenance, use workload identity and ephemeral runners
Scan backlog blocks releases	central bottleneck or repeated unchanged artifacts	parallelize, cache by digest, risk-prioritize and scale scanner

14.12 Scalability, reliability, performance and cost optimization

- tier test suites by speed and risk; fail fast before expensive training/evaluation;
- cache immutable dependency layers and scan results by digest;
- use event-driven pipelines and concurrency limits to protect downstream systems;
- shard evaluation cases and parallelize independent jobs;
- maintain dedicated capacity for urgent production fixes;
- coalesce frequent data/prompt changes into controlled release windows where appropriate;
- reduce deployment blast radius through cell/tenant/cohort rollouts;
- use shadow traffic before provisioning full duplicate capacity if privacy and cost permit;
- retain only required pipeline logs and artifacts while preserving evidence summaries;
- measure pipeline cost and developer wait, not only workload cost;
- design control-plane HA so existing production continues during CI/GitOps outage.

14.13 Production best practices and anti-patterns

Best practices: trusted build; digest promotion; policy-as-code; fast/slow test layers; progressive traffic; automatic hard-guardrail rollback; provider-change regression; CT reuses normal gates; rollback drills.

Anti-patterns: rebuilding in production; mutable tags; manual console changes; one giant serial pipeline; auto-deploying retrained models solely on offline score; approvals before evidence exists; retrying deterministic failures; shared admin CI token; canary based only on HTTP errors.

14.14 Exit criteria

A release may enter deployment when build provenance and signatures verify, all required tests/evaluations and approvals pass, desired state references immutable digests, rollback is available, progressive rollout policy is defined and production telemetry/capacity are ready.

15. Stage 8 — Deployment and delivery patterns

15.1 Purpose and objectives

Place an approved release into its target environment with the required isolation, capacity, resilience and rollback, then expose it progressively to workloads or users. Deployment is successful only when production state matches the approved manifest and the service can be operated within its SLOs.

Objectives:

- provision repeatable infrastructure and configuration;
- verify artifact, policy, identity, data and dependency compatibility;
- select delivery pattern for latency, volume, state and criticality;
- separate deployment from traffic activation;
- use shadow, canary, blue-green or cell-based exposure to bound risk;
- validate telemetry, capacity, cost and business integration;
- maintain previous known-good state and tested rollback;
- record actual production topology and release identity.

15.2 Core concepts and design principles

- **Deployment unit is the system release.** Model, prompt, index, feature contract, policy and runtime must be coherent.
- **Stateless where possible; explicit state where necessary.** State includes online features, sessions, caches, vector indexes, workflow state and feedback IDs.
- **Traffic is a control.** A deployed endpoint with zero traffic is safer to validate than an atomic cutover.
- **Backward compatibility enables rollback.** Schema, feature and API changes must tolerate coexistence during rollout.
- **Cell architecture bounds blast radius.** Partition by tenant, geography or cohort when service criticality justifies it.
- **Production parity does not mean identical size.** Nonproduction can be smaller, but architecture, security, versions and critical dependencies must be equivalent.
- **Fail safe, not merely fail closed.** Select fallback behavior from business harm: reject, queue, use previous model, use rules, defer to human or degrade features.

15.3 Fine-grained platform capabilities

- infrastructure-as-code and environment overlays;
- admission verification of signatures, provenance, risk state and allowed regions;
- deployment controller for batch jobs, online endpoints, streams, RAG services and agents;
- model/data locality and hardware compatibility validation;
- service discovery, private DNS, load balancing and certificates;
- configuration/secret injection and rotation;
- autoscaling, queueing, concurrency and rate limits;
- progressive rollout, shadowing, feature flags and cohort routing;
- migration management for feature/vector/state stores;
- health, readiness and startup probes with model-aware checks;
- rollback, traffic drain and state compatibility;
- multi-region replication/failover and edge fleet management;
- post-deployment verification and change closure;
- runtime inventory and configuration drift detection.

15.4 Inputs, outputs and artifacts

Inputs: approved signed release, deployment manifest, environment policy, SLO, capacity model, rollout strategy, dependency map, secrets/identity, runbook and rollback target.

Outputs: running service or scheduled job, production endpoint/topic/schedule, traffic state, deployment evidence, post-launch verification and updated inventory.

Artifacts:

- infrastructure and deployment definitions;
- environment configuration and policy decision;

- runtime image/model/index digests;
- capacity and autoscaling parameters;
- network, identity and secret bindings;
- rollout plan, cohort definition and abort thresholds;
- health/smoke/load results;
- actual topology and dependency record;
- rollback result or successful launch record;
- production alias and change closure.

15.5 System architecture and reference designs

Batch inference

Use an immutable job that reads a versioned input manifest and writes a versioned output with prediction IDs and reconciliation totals. Partition work, checkpoint progress and publish atomically. Downstream consumers should not observe partial results. Support replay without duplicate side effects.

Online predictive serving

Place API gateway and authorization before the endpoint; retrieve online features through a bounded-latency service; execute preprocessing and model inference in the same tested release; return prediction plus correlation ID; write asynchronous telemetry and outcome linkage. Maintain a rules or previous-model fallback where business-appropriate.

Streaming inference

Run stateful stream processors with checkpoints, event-time semantics, idempotent sinks and bounded late-event handling. Version feature and model logic together. During model change, use side-by-side consumers or a controlled state migration.

LLM/RAG serving

Deploy gateway, policy, prompt/workflow orchestrator, retrieval, model router, inference and output controls as independently scalable services where operational scale justifies it. For simpler cases, combine components but retain trace boundaries and version identity.

Edge/on-device

Package model, tokenizer, preprocessing, policy and signature into a device-compatible bundle. Roll out by cohort, verify hardware/firmware compatibility, retain local rollback, buffer privacy-safe telemetry and support revocation. Assume intermittent connectivity.

Multi-region

Choose active-active, active-passive or regional autonomy based on RTO/RPO, data residency, consistency and cost. Replicate immutable artifacts globally; replicate mutable feature/vector/workflow state only where permitted. Test DNS/traffic-manager, provider and data failover, not only compute recreation.

15.6 Technologies, frameworks and tooling

- Kubernetes Deployments/Jobs, managed endpoints, serverless and VM services;
- KServe, Seldon, BentoML, Ray Serve, NVIDIA Triton or managed predictive endpoints;
- vLLM, Hugging Face TGI, Triton or managed foundation-model endpoints;
- Argo CD/Flux and progressive-delivery controllers;
- API gateways, ingress/load balancers and service mesh only when its capabilities justify complexity;
- Kafka/Flink/Spark streaming runtimes;

- Terraform or equivalent IaC and policy-as-code;
- edge runtimes and device-management platforms;
- secrets/KMS, certificate and workload-identity services.

15.7 Roles and responsibilities

- **Service owner:** production acceptance, rollout, fallback and business communication.
- **ML/LLM/application engineer:** deployment contract and integration.
- **Platform/release engineer:** automation, runtime and environment conformance.
- **SRE:** SLO, capacity, readiness, on-call and rollout observation.
- **Security:** admission, network, identity, secret and runtime controls.
- **Data/knowledge owner:** production feature/index availability and migration.
- **Validator/risk owner:** launch conditions for high-risk systems.
- **FinOps:** capacity and cost envelope.

15.8 Decision points and trade-offs

Batch versus online

Batch maximizes throughput and simplicity but adds staleness. Online supports immediate decisions but requires low-latency features, capacity and 24×7 reliability. Use asynchronous APIs when response time may be seconds/minutes and clients can poll or receive callbacks.

Serverless/managed versus dedicated

Managed/serverless reduces operations and handles spiky demand, but may limit version pinning, custom runtimes, data locality and unit economics. Dedicated capacity improves predictability and control at the cost of idle risk and operational burden.

Canary versus blue-green versus shadow

- Shadow compares behavior without affecting decisions but doubles compute and may duplicate sensitive processing.
- Canary exposes a small real cohort and detects integration/user effects, but requires reliable guardrails.
- Blue-green offers rapid cutover/rollback and full duplicate capacity.
- Cell rollout bounds failures by tenant/region and suits large critical platforms.

Shared versus dedicated endpoint

Shared multi-model endpoints improve utilization but create noisy-neighbor, cache isolation and blast-radius risks. Dedicated endpoints improve isolation and predictable latency. Use risk, traffic shape and model size to choose.

15.9 KPIs and operational metrics

- deployment success and rollback rate;
- time from approved release to healthy deployment;
- configuration drift and manual change count;
- startup/model-load time and cold-start frequency;
- canary exposure, decision duration and guardrail breaches;
- capacity headroom, autoscaling reaction and queue depth;
- percentage of releases with verified rollback;
- regional replication lag and failover readiness;
- endpoint/job availability and dependency health;
- infrastructure cost before and after traffic activation;
- edge fleet adoption, failed update and rollback rates.

15.10 Security, governance, compliance and risk controls

- signature/provenance verification at admission and runtime startup;
- deploy only from approved registries using digest pins;
- private endpoints, network segmentation and deny-by-default egress;
- workload identity, least privilege and short-lived secrets;
- separate production deployer from developer identities;
- region/residency policy and tenant isolation;
- non-root, read-only, minimal runtime images and restricted system calls;
- encrypted storage and mTLS where required;
- approved logging/redaction configuration before traffic;
- production access via audited just-in-time or break-glass process;
- policy-controlled rollout for high-risk cohorts;
- recall mechanism for compromised model or provider.

15.11 Common failure modes and troubleshooting

Symptom	Likely cause	Response
Pod/endpoint never ready	model download/load, memory, probe, dependency or driver mismatch	inspect startup phases; pre-stage artifact, right-size memory, validate runtime/hardware
Canary latency spikes	cold cache, insufficient capacity, feature/retrieval dependency	prewarm, isolate queue, scale dependency, compare trace spans
Rollback restores service but not quality	index/feature/prompt state not rolled back	treat system bundle as unit; version and restore dependent state
Multi-model noisy neighbor	shared GPU memory/queue/cache	admission limits, per-model concurrency, dedicated pool or priority
Streaming duplicates outputs	replay without idempotent sink	stable event/prediction ID, dedupe and transactional publication
Regional failover leaks/residency issue	replicated state or provider route not policy-aware	region-scoped data and routing; failover policy tests
Edge update bricks devices	incompatible runtime or interrupted install	compatibility precheck, atomic A/B partition and signed rollback

15.12 Scalability, reliability, performance and cost optimization

- use Little's Law for initial concurrency sizing: in-flight work ≈ arrival rate × service time;
- keep target utilization below saturation to absorb variance and failures;
- prewarm large models and common prompt prefixes for predictable launches;
- separate prefill/decode or use model parallelism only after profiling;
- batch offline work, dynamic-batch online requests and cap queue delay;
- autoscale on work-conserving signals such as queue, in-flight requests or tokens—not CPU alone;
- pool compatible small models where isolation allows; dedicate very large or critical models;
- place compute near data and users while respecting residency;
- use spot for restartable batch, not unprotected critical online serving;
- test scale-down, connection drain and cache invalidation;
- retain capacity for one failure domain in high-availability designs.

15.13 Production best practices and anti-patterns

Best practices: zero-traffic validation; immutable deployment; progressive cohorts; prewarmed capacity; state/version coupling; model-aware readiness; capacity/load test; rollback target; private networking; drift detection.

Anti-patterns: deploying from a notebook; model download on every cold start; autoscaling only on CPU; all-at-once cutover; canary without business/quality metrics; unversioned vector index; production secrets in environment files; active-active without data-consistency design.

15.14 Exit criteria

G5 launch succeeds when the approved release is healthy under production-equivalent traffic, telemetry and policy are verified, capacity and cost remain within bounds, canary/shadow guardrails pass, rollback works and the service owner accepts operational responsibility.

16. Stage 9 — Inference, serving, RAG and agent runtime

16.1 Purpose and objectives

Execute predictions, generations, retrieval and actions reliably under real traffic while enforcing identity, authorization, policy, quality, latency and budget. This stage is the operational heart of the platform and usually the largest recurring cost.

Objectives:

- expose stable, versioned interfaces independent of model vendor/runtime;
- route requests to fit-for-purpose models based on policy and economics;
- enforce input, context, output and action controls;
- maximize throughput without violating tail-latency or quality SLOs;
- maintain tenant isolation, data residency and traceability;
- degrade safely during model, data, feature, retrieval or tool failures;
- capture feedback and outcome linkage without excessive sensitive logging.

16.2 Core concepts and design principles

- **Inference is a queueing system.** Arrival variability, service-time distribution and concurrency determine tail latency and capacity.
- **The gateway is not the model.** Authentication, quotas, routing, versioning and policy belong in a controlled boundary.
- **Separate deterministic authority from probabilistic reasoning.** Models propose; policy engines authorize.
- **Context is untrusted and expensive.** Minimize, authorize, sanitize and budget prompts, retrieved text, memory and tool output.
- **Fallback must preserve business safety.** A cheaper model is not a valid fallback if it changes policy or quality beyond accepted bounds.
- **Cache identity includes tenant and version.** Missing tenant, policy, corpus or model dimensions creates leakage and stale behavior.
- **Structured interfaces reduce ambiguity.** Prefer schemas, typed tool calls and constrained outputs for machine-consumed results.

16.3 Fine-grained platform capabilities

Common serving plane

- API gateway, authentication, authorization, request IDs and schema validation;
- tenant quotas, rate limits, concurrency limits and budget enforcement;
- endpoint/model discovery, version negotiation and idempotency keys;
- routing by task, risk, quality, latency, region, availability and cost;
- synchronous, streaming and asynchronous interfaces;

- load balancing, retries, deadlines, hedging where safe and circuit breakers;
- request/response transformation and deterministic business rules;
- privacy-aware logging, tracing and sampling;
- usage metering, chargeback and abuse detection;
- fallback and human-escalation paths.

Predictive ML serving

- online feature lookup and freshness checks;
- preprocessing and postprocessing parity;
- calibration, threshold and decision-policy application;
- batch and streaming scoring;
- prediction IDs and outcome join;
- explanation/reason-code generation where required;
- champion/challenger or shadow comparison;
- model ensembles and deterministic aggregation.

LLM serving

- prompt registry and template rendering;
- context assembly and token budgeting;
- provider abstraction and model routing;
- streaming tokens and cancellation;
- structured output, constrained decoding and schema validation;
- dynamic/continuous batching, prefix/KV caching and quantized runtimes;
- model/tensor/pipeline parallelism and disaggregated execution where justified;
- moderation/input-output policy, DLP and secret redaction;
- retry/fallback rules aware of non-idempotent generation and cost;
- version-pinned managed API use or provider-change detection.

RAG runtime

- identity-aware query and source filters;
- query normalization, rewrite and intent classification;
- hybrid lexical/vector search and reranking;
- context selection, deduplication and source ordering;
- malicious-content and conflicting-source handling;
- citation generation with stable source IDs and offsets;
- freshness and index-version checks;
- semantic cache with tenant, policy, model and corpus keys;
- retrieval trace and component-level metrics.

Agent runtime

- explicit workflow/state machine with durable checkpoints;
- typed tool registry and deterministic permission mapping;
- sandboxed execution, network/filesystem limits and resource budgets;
- per-step and total token/time/cost/action limits;
- human approval and escalation nodes;
- transaction, idempotency and compensating-action support;
- memory scopes, retention and user consent;
- tool-result validation and instruction/data separation;
- loop detection, dead-letter state and replay/simulation;
- complete action audit with actor, model, policy and tool identity.

16.4 Inputs, outputs and artifacts

Inputs: authenticated request, tenant/user context, system release, features or authorized corpus, policy, tool grants, SLO and budget.

Outputs: prediction, generated response, citation set, queued job or controlled action; trace, usage and feedback identifier.

Operational artifacts:

- API and schema contract;
- model/prompt/router/policy configuration;
- feature and index references;
- tool registry and permission mappings;
- fallback matrix and error taxonomy;
- cache key/TTL policy;
- rate/quota and token-budget policy;
- traces, metrics, logs and cost records;
- prediction/action IDs and outcome links;
- runtime compatibility and performance profile.

16.5 System architecture and reference designs

LLM gateway contract

The gateway should expose a stable enterprise interface while recording underlying provider/model version. It authenticates, classifies data, checks allowed use, applies quotas, routes, normalizes telemetry and handles provider-specific errors. It must not conceal material behavior changes; provider/model transitions create explicit releases or regression events.

Model router

Use deterministic policy first: data residency, sensitivity, approved tasks, context size, required tools and service tier. Within the permitted set, optimize a measured objective such as expected task utility minus cost/latency. Routing models themselves require evaluation, monitoring and rollback.

Feature serving

Online features should be retrieved by stable entity keys with TTL/freshness metadata. If a critical feature is stale or missing, apply an explicit policy: reject, fallback model, default with confidence reduction or human review. Silent defaults are dangerous.

RAG context boundary

Retrieved documents are data. Delimit them, preserve source labels and instruct the model not to treat embedded content as authority. Deterministic policy filters sources and tool permissions; the model may rank or summarize but may not override access control.

Agent action boundary

Tool calls cross an execution broker. The broker validates schema, identity, permission, preconditions, risk class and approval. Irreversible actions require stronger confirmation, idempotency and postcondition verification. The model never receives raw long-lived secrets; the broker injects scoped credentials.

16.6 Technologies, frameworks and tooling

- predictive serving: KServe, Seldon, BentoML, Ray Serve, Triton or managed endpoints;
- LLM serving: vLLM, TGI, Triton, llama.cpp or managed APIs;
- API/gateway: enterprise API management, Envoy/Kong/Apigee-class systems or purpose-built LLM gateways;
- orchestration: deterministic application code, workflow engines and agent frameworks with durable state;
- feature stores and caches; vector/search databases and rerankers;
- policy: OPA or equivalent authorization/policy service;
- telemetry: OpenTelemetry with Prometheus/Grafana and log/trace backends;
- secrets/workload identity and sandbox/container isolation;
- service mesh only for requirements such as mTLS, traffic policy or deep observability that cannot be met more simply.

16.7 Roles and responsibilities

- **Service/application owner:** API contract, user experience, fallback and outcomes.
- **ML/LLM engineer:** runtime behavior, model/prompt/router and quality.
- **Knowledge engineer:** retrieval, corpus, citations and freshness.
- **Platform inference team:** gateway, runtimes, scheduling, quotas and upgrades.
- **SRE:** SLOs, capacity, incidents and resilience.
- **Security:** data classification, policy, tool sandbox, abuse and egress.
- **Data/feature owner:** online freshness and semantics.
- **FinOps:** rate cards, routing economics and budget controls.
- **Risk/validator:** allowed tasks, autonomy and human-oversight controls.

16.8 Decision points and trade-offs

Self-hosted versus managed model API

Self-hosting provides version, data-path and optimization control but requires accelerator capacity and 24×7 operations. Managed APIs provide rapid access and elasticity but introduce provider change, rate limits, data-flow and concentration risk. Use a common gateway and maintain workload-specific exit plans.

Model routing

A single model is simpler and more predictable. Routing can reduce cost and improve availability but adds evaluation surface, inconsistent behavior and debugging complexity. Begin with policy routing; add learned/semantic routing only when measured benefit exceeds complexity.

Caching

Exact/prefix/KV caches are safer than semantic response caches. Semantic caches can return stale or cross-context answers; keys must include tenant, model, prompt, policy, corpus and authorization context, with conservative TTL and invalidation.

Retries

Retries improve transient reliability but can multiply cost and duplicate side effects. Use bounded retries within deadlines; do not retry non-idempotent tool actions without an idempotency key and state check.

Agents versus deterministic workflows

Use deterministic workflow steps whenever the path is known. Use model choice only where ambiguity or unstructured inputs require it. Constrain autonomy by action class, not by subjective confidence alone.

16.9 KPIs and operational metrics

Common: request rate; p50/p95/p99 latency; error and timeout rate; queue delay; concurrency; saturation; availability; fallback rate; cost/request and cost/successful outcome.

Predictive: feature read latency/freshness/missing rate; inference latency; prediction distribution; explanation generation; champion/challenger delta.

LLM: time to first token; time per output token; tokens/second; input/output tokens; context utilization; truncation; cache hit; batching efficiency; provider/model mix; refusal and schema-validity rate.

RAG: retrieval latency; zero-result; recall proxy; rerank latency; context tokens; citation coverage/freshness; authorization denials; index version.

Agents: steps/task; tool success; policy denial; human intervention; loop/timeout; duplicate/compensation; action failure; task completion; cost and elapsed time per successful task.

16.10 Security, governance, compliance and risk controls

- authenticate every request and propagate workload/user identity;
- authorize model, corpus and tool separately;
- classify/minimize data before external provider calls;
- enforce private connectivity/egress allowlists and provider contracts;
- input/output schema validation, DLP, secret and content controls;
- prompt injection defenses using instruction/data separation, source trust and deterministic authorization;
- no direct model access to databases, shell, email, payments or production APIs;
- sandbox tools with scoped credentials, network and filesystem;
- human approval for defined action classes;
- tenant-aware caches, vector indexes, logs and memory;
- rate, token, spend and recursion limits to prevent unbounded consumption;
- retention/redaction of prompts and outputs according to purpose;
- immutable audit for high-impact decisions/actions;
- detect model extraction, abuse, anomalous probing and credential misuse.

16.11 Common failure modes and troubleshooting

Symptom	Likely cause	Response
Tail latency explodes	saturation, long prompts, queueing, dependency slowdown	inspect trace/queue/token distribution; shed load, cap context, scale bottleneck
Throughput falls despite free GPU	batching/config, CPU tokenizer, network or single-thread stage	profile prefill/decode/tokenization and scheduler; batch/tune/pipeline stages
Cost spike	prompt growth, retry storm, model-route change, agent loop	halt offending route/workflow, enforce budgets, compare release and traffic mix
Hallucination rises	model/prompt/provider/index change or retrieval miss	correlate release/index/provider, run component eval, rollback or tighten grounding
Wrong tenant data	cache/index/session key or auth propagation bug	contain, invalidate, rotate, investigate scope; enforce tenant in every key and test
Tool action duplicated	retry without idempotency or lost state	stop action path, reconcile external state, implement idempotency/transaction log
Over-refusal	policy/filter/model update	segment denied valid requests, compare release and tune approved controls
Provider outage	upstream failure/rate limit	circuit break, queue/degrade, route to approved fallback, communicate limitations

Symptom	Likely cause	Response
Agent loops	ambiguous goal, repeated tool error, no state guard	max steps/time, detect repeated state, escalate, fix workflow/tool contract

16.12 Scalability, reliability, performance and cost optimization

- model capacity from real service-time and token distributions, not average requests;
- dynamic/continuous batching bounded by latency SLO;
- separate request admission from execution; shed or queue before saturation;
- prefix/KV cache for repeated authorized context; measure memory trade-off;
- quantize or route to SLMs for bounded tasks after non-inferiority evaluation;
- use streaming responses to improve perceived latency while preserving cancellation;
- set context budgets and retrieve/rerank fewer, better sources;
- batch embeddings and noninteractive generation;
- use tensor/pipeline parallelism for models that do not fit one device; avoid distributed overhead for smaller models;
- isolate high-priority production from best-effort workloads;
- maintain fallback capacity and test provider/model failover;
- autoscale on queue/in-flight/tokens and account for model load time;
- optimize for cost per successful task, including retries and review.

16.13 Production best practices and anti-patterns

Best practices: stable enterprise API; deterministic policy before model; typed outputs/tools; tenant-aware caches; deadline propagation; bounded retries; explicit fallback; component traces; token budgets; safe tool broker; previous known-good route.

Anti-patterns: direct app-to-provider calls; prompt strings embedded across codebases; model decides its own permissions; open internet tools; semantic cache without authorization key; unlimited context/steps; retrying financial actions; logging every prompt indefinitely; one giant agent for a deterministic process.

16.14 Exit criteria

The runtime is production-ready when it meets service and quality SLOs under representative load, enforces identity/policy/tenancy, has bounded resource and agent behavior, produces end-to-end traces, supports safe fallback and rollback, and exposes unit economics and outcome linkage.

17. Stage 10 — Monitoring, observability and value measurement

17.1 Purpose and objectives

Detect when the AI system, its inputs or its business context leave approved operating bounds; explain why; trigger proportionate response; and verify that value persists. Monitoring must cover infrastructure, data, model, LLM/RAG/agent behavior, security, cost and business outcomes.

Objectives:

- establish end-to-end traces from request to feature/retrieval/model/tool/result;
- define service, quality, safety, data and value SLOs;
- detect availability, latency, drift, degradation, abuse and cost anomalies;
- connect delayed outcomes and user feedback to release/prediction IDs;
- support diagnosis without unnecessary sensitive-data retention;
- trigger rollback, containment, investigation, retraining or recertification;

- provide operational, executive and audit views from consistent telemetry.

17.2 Core concepts and design principles

- **Observability explains unknown failures; monitoring watches known risks.** Use traces, structured logs, metrics, profiles and lineage together.
- **No labels does not mean no monitoring.** Use input, output, confidence, retrieval and business proxies, then reconcile when labels arrive.
- **Drift is a signal, not a verdict.** Data drift may be harmless; performance can degrade without obvious drift.
- **Quality SLOs need windows.** Define sample size, delay, segments and uncertainty before paging.
- **Telemetry is sensitive data.** Prompts, features, documents and outputs require minimization, redaction and access control.
- **Alert on user impact and actionable symptoms.** Dashboards are not alerts; every alert has an owner and runbook.
- **Monitoring must monitor itself.** Missing traces, stale dashboards or broken feedback joins are control failures.

17.3 Fine-grained platform capabilities

Telemetry foundation

- OpenTelemetry-style context propagation across gateway, retrieval, model, tools and downstream apps;
- metrics, traces, structured logs and profiles with common release/tenant/request attributes;
- sampling policies, redaction and secure trace access;
- time-series dashboards, alerting, event correlation and status pages;
- synthetic transactions and black-box probes;
- telemetry pipeline health and retention controls;
- immutable audit stream separated from operational logs.

Infrastructure and service monitoring

- availability, traffic, latency, errors and saturation;
- CPU/GPU/memory/network/storage, accelerator errors and thermal/power signals;
- queue depth, wait, batch size, model load and cache utilization;
- dependency and provider health, rate limits and regional status;
- autoscaling events, cold starts and capacity headroom;
- job/pipeline success, checkpoint and schedule adherence.

Data and feature monitoring

- schema, volume, freshness, missingness and validity;
- distribution shift, category emergence and outliers;
- feature materialization lag, online/offline skew and default rate;
- label delay, feedback coverage and join success;
- corpus/index freshness, parse failures, deletion and ACL consistency.

Model and decision monitoring

- prediction/output distribution, confidence and calibration proxies;
- performance when labels arrive, by segment and threshold;
- residual/error analysis and concept-drift evidence;
- fairness/impact indicators and override/appeal outcomes;
- explanation/reason-code availability;
- champion/challenger comparison and policy-decision distribution.

LLM/RAG/agent monitoring

- model/provider/prompt/index/tool versions per trace;
- token counts, TTFT, output rate, truncation and context composition;

- groundedness/citation/relevance proxies and sampled human review;
- refusal, policy violation, unsafe-output and injection-detection rates;
- retrieval zero-result, source mix, stale/denied sources and citation correctness;
- agent step count, loop, tool errors, denied actions, intervention and compensation;
- model/router fallback and provider change;
- semantic/prefix cache hit and invalidation.

Business and value monitoring

- business KPI and guardrails tied to cohorts and releases;
- adoption, completion, cycle time, conversion, loss avoided or quality improvement;
- human review effort, correction/override and escalation;
- user feedback, complaint, appeal and abandonment;
- realized benefit versus forecast and full run cost;
- causal experiment or counterfactual method where feasible.

17.4 Inputs, outputs and artifacts

Inputs: SLOs, baselines, segment definitions, telemetry schema, release IDs, outcome/feedback events, risk controls, cost data and alert policy.

Outputs: dashboards, alerts, anomaly/drift events, incident tickets, retraining/recertification requests, value reports and audit evidence.

Artifacts:

- service level indicators/objectives and error budgets;
- telemetry data dictionary and redaction policy;
- dashboard/alert definitions as code;
- drift/performance baselines and thresholds;
- synthetic probe and canary cases;
- monitoring model/version where ML detection is used;
- feedback/outcome join specification;
- alert runbooks and escalation;
- periodic service health and value review;
- control-evidence snapshots.

17.5 System architecture and reference workflow

1. Request receives a trace and prediction/action ID at ingress.
2. Each service emits spans and metrics with tenant, release, environment and dependency identity; sensitive payloads are excluded or separately protected.
3. Telemetry collectors apply redaction, sampling, aggregation and routing.
4. Real-time alerts cover availability, safety/security hard limits and severe cost anomalies.
5. Near-real-time jobs compute data/feature/retrieval/output drift and operational quality proxies.
6. Delayed labels and business outcomes join to prediction IDs in a controlled analytics zone.
7. Scheduled evaluation computes performance, calibration, segment and value results with confidence.
8. Policy engine compares results to SLOs and triggers ticket, traffic reduction, rollback, review or retraining.
9. Service owner conducts periodic review of value, incidents, provider changes, access and residual risk.

17.6 Technologies, frameworks and tooling

- OpenTelemetry for vendor-neutral telemetry and context propagation;
- Prometheus and Alertmanager, Grafana and log/trace backends;
- cloud-managed observability where appropriate;

- model/data observability platforms or custom statistical pipelines;
- stream processing for near-real-time metrics;
- data warehouse/lakehouse for delayed outcome joins and cohort analysis;
- incident management/on-call platforms;
- security information and event management for threat correlation;
- FinOps/cost allocation platforms;
- evaluation service for periodic sampled regression.

17.7 Roles and responsibilities

- **Service owner:** quality/value SLO and corrective decision.
- **SRE/on-call:** availability, latency, saturation and incident response.
- **ML/LLM engineer:** model/RAG/agent diagnostics and monitoring logic.
- **Data/knowledge owner:** source, feature and corpus quality.
- **Security operations:** abuse, exfiltration, anomalous access and prompt/tool threats.
- **Risk/compliance/model validation:** control thresholds and periodic review.
- **Product/analytics:** business outcomes, experiments and user feedback.
- **FinOps:** cost anomaly and value efficiency.
- **Platform observability team:** telemetry schema, collectors, storage and reliability.

17.8 Decision points and trade-offs

Payload logging versus privacy

Full prompts/features simplify diagnosis but create privacy, security and retention risk. Prefer metadata, hashes, redacted samples and user-consented or incident-triggered capture. For high-risk debugging, use controlled escrow with limited access and expiry.

Fixed thresholds versus statistical detection

Fixed SLOs are interpretable and auditable. Statistical detectors adapt to seasonality but can be noisy and themselves drift. Use fixed hard guardrails plus statistical warning signals with human triage.

Real-time versus delayed quality

Real-time quality proxies enable containment; delayed labels provide truth. Define leading and lagging indicators and expected label delay. Do not page on weak proxies unless action is safe and reversible.

Sampling

Trace and human-review sampling controls cost and privacy. Use tail/risk-based sampling to retain errors, safety events, rare segments and new releases; maintain unbiased samples for performance estimation.

17.9 KPIs and operational metrics

- SLO attainment and error-budget burn;
- mean time to detect, acknowledge, mitigate and recover;
- telemetry completeness, trace correlation and monitoring-pipeline availability;
- false-positive/false-negative alert rates and alert volume per on-call;
- data/feature/index freshness and drift event rate;
- model performance/calibration by segment and time;
- LLM groundedness/refusal/policy and agent completion/intervention trends;
- feedback coverage and outcome-join success;
- time from drift/performance breach to decision;

- cost anomaly detection time and avoided spend;
- realized business value and cost per successful outcome;
- overdue monitoring reviews and unresolved control breaches.

17.10 Security, governance, compliance and risk controls

- least-privilege access to telemetry, with separate roles for payload and aggregate views;
- redaction/tokenization before logs leave workload boundary;
- encryption, retention, deletion and regional storage policy;
- immutable audit for consequential decisions and agent actions;
- monitor unauthorized provider/model/tool use and policy bypass;
- alert on missing telemetry, disabled controls and configuration drift;
- protect monitoring rules and baselines through code review and signatures;
- validate monitoring models and avoid hidden discriminatory proxies;
- preserve evidence for complaints, appeals and incidents;
- establish legal hold and breach-notification integration;
- record human overrides and recourse outcomes for high-impact systems.

17.11 Common failure modes and troubleshooting

Symptom	Likely cause	Corrective action
No quality alert despite complaints	missing labels/segment, weak proxy, join failure	audit telemetry/outcome pipeline, add complaint cases and leading signals
Drift alerts constantly	seasonal change, unstable baseline, too many dimensions	segment/time-aware baselines, multiple-testing control, actionable thresholds
Trace cannot explain result	missing version/context/tool spans	enforce telemetry contract at admission and SDK level
Monitoring cost spikes	high-cardinality labels, full payloads, 100% traces	cardinality budgets, sampling, aggregation and payload minimization
Dashboard says healthy during outage	monitor blind spot or same failure domain	external synthetic probe and metamonitoring
Performance falls only for one group	aggregate dashboards	mandatory segment views and minimum sample/confidence rules
LLM safety metric changes suddenly	judge/provider/prompt version change	version and evaluate monitor itself; compare human sample
Alert fatigue	cause-based noisy alerts/no runbook	symptom/error-budget alerts, dedupe/inhibit, remove nonactionable pages

17.12 Scalability, reliability, performance and cost optimization

- define a low-cardinality metric schema; keep high-cardinality detail in traces/logs;
- sample adaptively by error, risk, tenant, release and rarity;
- aggregate tokens, latency and cost at gateway to reduce duplicate telemetry;
- separate security/audit retention from operational debugging retention;
- precompute common segment metrics in streaming or incremental jobs;
- use approximate/sketch algorithms for high-volume distributions where validated;
- backpressure telemetry rather than blocking critical inference, while buffering key audit events durably;
- replicate monitoring across failure domains and use external probes;
- budget telemetry cost as a percentage of service cost and review value;
- maintain raw outcome data long enough to recompute corrected metrics under policy.

17.13 Production best practices and anti-patterns

Best practices: end-to-end context; release IDs; SLO/error budget; layered leading/lagging indicators; segment monitoring; metamonitoring; privacy-aware sampling; incident-derived tests; business outcome linkage.

Anti-patterns: infrastructure-only monitoring; logging all prompts forever; drift equals retrain; aggregate-only accuracy; alerts without owner/runbook; quality computed against a moving unversioned label definition; high-cardinality tenant/user IDs in metrics; provider dashboard as sole evidence.

17.14 Exit criteria

Operational monitoring is complete when all critical dependencies and behavior dimensions have owned SLOs, telemetry is complete and privacy-controlled, alerts are actionable and tested, outcomes join to releases, drift/quality response paths exist, and business value is reviewed on a defined cadence.

18. Stage 11 — Incident response, resilience and continuity

18.1 Purpose and objectives

Contain harm, restore acceptable service, preserve evidence and prevent recurrence when an AI system fails. AI incidents include conventional outages plus incorrect decisions, data or privacy leakage, unsafe generations, model/provider compromise, poisoning, prompt injection, agent action errors and cost runaway.

Objectives:

- detect and classify incidents by user/business harm, not only infrastructure severity;
- stop or bound unsafe behavior quickly;
- maintain clear command, communication and decision rights;
- preserve forensic evidence and comply with notification obligations;
- restore via rollback, fallback, traffic reduction or manual process;
- analyze technical, data, model, process and governance causes;
- feed lessons into tests, controls, training and architecture;
- meet RTO/RPO and business-continuity objectives.

18.2 Core concepts and design principles

- **Safety over availability when required.** A healthy endpoint producing harmful decisions is unavailable in the business sense.
- **Containment options are predesigned.** Kill switches, model/provider deny rules, tool revocation and traffic controls must exist before incidents.
- **Evidence is perishable.** Preserve release, prompts/inputs under lawful controls, retrieval, tool calls, policy decisions, logs and identities.
- **Incident command is role-based.** The model team does not unilaterally decide legal, privacy or customer communication.
- **Rollback may be insufficient.** Source data, feature store, index, cache or external action may require repair or compensation.
- **Recovery includes trust.** User notification, correction, appeal and remediation can be part of restoration.
- **Blameless does not mean control-free.** Focus on system causes while assigning corrective ownership and deadlines.

18.3 Fine-grained platform capabilities

- incident severity matrix covering availability, quality, safety, privacy, security, financial and regulatory impact;
- automated paging and incident-room creation;

- service/model/prompt/index/provider kill switches;
- traffic shaping, tenant isolation and feature flags;
- agent tool/credential revocation and action freeze;
- rollback to known-good system bundle;
- fallback rules/manual queues and degraded modes;
- immutable evidence capture and forensic snapshots;
- dependency and lineage lookup from affected output to source/release;
- customer/user notification and correction workflow;
- model/provider recall and blocklist;
- backup/restore, regional failover and disaster-recovery orchestration;
- post-incident review, problem management and control/test updates;
- incident simulation and game days.

18.4 Inputs, outputs and artifacts

Inputs: alert, complaint, security finding, provider notice, audit issue or observed harm; release and trace data; runbook and severity policy.

Outputs: contained incident, restored/degraded service, notifications, corrected actions/data, root-cause analysis and preventive work.

Artifacts:

- incident timeline, severity and commander;
- affected releases, tenants, users, decisions and data;
- containment actions and approvals;
- forensic evidence references and chain of custody;
- regulatory/legal assessment and notifications;
- recovery/rollback/failover test results;
- corrected outputs or compensating actions;
- post-incident review and root/contributing causes;
- action items, owners and due dates;
- new evaluation cases, monitors and policy changes.

18.5 System architecture and reference workflow

1. Alert or report creates incident with service/release context.
2. Incident commander establishes severity, roles and communication cadence.
3. Triage distinguishes infrastructure outage, data/feature issue, model/system regression, abuse/security, privacy, provider or business-process failure.
4. Containment applies the smallest reliable action: block route/tool/model, reduce cohort, disable feature, revert bundle, isolate tenant or switch to manual processing.
5. Evidence service snapshots relevant configuration, traces, artifacts and access events under legal/privacy policy.
6. Recovery restores a known-good state, repairs derived data/index/cache and verifies with synthetic and targeted tests.
7. Impact analysis identifies affected decisions/actions and required correction, notification or recourse.
8. Post-incident review creates causal map, not merely “human error,” and updates tests, monitoring, runbooks and architecture.
9. Closure requires verified corrective actions or accepted residual risk.

18.6 Technologies, frameworks and tooling

- on-call and incident management systems;
- feature flags, API gateway and GitOps rollback;

- registry recall/quarantine and policy blocklists;
- SIEM, forensic logging and immutable object storage;
- lineage/catalog search and trace backends;
- backup/restore, replication and traffic-management systems;
- workflow engines for correction/notification and agent compensation;
- chaos/failure-injection tools in nonproduction or controlled production;
- status pages and communication tooling.

18.7 Roles and responsibilities

- **Incident commander:** authority, coordination and closure.
- **Service owner:** business impact, fallback and user/process response.
- **SRE/operations:** technical containment and recovery.
- **ML/LLM/data/knowledge engineers:** diagnosis and artifact/data repair.
- **Security incident response:** adversary, credentials, exfiltration and forensics.
- **Privacy/legal/compliance:** notification, preservation and regulator/customer obligations.
- **Communications/customer support:** accurate external and internal messaging.
- **Risk/model validation:** decision impact and recertification requirement.
- **Executive sponsor:** severe risk acceptance and business continuity.

18.8 Decision points and trade-offs

Rollback versus forward fix

Rollback is usually fastest if the previous bundle and dependent state are compatible. Forward fix may be required for data corruption, security vulnerabilities or irreversible external changes. Prefer containment first, then choose based on time-to-safe-state and evidence.

Availability versus quality/safety

For assistive low-risk systems, degraded output may be acceptable with disclosure. For consequential decisions or autonomous actions, disable or route to human when confidence in safety is lost. Predefine this policy.

Evidence capture versus data minimization

Capture enough to investigate and meet obligations, but restrict scope, access and retention. Use legal hold and controlled forensic stores rather than expanding routine logging after an incident.

Regional failover

Failover can violate data residency, overwhelm a secondary region or route to a different model/provider. Execute only approved failover plans with current capacity and policy checks.

18.9 KPIs and operational metrics

- mean time to detect, acknowledge, contain, recover and correct;
- incident frequency and severity by failure domain;
- percentage with release/trace identity and complete evidence;
- rollback/failover success and time;
- user/decision impact and correction completion;
- repeat-incident rate and action-item aging;
- false positive/negative of incident alerts;
- notification timeliness and breach of obligations;
- backup restore and DR exercise success;

- kill-switch/tool-revocation execution time;
- error-budget consumption and downtime/business loss.

18.10 Security, governance, compliance and risk controls

- predefined security/privacy/AI incident taxonomy and notification routes;
- least-privilege incident access with audited elevation;
- chain of custody and immutable evidence;
- legal hold, breach and regulator/customer notification procedures;
- model/provider/artifact recall capability;
- compromised credential/signing-key revocation;
- human recourse, correction and appeal for consequential outputs;
- post-incident independent review for high-impact events;
- mandatory recertification after material incidents or control failure;
- periodic tabletop, red-team and disaster-recovery exercises;
- protect incident data from broad distribution and secondary use.

18.11 Common failure modes and troubleshooting

Failure mode	Why it fails	Corrective approach
Only HTTP outage severity exists	harmful quality incidents are missed	multidimensional severity and business-health probes
Kill switch disables wrong scope	poor tenant/model mapping	test scoped controls and maintain dependency/release inventory
Rollback repeats incident	corrupt feature/index/cache persists	restore full system state and invalidate derived artifacts
No affected-user list	missing prediction/action IDs and lineage	make outcome identity mandatory and reconstruct from audit/trace
Provider changes without notice	unpinned model or weak vendor controls	gateway detection, regression probes and alternate route
Agent cannot compensate	external API lacks idempotency/transaction record	brokered actions, pre/post state and compensating workflow
Evidence contains excessive PII	emergency logging bypass	forensic capture policy, restricted vault and expiry
PIR blames operator	weak system analysis	causal analysis across design, process, alerting, workload and incentives

18.12 Scalability, reliability, performance and cost optimization

- maintain automated global and scoped kill switches with low control-plane dependency;
- replicate known-good artifacts, configuration and critical metadata;
- design regional/cell isolation so containment does not require enterprise-wide shutdown;
- keep manual fallback capacity proportional to critical process demand;
- regularly test backup restore including registry, feature/vector state and audit mappings;
- use chaos/game days to validate retry, circuit breaker, provider and region failure;
- retain incident-ready diagnostic sampling without full-time full-payload cost;
- establish vendor escalation and capacity arrangements before critical use;
- prioritize recovery work by harm reduction, not system component ownership.

18.13 Production best practices and anti-patterns

Best practices: multidimensional severity; tested kill switch; full-bundle rollback; action freeze; immutable evidence; affected-output identity; user correction/recourse; game days; incident-derived regression cases.

Anti-patterns: model team handles incidents alone; relying on provider status page; rollback only weights; no manual process; deleting logs before impact analysis; enabling broad debug logging indefinitely; closing when endpoint is up but bad decisions remain uncorrected.

18.14 Exit criteria

An incident closes only when service and business process are safe, affected users/actions are addressed, evidence and notifications are complete, root/contributing causes are understood, preventive actions are owned and required recertification has been initiated or completed.

19. Stage 12 — Feedback, retraining, recertification and retirement

19.1 Purpose and objectives

Use operational evidence to improve or deliberately end the system. Feedback must be converted into reliable labels, evaluation cases, product changes or policy updates; it must not become an uncontrolled online-learning stream.

Objectives:

- collect explicit and implicit feedback with context and consent;
- distinguish model failure from data, UX, policy and process failure;
- curate new labels and incident-derived test cases;
- decide whether to retrain, retune, change retrieval/prompt, redesign workflow or do nothing;
- recertify systems on schedule and after material changes;
- retire low-value, unsafe, obsolete or unsupported systems cleanly;
- preserve required evidence and delete unnecessary data/artifacts.

19.2 Core concepts and design principles

- **Feedback is biased.** Users who respond are not representative; implicit signals may reflect position, incentives or interface design.
- **Do not learn directly from abuse.** Production inputs can be poisoned or manipulated. Curate and validate before training.
- **Retraining is one intervention.** Drift may require source repair, threshold change, policy update or process redesign.
- **Recertification is lifecycle control.** Approval expires based on risk, change and evidence.
- **Retirement is a release.** Dependencies, users, data, endpoints, credentials and records need an ordered plan.
- **Value can decay without model decay.** Process, policy, market or user behavior may make a once-useful system unnecessary.

19.3 Fine-grained platform capabilities

- feedback API and UI with prediction/trace/release identity;
- user correction, rating, appeal and complaint workflows;
- implicit outcome ingestion with causal/selection-bias safeguards;
- triage taxonomy: data, feature, retrieval, model, prompt, policy, tool, UX or process;
- secure review and annotation queues;
- active-learning candidate selection with diversity and risk constraints;
- incident/feedback-to-evaluation-case promotion;
- retraining trigger engine with cooldown, minimum evidence and budget;
- scheduled and event-driven recertification;
- provider/model/license/end-of-life monitoring;
- shadow/challenger evaluation and value review;

- deprecation notices, consumer discovery and migration tracking;
- endpoint/key/data/index/artifact cleanup and archival evidence.

19.4 Inputs, outputs and artifacts

Inputs: feedback, corrections, outcomes, incidents, monitoring trends, provider notices, data/model changes, business value and control review.

Outputs: prioritized backlog, new labels/evaluation cases, retraining or redesign decision, recertification, deprecation or retirement.

Artifacts:

- feedback record and provenance;
- triage classification and disposition;
- curated label/data additions;
- updated evaluation set/rubric;
- retraining request and trigger rationale;
- change-impact assessment;
- recertification report and approvals;
- deprecation/retirement plan and notices;
- dependency migration evidence;
- archival, deletion, key revocation and inventory closure.

19.5 System architecture and reference workflow

1. Application records feedback against prediction/action and release ID.
2. Feedback pipeline validates identity, removes spam/abuse, applies privacy policy and routes high-risk complaints immediately.
3. Triage links feedback to traces and categorizes root domain.
4. Selected cases enter secure annotation/adjudication; confirmed failures become evaluation tests before training data.
5. Decision service compares intervention options: source/feature fix, prompt/index/policy change, model retraining, workflow/UX change or no action.
6. Approved retraining freezes a versioned data window and re-enters Stage 4; candidate follows all normal gates.
7. Periodic recertification reviews value, incidents, drift, access, provider/license, human oversight and control effectiveness.
8. Retirement workflow discovers consumers, migrates/blocks new use, archives required evidence, removes access and computes final value.

19.6 Technologies, frameworks and tooling

- product feedback and customer-support systems integrated with trace IDs;
- data/label workflow and annotation tools;
- issue/portfolio management;
- pipeline/orchestration and model registry;
- monitoring/evaluation platforms;
- catalog/lineage for dependency discovery;
- contract/license and vendor-risk monitoring;
- records management, deletion and key-revocation systems;
- feature flags and deprecation telemetry.

19.7 Roles and responsibilities

- **Product owner:** feedback priority, value decision and retirement.
- **Domain experts/support:** complaint interpretation and user recourse.

- **Data/model/knowledge engineers:** root cause and candidate intervention.
- **Annotators/validators:** curated labels and independent tests.
- **Risk/privacy/legal/security:** material-change assessment and recertification.
- **Platform:** trigger workflow, registry lifecycle and cleanup automation.
- **SRE:** deprecation traffic, dependency and shutdown safety.
- **Finance/FinOps:** value and retirement economics.
- **Records/data owner:** retention, archival and deletion.

19.8 Decision points and trade-offs

Retrain versus repair

Retrain when the target relationship or representation has changed and reliable new labels exist. Repair data pipelines when quality/freshness is wrong; adjust threshold/calibration when decision costs changed; update retrieval when knowledge changed; redesign UX when users misunderstand outputs.

Online learning versus controlled batches

Online learning adapts quickly but increases poisoning, instability and rollback complexity. Use only for bounded, well-instrumented problems with robust safeguards. Most enterprise systems should accumulate feedback, curate it and release controlled candidates.

Recertification cadence

Use risk and change: high-impact or rapidly changing systems may need quarterly or continuous evidence review; low-risk stable systems may be annual. Material incidents, provider/model changes, new data use, new autonomy or major population shift trigger out-of-cycle review.

Retirement versus maintenance

Retire when value falls below operating/control cost, a simpler method suffices, data/license support ends, risk becomes unacceptable or a platform capability supersedes the service. Avoid perpetual “temporary” systems.

19.9 KPIs and operational metrics

- feedback coverage, correction rate and time to disposition;
- complaint/appeal volume and resolution time;
- confirmed failure rate by category and segment;
- percentage of incidents/feedback converted to regression tests;
- label acceptance and active-learning yield;
- trigger-to-candidate and candidate-to-value lead time;
- retraining frequency, unnecessary retrain and regression rate;
- recertification on-time rate and exception age;
- value trend, usage and cost per outcome;
- deprecation consumer migration and residual traffic;
- retired services, spend avoided and cleanup completion;
- deletion/archival SLA compliance.

19.10 Security, governance, compliance and risk controls

- verify consent/purpose for feedback and outcome reuse;
- prevent production data from entering training automatically;
- detect poisoning, spam and coordinated manipulation;
- restrict feedback payload access and redact sensitive content;

- preserve appeals and corrections for consequential decisions;
- material-change policy determines reevaluation scope;
- recertify ownership, access, data/provider contracts, human oversight and monitoring;
- block expired approvals or unsupported provider/model versions;
- retirement revokes identities, secrets, network, tools and endpoints;
- propagate deletion and legal hold across training data, indexes and logs;
- retain minimum evidence required to explain past decisions.

19.11 Common failure modes and troubleshooting

Symptom	Likely cause	Corrective action
Feedback improves metric but hurts users	selection bias or gaming	stratify, causal experiment, representative sampling and guardrails
Same incident recurs after retrain	root cause was data/process/policy	causal triage and non-model remediation
Training data poisoned	direct ingestion of user content	quarantine, provenance, anomaly/abuse detection and human curation
Recertification becomes paperwork	no live evidence or decision rights	automate evidence, review residual risk/value and allow retirement
Deprecated endpoint never dies	unknown consumers or no enforcement	usage telemetry, owner notices, migration tooling, staged deny
Deletion incomplete	missing lineage to embeddings/caches/backups	full derived-data inventory, tombstones and restore-time deletion replay
Provider EOL surprises team	no lifecycle monitoring	contract/catalog alerts, portability test and fallback roadmap

19.12 Scalability, reliability, performance and cost optimization

- sample feedback strategically, prioritizing uncertainty, high impact, diversity and novel failures;
- automate triage suggestions but keep deterministic routing for severe complaints;
- coalesce retraining triggers and require minimum new information;
- use incremental training only when reproducibility and rollback remain strong;
- maintain reusable evaluation cases and labeled datasets as enterprise assets;
- quantify maintenance/control cost to identify retirement candidates;
- automate dependency discovery and access cleanup;
- archive large artifacts to cold storage or cryptographically erase where policy permits;
- test that retired aliases/endpoints cannot be invoked and old credentials are invalid;
- keep previous decision evidence queryable without maintaining live service.

19.13 Production best practices and anti-patterns

Best practices: trace-linked feedback; human-curated labels; failure taxonomy; incident cases become tests; trigger cooldown; material-change analysis; periodic value review; enforced deprecation; verifiable cleanup.

Anti-patterns: training directly on thumbs-up/down; retraining on every drift alert; recertification by copy/paste; retaining systems because “someone may use it”; deleting registry records before audit retention; leaving provider keys and endpoints active after retirement.

19.14 Exit criteria

A system remains approved only while value, controls, ownership, provider support and operational evidence remain current. A retired system has no active traffic or credentials, consumers have migrated, required records are archived, derived data is handled per policy and the enterprise inventory is closed.

20. AI governance, model risk and compliance

20.1 Governance objective

Governance should make acceptable delivery repeatable and unacceptable behavior difficult. It defines decision rights, risk appetite, control requirements, evidence and recourse across the AI-system lifecycle. It is not a review board that sees systems only before launch.

Use a management-system approach: establish policy and accountability, map context and harms, measure system behavior, manage residual risk and continuously improve. NIST AI RMF structures these activities as **Govern, Map, Measure and Manage** [R1]; ISO/IEC 42001 defines an AI management system with continual improvement [R3]. The enterprise control model should align to these concepts while integrating existing information-security, privacy, model-risk, quality and service-management systems.

20.2 Governance structure and lines of accountability

Governing body

An executive AI governance council approves risk appetite, prohibited uses, high-impact exceptions, enterprise policy and material incidents. It should include business, technology, security, privacy, legal/compliance, risk, HR where relevant and customer/operations representation. It does not approve every low-risk release.

First line: product and engineering

Business and technical owners identify risk, implement controls, operate the service and remain accountable for outcomes. Platform automation helps them produce evidence but does not transfer ownership.

Second line: risk, privacy, security and compliance

Defines policy, advises teams, performs challenge, approves defined high-risk decisions and monitors control effectiveness. Independence must be sufficient to reject or constrain a release.

Third line: internal audit

Assesses whether governance design and controls operate effectively. Audit should sample system evidence, release lineage, exceptions, access, monitoring and incidents rather than relying only on policy documents.

External assurance

Regulators, customers, certification bodies and independent assessors may require evidence. Maintain exportable evidence packages and map controls once to multiple obligations.

20.3 Enterprise governance capabilities

- authoritative AI-system and model/provider inventory;
- use-case and risk classification with documented rationale;
- algorithmic/AI impact and privacy assessments;
- policy library, control profiles and controls-as-code;
- owner, approver, validator and human-oversight assignments;
- data/model/provider/license due diligence;
- evaluation and release approval workflow;
- model/system/data/prompt cards and user disclosures;
- exception, residual-risk and expiry management;
- periodic recertification and material-change assessment;

- complaint, appeal, correction and recourse process;
- incident, regulator/customer notification and recall;
- evidence retention, audit access and control testing;
- regulatory profile mapping by jurisdiction and domain;
- AI literacy and role-based training.

20.4 Policy set

At minimum establish policies for:

1. **Acceptable and prohibited use.** Bound use cases, populations, decisions, surveillance, manipulation, biometric or other sensitive applications.
2. **AI inventory and ownership.** Define what counts as AI, registration timing and accountable roles.
3. **Risk classification.** Impact, autonomy, data, exposure, reversibility and service criticality.
4. **Data and knowledge use.** Purpose, minimization, consent, licensing, provenance, retention and deletion.
5. **Model and provider sourcing.** Approved catalogs, licenses, security, data terms, versioning and exit.
6. **Development and validation.** Reproducibility, protected test sets, metrics, segments, independent validation and red teaming.
7. **Human oversight and recourse.** Intervention modes, competence, authority, workload, escalation and appeals.
8. **Transparency.** User notice, AI interaction disclosure, limitations, source citations and decision explanations where required.
9. **Release and change.** Material changes, separation of duties, approvals, progressive rollout and rollback.
10. **Monitoring and incidents.** SLOs, drift/quality/safety, complaint handling, notification and post-incident review.
11. **Agent autonomy.** Tool classes, permissions, approvals, transaction controls, budgets and prohibited actions.
12. **Records and audit.** Evidence, retention, access and legal hold.
13. **Exceptions.** Compensating controls, approver, expiry and remediation.
14. **Retirement.** Consumer migration, access revocation, archival and deletion.

20.5 Risk-tier control profiles

Control area	Tier 1 low	Tier 2 moderate	Tier 3 high
Impact assessment	abbreviated	full contextual assessment	full assessment plus independent challenge
Data review	owner approval and classification	privacy/security review as applicable	formal data-flow, legal basis, representativeness and deletion evidence
Evaluation	baseline and task metrics	segmented, robustness and safety suite	independent validation, red team, human factors and stress tests
Human oversight	escalation path	defined review/override	tested oversight, recourse and authority before consequential action
Release approval	owner + automated gates	owner + risk/control approvals	separation of builder/validator/promoter and formal change authority
Monitoring	service and basic quality	segment, drift, safety and complaints	continuous controls, independent review and shorter recertification
Audit/retention	standard release evidence	expanded trace and approval	consequential-decision logs, immutable evidence and regulatory retention
Recertification	annual or material change	at least annual / risk-based	quarterly or continuous evidence review, plus incident-triggered

These are reference defaults. The control profile must be tailored to legal obligations and actual harm pathways.

20.6 AI impact assessment

A useful impact assessment answers:

- What decision, content or action does the system influence?
- Who is affected directly and indirectly, including vulnerable groups?
- What benefits and harms are plausible, severe, scalable and reversible?
- What data and proxies are used, and what population is underrepresented?
- What model/provider limitations, opacity and dependencies matter?
- What autonomy and tool permissions exist?
- How will people know AI is used, contest results and obtain human review?
- Which controls prevent, detect, contain and correct harm?
- What residual risk remains, who accepts it and for how long?
- What evidence will prove the controls continue to work?

The assessment should link to operational artifacts rather than duplicate them. For example, data lineage, evaluation reports, SLO dashboards and incident plans remain source evidence.

ISO/IEC 42005:2025 provides guidance for AI-system impact assessment [R5]. Use it as a reference while preserving jurisdiction-specific requirements.

20.7 Model risk management

Classical model-risk principles remain applicable to generative systems: clear purpose, sound development, independent validation, implementation controls, ongoing monitoring and governance. Extend the unit from model to system.

Validation scope

- conceptual soundness and appropriateness of algorithm/model class;
- data quality, lineage, representativeness and leakage;
- implementation verification and training reproducibility;
- outcome analysis, calibration, segments and uncertainty;
- benchmark relevance and contamination;
- RAG retrieval, prompt, policy and tool behavior;
- robustness, security, privacy and misuse;
- production implementation and monitoring;
- limitations, compensating controls and residual risk.

Independence

For Tier 3 systems, validators should not report to the builder for the decision under review and must control or have protected access to challenge data. Independence does not prohibit collaboration; it prevents builders from being sole judges of their own evidence.

Model inventory versus AI-system inventory

Maintain both when useful. One AI system may use multiple models; one model may support multiple systems with different risks. Approval at model level does not authorize every downstream use.

20.8 Human oversight and contestability

Define oversight as an engineered process:

- **Human-in-the-loop:** approval before a decision/action.
- **Human-on-the-loop:** supervised automation with monitoring and intervention.

- **Human-in-command:** authority to set boundaries, stop and redesign the system.

For each oversight point specify:

- trigger and coverage—every case, threshold, sample or exception;
- information shown, including uncertainty, evidence and alternative;
- reviewer competence, training and conflict-of-interest rules;
- decision authority and ability to override;
- time budget and workload limits;
- logging of review, override, reason and outcome;
- escalation and user recourse;
- periodic test for automation bias and rubber-stamping.

Human review can worsen outcomes if reviewers lack context, time or authority. Evaluate the combined human-AI system against the baseline human process.

20.9 Transparency and documentation

Use layered transparency:

- **Users:** disclose AI interaction or assistance where required or material; state limitations, data use, escalation and source basis.
- **Affected persons:** explain consequential outcomes in an understandable form and provide contest/recourse.
- **Operators:** system card, runbook, known failure modes, confidence and fallback.
- **Validators/auditors:** full lineage, evaluation, controls, changes and incidents.
- **Regulators/customers:** evidence package mapped to applicable obligations.

Model cards describe model-level characteristics; system cards describe the deployed context, prompts/retrieval/tools, controls and user process. Neither substitutes for evidence.

20.10 Third-party and foundation-model governance

Before approval, assess:

- provider identity, service location, subprocessors and security assurance;
- whether inputs/outputs are retained, reviewed or used for training;
- data residency, deletion, encryption and access controls;
- model version pinning, change notice and deprecation policy;
- performance, content policy, rate limits and availability commitments;
- training-data/model-weight provenance and license where disclosed;
- IP/indemnity and output-use terms;
- vulnerability, abuse and incident notification;
- audit rights and evidence availability;
- export, portability, fallback and termination assistance;
- pricing, capacity commitments and hidden network/storage costs.

For open-weight models, add artifact hash/signature, source reputation, license compatibility, unsafe serialization scan, benchmark contamination, model-card limitations and redistribution constraints.

20.11 Regulatory profiles

Maintain a base enterprise control set and overlays for jurisdiction/industry. Examples:

- **EU AI Act:** risk classification, prohibited practices, provider/deployer duties, transparency, high-risk system controls, general-purpose AI obligations and incident/market-surveillance evidence. The regulation entered into force on 1

August 2024; obligations phase in on multiple dates, and the current Commission timeline should be verified because legislative adjustments were under consideration in 2026 [R7].

- **Privacy regimes:** lawful basis/purpose, minimization, data-subject rights, automated-decision safeguards, DPIA/impact assessments, cross-border transfer, retention and breach response.
- **Financial services:** model-risk governance, independent validation, change control, explainability, fair lending/conduct and records.
- **Health/life sciences:** clinical safety, intended use, quality systems, human oversight, validation, change controls and regulated-device requirements where applicable.
- **Employment:** bias/impact testing, notice, human review, recordkeeping and local automated-employment-decision rules.
- **Public sector:** due process, accessibility, records, procurement transparency and public accountability.

Do not encode a regulation as a one-time checklist. Convert obligations into inventory fields, policies, tests, approvals, monitoring and evidence retention.

20.12 Control mapping

Governance need	Primary enterprise mechanism	Reference anchors
AI risk lifecycle	Govern/Map/Measure/Manage workflow	NIST AI RMF [R1], GenAI Profile [R2]
AI management system	policy, roles, objectives, audit and improvement	ISO/IEC 42001 [R3]
Risk process	identification, analysis, treatment and monitoring	ISO/IEC 23894 [R4]
Impact assessment	contextual harms and affected parties	ISO/IEC 42005 [R5]
Quality evaluation	evaluation process and quality characteristics	ISO/IEC TS 25058 [R6]
Security/privacy controls	control catalog, zero trust and privacy framework	NIST SP 800-53 [R8], SP 800-207 [R9]
Secure development	software and AI-model development practices	NIST SSDF [R10], SP 800-218A [R11]
LLM threats	prompt injection, data disclosure, agency and resource abuse	OWASP LLM Top 10 [R12], MITRE ATLAS [R13]
Supply chain	provenance, signing and workload identity	SLSA [R14], Sigstore [R15], SPIFFE [R16]

20.13 Governance KPIs

- AI inventory coverage and orphan-system count;
- percentage with current owner, risk tier, intended use and impact assessment;
- approval lead time by risk tier and false-block/exception rate;
- evaluation/control evidence completeness;
- high-risk independent-validation coverage;
- exception count, age and expiry breaches;
- overdue recertification and unsupported provider/model versions;
- complaints, appeals, corrections and time to resolution;
- incidents by control failure and repeat rate;
- AI literacy/training completion by role;
- audit findings and time to remediation;
- percentage of controls executed automatically with reviewable evidence.

20.14 Governance failure modes and anti-patterns

- **Committee bottleneck:** every use case waits for monthly review. Remedy: risk-tiered automated workflows and delegated approvals.
- **Inventory without enforcement:** systems are registered after launch. Remedy: gate provider access, production identities and deployment on inventory ID.

- **Documentation theater:** cards copied between releases. Remedy: auto-populate from lineage/evaluation and validate required change-specific fields.
- **One model approval for all uses:** ignores context. Remedy: approve model source separately from system/use-case release.
- **Human-in-the-loop as blanket mitigation:** untested review. Remedy: measure reviewer accuracy, workload, overrides and recourse.
- **Permanent exceptions:** risk accumulates. Remedy: expiry, compensating-control monitoring and executive visibility.
- **Compliance after architecture:** costly redesign. Remedy: regulatory profiles at intake and data/provider selection.
- **Governance owns outcomes:** product teams disengage. Remedy: first-line accountability with independent challenge.

21. Security and privacy architecture

21.1 Security objective and threat model

Protect confidentiality, integrity, availability and authorized behavior across data, models, prompts, tools and infrastructure. AI expands the attack surface because probabilistic components consume untrusted natural language, model artifacts may embed opaque code/data, and agents can translate text into actions.

Use a zero-trust architecture: no implicit trust from network location; authenticate and authorize users, workloads and devices for each resource [R9]. Map AI-specific techniques using NIST adversarial ML guidance, OWASP LLM risks and MITRE ATLAS [R12][R13][R17].

Threat categories:

- source-data tampering, poisoning, label manipulation and backdoors;
- feature, embedding or vector-index poisoning;
- model-weight compromise, malicious serialization and supply-chain substitution;
- adversarial examples, evasion and decision manipulation;
- model extraction, inversion, membership inference and memorization leakage;
- prompt injection, jailbreak and indirect injection from retrieved/tool content;
- sensitive information disclosure in prompts, outputs, logs or caches;
- insecure output handling and code/command injection;
- excessive agency, confused deputy and unauthorized tool use;
- unbounded token, recursion, tool or compute consumption;
- cross-tenant leakage through caches, memory, GPU, logs or indexes;
- provider compromise, silent model change and dependency outage;
- credentials/secrets exposed to models or notebooks;
- denial of service, queue starvation and accelerator exhaustion;
- insider misuse and unapproved shadow AI.

21.2 Identity, authorization and tenancy

Human identity

- central SSO and MFA;
- role and attribute-based authorization tied to project, data class, environment and action;
- just-in-time privileged access with approval and expiry;
- separate administration, development, validation and audit roles;
- periodic access review and automatic offboarding;
- break-glass accounts stored and monitored separately.

Workload identity

Use short-lived, cryptographically verifiable workload identities rather than static API keys. SPIFFE/SPIRE is one portable pattern [R16]. Bind identity to environment, namespace/project, service account and workload attestation. Scope access to exact datasets, models, secrets and tools.

Authorization layers

- platform resource authorization;
- data/row/column/document authorization;
- model/provider/task authorization;
- tool/action authorization;
- release/lifecycle transition authorization;
- user/business decision authorization.

A model's generated intent never grants authorization. Tool brokers and policy engines decide using authenticated context.

Tenant isolation

Test isolation across:

- namespaces/accounts/clusters and node pools;
- keys, buckets, databases, topics and feature stores;
- vector indexes, filters and backups;
- prompt/KV/semantic caches and session memory;
- logs, traces, evaluation data and billing;
- GPU memory/process boundaries;
- model adapters and routing configuration;
- support and break-glass access.

21.3 Network security

- private endpoints for control plane, data, registries and production inference;
- deny-by-default east-west and egress policy;
- egress proxy with provider/domain allowlists, inspection and usage attribution;
- mTLS where service identity and traffic protection justify it;
- separate management, workload and data networks or equivalent policy boundaries;
- no direct production database or internet access from notebooks;
- rate limits, WAF/API protections and DDoS controls at ingress;
- controlled DNS and certificate lifecycle;
- region-aware routing and data-loss prevention;
- service mesh only when operational maturity supports its control plane and certificate complexity.

21.4 Data and privacy controls

- classify data before ingestion and propagate labels through lineage;
- minimize fields, records, context and retention to the approved purpose;
- encrypt in transit and at rest; separate keys for high-risk tenants/domains;
- tokenize or pseudonymize direct identifiers where linkage is needed;
- de-identify with measured re-identification risk, not name removal alone;
- field/row/document-level authorization and dynamic masking;
- consent, purpose and use restrictions expressed as enforceable policy;
- DLP for prompts, documents, outputs, logs and external provider calls;
- deletion propagation to features, embeddings, indexes, caches and derived datasets;

- privacy-preserving analysis/training—federated learning, differential privacy or confidential computing—where the threat model and utility justify complexity;
- privacy-aware telemetry sampling and controlled forensic capture;
- data-subject/employee/customer notice, access, correction and appeal workflows as applicable.

Confidential computing can protect data in use inside hardware-backed attested trusted execution environments, but it does not solve application authorization, model behavior or compromised inputs [R18].

21.5 Secrets and cryptography

- centralized secret management with dynamic/short-lived credentials;
- never place secrets in prompts, notebooks, environment files, model artifacts or logs;
- inject credentials only into the tool broker or workload that needs them;
- rotate provider keys, certificates and signing keys; support immediate revocation;
- use KMS/HSM-backed keys for high-value artifacts and trust roots;
- sign containers, models, prompts/policies and deployment bundles;
- verify signatures and provenance at admission and startup;
- manage key separation by environment and region;
- audit secret access and detect unusual use;
- plan cryptographic agility and trust-root recovery.

21.6 Secure software and model supply chain

Apply secure-development practices to code, data pipelines and AI models. NIST SSDF and the AI-specific SP 800-218A provide primary guidance [R10][R11].

Controls:

- trusted source repositories, protected branches and code owners;
- dependency pinning, internal mirrors, vulnerability and license scans;
- isolated ephemeral builds and reproducible recipes;
- SBOM plus model/AI BOM for base model, adapter, tokenizer, datasets and runtime;
- build provenance at an appropriate SLSA level [R14];
- artifact signing with keyless or managed identity, verified through policy [R15];
- malware and unsafe model-format/deserialization scanning;
- base-image minimization, patching and end-of-life policy;
- model/provider allowlist and digest pinning;
- quarantine, recall and signing-identity revocation;
- no production-time downloads from public repositories;
- security review of custom kernels, plugins and model code requiring remote execution.

21.7 Training and data-pipeline security

- isolate training networks and restrict egress;
- validate source and label provenance; monitor unexpected distribution or contributor change;
- separate high-trust gold/evaluation data from builder write access;
- use checksums, signatures and versioned snapshots;
- detect duplicate, poisoned, canary or prohibited records;
- protect checkpoints and optimizer state as sensitive assets;
- avoid loading arbitrary pickle or remote model code;
- rate/quota large jobs to prevent resource denial;
- scan logs for secrets and sensitive data;
- audit dataset access and export;
- require approval for external data/model changes.

21.8 Inference, RAG and prompt-injection controls

No prompt-injection detector is complete. Use defense in depth:

1. authenticate caller and determine allowed task, data and tools;
2. classify/minimize input and deny prohibited data/provider routes;
3. delimit system instructions, user input, retrieved content and tool output;
4. treat retrieved/tool content as untrusted data;
5. filter corpus sources and preserve origin/trust metadata;
6. enforce source/document ACLs before retrieval and cache lookup;
7. use deterministic tool authorization and typed schemas;
8. restrict tool network/filesystem and credentials;
9. validate outputs before rendering, executing or storing;
10. apply content security controls to generated HTML/SQL/code;
11. cap context, tokens, retries, steps and cost;
12. log policy decisions and high-risk action evidence;
13. red-team direct and indirect injection with realistic tools and data.

RAG improves grounding but adds a content supply chain. Protect connector identities, source integrity, parser sandbox, chunk metadata, embedding model, index publication and deletion.

21.9 Agent security

Classify tools by action risk:

- **Class A — read-only public/low sensitivity:** bounded automated access.
- **Class B — read sensitive:** explicit user/workload authorization and logging.
- **Class C — reversible write:** preconditions, idempotency, scoped credentials and postcondition checks.
- **Class D — irreversible/material:** human approval, transaction limits, dual control where appropriate and compensating process.
- **Class E — prohibited:** no tool registration or policy denies all use.

Additional controls:

- deterministic workflow shell around model decisions;
- allowlisted tools and parameters; deny arbitrary URLs/commands;
- sandbox with CPU/memory/time/network/filesystem limits;
- separate planning from execution and show action preview for approval;
- state machine with maximum steps and repeated-state detection;
- per-user/tenant/session memory boundaries and retention;
- transaction journal and replay protection;
- user presence or step-up authentication for sensitive actions;
- independent verification of high-impact outcomes;
- emergency global and scoped tool revocation.

21.10 Runtime and infrastructure hardening

- non-root containers, read-only root filesystem and minimal capabilities;
- seccomp/AppArmor/SELinux or platform equivalents;
- image admission, signature and vulnerability policies;
- namespace/project quotas and limit ranges;
- separate critical inference and untrusted experimentation pools;
- node hardening, patching, secure boot and accelerator driver lifecycle;
- encrypted disks and protected ephemeral storage;

- metadata-service protection and workload identity instead of node credentials;
- Kubernetes/API audit, network policy and secrets encryption;
- backup protection against ransomware and unauthorized deletion;
- control-plane, registry and policy-service high availability;
- GPU resource isolation and process cleanup between tenants;
- vulnerability management with exploitability and service exposure context.

21.11 Security testing and continuous assurance

- threat model at intake and material change;
- static/dynamic/application/API testing;
- IaC, container and dependency scanning;
- model artifact and deserialization tests;
- data poisoning and integrity exercises;
- adversarial ML testing for evasion, extraction, inversion and leakage as relevant;
- direct/indirect prompt injection and jailbreak suites;
- agent tool-abuse and privilege-escalation simulation;
- tenant-isolation and cache/index authorization tests;
- red team before high-risk launch and periodically thereafter;
- penetration tests and provider evidence review;
- continuous control monitoring and security telemetry;
- incident/tabletop exercises with legal/privacy/governance participation.

21.12 Security metrics

- percentage of workloads using workload identity versus static secrets;
- unsigned/untrusted artifact blocks and signature verification coverage;
- critical/high vulnerability exposure age;
- model/provider allowlist violations and shadow-AI calls;
- egress denials, DLP events and sensitive-data routing violations;
- prompt-injection/jailbreak success on regression suite;
- tool-policy denial, unauthorized-action and compensation rates;
- tenant-isolation test coverage and failures;
- mean time to revoke key/model/provider/tool;
- security-incident detection/containment and repeat rate;
- secrets found in code/logs/artifacts;
- access-review and privileged-session compliance;
- supply-chain provenance coverage.

21.13 Security failure modes and anti-patterns

- **“Private network means trusted.”** Internal compromise and confused-deputy paths remain. Use identity and authorization.
- **Prompt filtering as sole defense.** Attackers can encode or inject through documents/tools. Enforce deterministic permissions and output handling.
- **Shared provider key across enterprise.** No attribution or revocation scope. Use brokered identities/keys per workload or tenant.
- **Secrets given to model context.** They can leak. Inject at tool execution boundary.
- **Vector database without ACL test.** Similarity returns unauthorized content. Use pre-filtering and release-blocking isolation tests.
- **Unsigned open-weight downloads.** Supply-chain substitution. Mirror, hash, scan, sign and approve.
- **Broad notebook production access.** Exfiltration and accidental change. Use governed pipelines and JIT break-glass.
- **Agent with shell/browser and open internet.** Excessive agency. Use specific tools, sandboxes and allowlists.

- **Logging as forensic insurance.** Full payload logs become a breach asset. Minimize and use controlled capture.

22. Infrastructure, capacity and performance engineering

22.1 Infrastructure objective

Provide the right compute, storage and network at the right time, with isolation and recovery proportional to workload criticality. AI infrastructure must serve conflicting traffic: interactive notebooks, bursty training, long batch pipelines, latency-sensitive inference and stateful retrieval. One undifferentiated cluster or capacity pool will produce contention and weak economics.

22.2 Substrate selection

Managed AI services

Use when delivery speed, elasticity and reduced operations outweigh constraints in runtime customization, version control, networking, telemetry or portability. Validate provider behavior, quotas, model versioning, data terms and exit.

Kubernetes

Use when the enterprise needs a shared portable control surface, custom runtimes, multi-model serving, accelerator scheduling and policy integration. Kubernetes automates deployment, scaling and management of containerized workloads [R19], but it adds control-plane, upgrade, networking, storage and security responsibilities. Do not adopt it solely because AI examples use it.

Virtual machines

Use for specialized drivers, licensed software, appliances or workloads that do not fit container orchestration. Manage images, patching, identity and scaling through the same control plane.

Serverless

Use for event-driven preprocessing, light inference and orchestration where startup and runtime limits are acceptable. Large models often require prewarmed or dedicated capacity.

Edge/on-premises

Use for data sovereignty, local latency, disconnected operation or hardware integration. Budget for fleet heterogeneity, signed updates, limited observability and physical threat.

A hybrid platform can expose one lifecycle while dispatching to multiple substrates.

22.3 Workload pools and scheduling

Create separate logical or physical pools:

- platform control services;
- interactive CPU workspaces;
- interactive GPU development;
- batch data processing;
- classical/deep training;
- production predictive inference;
- production LLM inference;
- untrusted artifact evaluation/red team;

- edge build/test.

Scheduling controls:

- per-tenant quotas and budget limits;
- priority classes and fair-share queues;
- preemption rules and checkpoint requirements;
- accelerator type, memory and topology constraints;
- gang scheduling for distributed jobs;
- node affinity/anti-affinity and failure-domain spread;
- taints/tolerations or equivalent isolation;
- reserved critical capacity and best-effort pools;
- maximum runtime, idle shutdown and queue admission;
- capacity calendars for planned large jobs.

Protect control plane and production inference from training bursts. A queue that accepts more work than the platform can complete within business deadlines is not elastic; it is hiding backlog.

22.4 Accelerator architecture

Device selection

Evaluate:

- memory capacity and bandwidth;
- compute formats and kernel support;
- interconnect topology and bandwidth;
- framework/runtime maturity;
- virtualization/partition support;
- availability, regional supply and commitment terms;
- energy/power/cooling and rack/network constraints;
- portability of model and toolchain.

Training memory

A rough training memory budget includes:

> **weights + gradients + optimizer states + activations + temporary buffers + communication/runtime overhead.**

For mixed-precision Adam-style training without sharding, optimizer and master-weight states can dominate parameter memory. FSDP/ZeRO-style sharding distributes parameters, gradients and optimizer state; activation checkpointing trades compute for memory [R30][R31]. Always profile the exact architecture, sequence length, batch and kernels.

LLM inference memory

Approximate total device memory:

> **model weights + KV cache + activations/workspace + runtime fragmentation/headroom.**

For a standard transformer, KV cache scales approximately with layers × hidden width × active tokens × batch/concurrency × bytes, adjusted for grouped/multi-query attention. Long contexts and concurrent sequences can consume more memory than weights. Capacity planning must use the observed prompt/output distribution.

Parallelism

- **Data parallelism:** replicas process different batches; common for training.

- **Tensor parallelism:** shards operations across devices; useful when a model does not fit one device but adds communication.
- **Pipeline parallelism:** shards layers/stages; useful across devices/nodes but creates bubbles and balancing complexity.
- **Expert parallelism:** routes mixture-of-experts components; requires load balancing.
- **Disaggregated prefill/decode:** can optimize differing compute/memory characteristics at very high serving scale; adds network and scheduler complexity.

Use the minimum parallelism required to meet memory and throughput. Distributed execution can reduce efficiency for models that fit on one device.

22.5 Capacity models

Online services

From Little's Law:

> **required in-flight concurrency ≈ peak arrival rate × target service time.**

Then:

> **replicas ≈ required concurrency ÷ (safe concurrency per replica × target utilization).**

Adjust for traffic variance, failover, cold start, autoscaling delay and one failure domain. Use p95/p99 service-time distributions, not averages.

Token-based LLM services

Model prefill and decode separately:

- input tokens drive prefill compute and TTFT;
- output tokens drive decode duration and inter-token latency;
- concurrent active tokens drive KV-cache memory;
- batching improves throughput until queue delay violates SLO.

Forecast by request class: model, prompt-length bucket, output-length bucket, tenant, region and priority.

Batch/training

Estimate:

> **elapsed time ≈ total work ÷ effective throughput + queue + startup + checkpoint/recovery overhead.**

Effective throughput includes data loading, communication and convergence efficiency. Size to complete within the business window and permit replay/failure.

22.6 Storage architecture

- **Object storage:** datasets, models, checkpoints, logs and immutable artifacts; enable versioning, lifecycle and replication.
- **Block/local NVMe:** high-throughput scratch, caches and shuffle; assume replaceable unless explicitly replicated.
- **Shared file systems:** legacy POSIX workloads and high-throughput training; guard metadata bottlenecks and cost.
- **Relational databases:** control-plane metadata, workflow state and registry indexes; design HA, backup and migration.
- **Caches:** feature, prompt, KV, artifact and metadata caches; keys include tenant/release and invalidation policy.
- **Vector/search:** derived knowledge state with backup or reproducible rebuild, ACL metadata and deletion.

Avoid using a single shared file system as the authoritative registry. Separate durable source/artifact state from ephemeral execution state.

22.7 Network architecture

AI workloads stress both north-south and east-west paths:

- distributed training requires high bandwidth and low latency across accelerators;
- model load/checkpoint can saturate object storage and network;
- RAG adds multiple service hops and data-store queries;
- token streaming creates many long-lived connections;
- cross-region provider calls add latency and egress cost.

Controls and optimizations:

- topology-aware placement and dedicated high-speed fabrics for large training;
- bandwidth/QoS separation for control, storage and workload traffic;
- model/artifact prefetch and local cache;
- private provider links where justified;
- compression and columnar formats for data transfer;
- egress proxy and regional route policy;
- connection pooling, keepalive and backpressure;
- network telemetry per job/service and cost attribution.

22.8 Autoscaling

Use signals aligned with work:

- queue depth and oldest wait time;
- in-flight requests/concurrency;
- tokens waiting/active and KV-cache pressure;
- batch backlog and deadline slack;
- accelerator utilization and memory, as secondary signals;
- feature/vector-store latency and saturation.

Guardrails:

- minimum warm capacity for load time and SLO;
- maximum cost/replica and tenant quotas;
- stabilization windows to avoid oscillation;
- scale-down drain and checkpoint;
- capacity-unavailable fallback;
- predictive scaling for known peaks;
- regional and provider quotas monitored before need.

22.9 Reliability and recovery infrastructure

- multiple availability zones/failure domains for critical services;
- replicated metadata, artifacts and trust roots;
- immutable backups and tested restore;
- regional failover strategy with policy/residency validation;
- independent monitoring and external probes;
- control-plane degradation behavior and cached policy;
- spare capacity or approved provider fallback;
- infrastructure replacement through IaC;
- edge offline operation and telemetry buffer;
- periodic chaos, node-loss and storage/network-failure tests.

22.10 Performance engineering workflow

1. Define user/business SLO and representative workload distribution.
2. Instrument queue, preprocessing, feature/retrieval, model, postprocessing and network spans.
3. Establish single-request and single-device baseline.
4. Profile compute, memory, I/O and synchronization.
5. Increase concurrency until saturation; identify knee and tail behavior.
6. Apply one optimization at a time and rerun quality regression.
7. Test cold start, scale-up, failover, mixed tenants and long-tail inputs.
8. Record runtime/hardware/config as a versioned performance profile.
9. Set safe operating limits and alerts from measured headroom.

22.11 Infrastructure KPIs

- queue wait and deadline miss;
- CPU/GPU utilization, memory headroom and accelerator error rate;
- tokens/examples per second and scaling efficiency;
- model load, checkpoint and restore time;
- storage throughput/IOPS, small-file count and object request cost;
- network throughput/latency/retransmit and egress;
- autoscaling reaction, cold start and rejected demand;
- capacity reservation utilization and spot interruption recovery;
- node/cluster availability and upgrade success;
- backup/restore and regional failover test success;
- infrastructure cost per workload unit.

22.12 Infrastructure failure modes and anti-patterns

- **One cluster for everything:** contention and large blast radius. Segment pools and priorities.
- **GPU utilization as sole KPI:** ignores queue, data loading, convergence and cost per outcome.
- **Autoscale on CPU for GPU inference:** late or wrong signal. Use queue/concurrency/token pressure.
- **No headroom:** p99 collapses under bursts/failures. Operate below saturation.
- **Model load from remote public source:** slow, unreliable and insecure. Mirror and pre-stage.
- **Local checkpoints only:** interruptions erase work. Atomic durable checkpoints.
- **Cross-region active-active by diagram:** mutable state and residency unresolved. Design and test each state class.
- **Over-sharding:** coordination dominates. Fit on fewer devices when possible.
- **Permanent reserved capacity for experiments:** low utilization. Portfolio on-demand/reserved/spot.

23. Reliability engineering and platform operations

23.1 Reliability objective

Deliver predictable platform services and customer AI workloads while controlling change and learning from failure. Reliability is the probability that users receive acceptable outcomes within required time, not merely process uptime.

23.2 Service-level framework

For each platform and AI service define:

- **SLIs:** measured user-visible indicators;
- **SLOs:** target over a window;
- **error budget:** acceptable failure amount;

- **dependencies:** inherited constraints and contracts;
- **support tier:** on-call and response;
- **RTO/RPO:** recovery and data loss;
- **quality/safety SLO:** model/system behavior;
- **cost guardrail:** budget and unit-cost target.

Example SLO set for an online AI service:

- availability: successful eligible responses / valid requests;
- latency: p95 end-to-end by request class;
- quality: rolling task-success or calibrated performance by key segment;
- safety: zero critical policy violations and bounded false-refusal rate;
- freshness: feature/index age within target;
- evidence: trace completeness above threshold;
- cost: p95 cost/request or monthly budget.

Use error budgets to govern release velocity. If reliability budget is exhausted, prioritize stabilization and risk reduction. Safety or legal limits are hard constraints, not consumable error budgets.

23.3 Platform service SLOs

Critical shared services:

- identity/workload identity;
- policy decision and admission;
- registry/artifact storage;
- orchestration and CI/GitOps;
- data/feature/vector services;
- inference gateway and routing;
- telemetry and audit;
- secret/KMS and network control.

Specify degraded behavior. For example, cached policy may permit existing low-risk traffic for a bounded interval, while new deployments fail closed. Registry outage should not terminate loaded models, but should block new release resolution.

23.4 Operational processes

Incident management

Multidimensional severity, incident commander, communications, containment, recovery, evidence and review as defined in Stage 11.

Problem management

Track recurring or systemic causes across incidents: driver instability, schema change, provider rate limits, alert gaps, model load and data quality. Maintain known-error workarounds and verify permanent fixes.

Change management

Automated standard changes use preapproved pipelines; normal/high-risk changes require evidence and approvals. Emergency changes use break-glass, narrow scope and retrospective review. Measure change failure, not ticket count.

Configuration management

Desired state is versioned; runtime inventory and drift are continuously reconciled. Track model, prompt, index, policy, runtime and infrastructure—not only servers.

Capacity management

Forecast demand, accelerator supply, provider quota, storage/network and support capacity. Review lead times and concentration risks. Maintain launch capacity and incident reserve.

Availability and continuity

Dependency SLOs, redundancy, backups, failover, manual processes and recovery exercises. Validate end-to-end business workflow.

Access management

Entitlement provisioning, JIT privilege, review, offboarding and break-glass audit.

Vulnerability and patch management

Risk-prioritized patching for hosts, cluster, drivers, images, libraries and models/providers. Validate accelerator/runtime compatibility and use progressive fleet upgrades.

Release/deprecation management

Supported versions, compatibility, notice, migration tools, enforcement and retirement.

23.5 On-call readiness

A service is not production-ready without:

- owner and 24×7 contact if service tier requires it;
- concise architecture and dependency map;
- dashboards showing user symptoms and saturation;
- actionable alerts with severity and runbooks;
- access and break-glass procedure;
- rollback/fallback/kill-switch instructions;
- data/model/provider incident paths;
- known limitations and capacity limits;
- communication templates and escalation;
- recent load, rollback and recovery test.

On-call engineers should not need model-training expertise to execute initial containment. Runbooks separate safe containment from deeper diagnosis.

23.6 Upgrade strategy

AI platforms combine fast-moving projects, drivers and model runtimes. Use:

- supported-version policy and compatibility matrix;
- separate upgrade rings: lab, nonproduction, canary production, broad production;
- immutable base-image rebuild rather than in-place package mutation;
- API and artifact compatibility tests;
- driver/runtime/firmware validation on representative hardware;
- model-load and numerical regression after runtime changes;

- provider/model regression when managed services update;
- maintenance windows and rollback plan;
- inventory-driven identification of affected workloads;
- deprecation notices and enforced end dates.

Avoid synchronized “big bang” upgrades of cluster, drivers, serving runtime and models. Change one layer where possible and preserve diagnosis.

23.7 Platform observability and metamonitoring

Monitor:

- platform API availability/latency/error;
- job and deployment queues;
- controller reconciliation lag;
- registry/object-store latency and replication;
- identity/policy decision latency and failures;
- telemetry ingestion loss and alert-delivery path;
- certificate, secret and token expiration;
- quota/capacity and provider limits;
- audit-log delivery and retention;
- customer journey synthetic tests from workspace creation through deployment.

Prometheus separates alert rules from Alertmanager functions such as grouping, deduplication, silencing and routing [R32]. Alert on symptoms and verify the monitoring path itself.

23.8 Reliability KPIs

- platform/customer service SLO attainment and error-budget burn;
- MTTA, containment and MTTR;
- change failure and rollback rate;
- repeat incident and problem backlog age;
- alert precision/actionability and pages per shift;
- backup restore, DR and failover exercise success;
- platform provisioning and deployment success;
- reconciliation drift and out-of-band changes;
- capacity-related rejections and deadline misses;
- upgrade duration, failure and version compliance;
- dependency/provider SLO and concentration;
- support response/resolution and self-service deflection.

23.9 Operational failure modes and anti-patterns

- **No service owner:** platform team becomes default owner. Enforce ownership at registration.
- **Runbook is a wiki essay:** unusable under pressure. Use decision trees, commands, checks and rollback.
- **Alerts on every component:** fatigue. Alert on user symptoms/error budget and route diagnostics to dashboards.
- **DR tested only as infrastructure restore:** application state/policy/data missing. Test complete business flow.
- **Platform upgrade without workload inventory:** hidden breakage. Use compatibility and canary rings.
- **Manual production repair:** drift and no evidence. Restore declarative state; record emergency action.
- **SLO copied from web service convention:** disconnected from loss. Derive from business needs and dependency reality.
- **Model quality treated outside operations:** users experience one service. Integrate quality/safety into incident and SLO process.

24. FinOps, cost and sustainability

24.1 Economic objective

Maximize business value per unit of technology and human effort while preserving quality, safety and reliability. FinOps is a collaborative operating practice across engineering, finance and business; the FinOps Framework provides a current operating model [R33], with AI-specific guidance emphasizing token economics, GPU scarcity, allocation, forecasting and optimization [R34].

24.2 Full-lifecycle cost model

Include:

- data acquisition, storage, transfer, processing and labeling;
- workspaces and experimentation;
- training/HPO, failed runs and checkpoints;
- evaluation, red-team and human review;
- model/provider inference;
- retrieval, embeddings, vector/search and reranking;
- gateway, orchestration, agents and tool calls;
- observability, logs/traces and security controls;
- platform shared services and engineering/support labor;
- reserved commitments, licenses and vendor minimums;
- network egress, backup and disaster recovery;
- migration, exit and retirement.

Avoid comparing a managed API's token price with only self-hosted GPU cost. Include utilization, engineering, support, redundancy, storage/network and opportunity cost.

24.3 Allocation and unit economics

Tag or attribute every cost to:

- product/use case and business owner;
- tenant/customer/domain;
- environment and region;
- workload stage: data, train, evaluate, infer, observe;
- model/provider and release;
- shared platform service;
- experiment or production.

Useful unit metrics:

- cost per accepted training candidate;
- cost per million examples/predictions/tokens;
- cost per retrieval query and indexed document;
- cost per successful user task or automated process;
- cost per avoided loss, conversion or hour saved;
- platform cost per active team/service;
- human review cost per accepted outcome;
- marginal and fully allocated cost.

Core formula:

> **Cost per successful outcome = total attributable lifecycle cost ÷ number of outcomes meeting defined quality and business criteria.**

A cheap response that fails or requires correction is not efficient.

24.4 Financial controls

- budgets by portfolio, product and environment;
- quota and rate limits tied to budget;
- showback first, chargeback where behavior/incentives justify it;
- pre-run cost estimates and maximum job spend;
- token/context/tool/step budgets at request and workflow level;
- anomaly detection and automated containment for runaway spend;
- commitment/reservation governance and utilization targets;
- approved provider/model rate cards and routing policy;
- storage/log retention lifecycle;
- purchase-order/contract consumption alerts;
- unit-cost and value review in release/recertification.

Hard budget stops should fail safely. A high-impact workflow may need manual or lower-cost fallback rather than abrupt denial.

24.5 Training cost optimization

- validate correctness and convergence on small scale;
- choose the smallest model and dataset sufficient for objective;
- use transfer learning, PEFT, distillation and active sampling;
- early stop unpromising trials and use efficient HPO;
- improve data loading and scaling before adding accelerators;
- use spot/preemptible capacity with checkpoints;
- schedule around commitment and supply constraints;
- deduplicate/cull low-value checkpoints and artifacts;
- measure accepted-candidate cost, not individual run cost only;
- reuse embeddings, features and base models under governance.

24.6 Inference cost optimization

- route bounded tasks to SLMs/classical models/rules where quality passes;
- cap and compress context; improve retrieval precision;
- use batch/asynchronous processing for noninteractive work;
- dynamic/continuous batching and appropriate quantization;
- prefix/KV/exact caches with safe keys;
- right-size replicas and autoscale on work signals;
- reserve steady base load and use elastic overflow;
- reduce retry and agent loop waste;
- stream/cancel promptly when users abandon;
- monitor output-token verbosity and structured response size;
- select provider/self-hosted capacity from measured demand and break-even analysis.

24.7 Capacity portfolio

Balance:

- reserved/committed capacity for predictable critical base load;

- on-demand for launches and uncertain peaks;
- spot/preemptible for restartable batch/training;
- managed APIs for burst/variety or rapid access;
- self-hosted SLM/LLM for steady high-volume or control-sensitive workloads;
- cross-region capacity only where residency and recovery justify cost.

Review concentration: a low unit price from one provider can create unacceptable outage, negotiation and migration risk.

24.8 Forecasting

Forecast drivers rather than top-line spend:

- active users/tenants and requests/user;
- prompt/output token distributions;
- retrieval/tool calls per request;
- task-success and retry rates;
- model/provider route mix;
- training experiments and candidate acceptance;
- data growth, embedding refresh and retention;
- service tier redundancy and region count;
- platform adoption and shared-service allocation.

Use scenarios for adoption, model change, provider pricing, context growth and regulatory/control cost. Compare forecast with actual weekly for fast-moving services.

24.9 Sustainability

Where material, track energy or carbon proxies alongside cost:

- accelerator-hours and utilization;
- training/inference work per successful outcome;
- region/time electricity characteristics if reliable data is available;
- hardware life and embodied impact in owned infrastructure;
- avoided duplicate training and unnecessary retention;
- model-size and precision trade-offs.

Do not report false precision. Cost and compute efficiency are often the most actionable proxies; quality and equity must not be sacrificed to improve an environmental metric superficially.

24.10 FinOps KPIs

- forecast accuracy and budget variance;
- allocation coverage and untagged spend;
- cost per successful outcome and its trend;
- GPU/reservation utilization and idle cost;
- spot recovery and interruption waste;
- token/context/retry/agent-step efficiency;
- cache and batching savings verified against quality;
- model-route cost and quality frontier;
- storage/log/checkpoint growth and deletion savings;
- commitment coverage and expiration risk;
- anomaly detection/containment time;
- realized value-to-cost ratio.

24.11 FinOps failure modes and anti-patterns

- **Optimize infrastructure before product:** low-value use case becomes cheaper. Revalidate business outcome.
- **Cost per token only:** ignores success, review and tools. Use end-to-end outcome cost.
- **Shared GPU cost unallocated:** no accountability. Meter job/service/tenant time and memory/useful work.
- **Blanket quotas:** block critical flows and encourage shadow accounts. Risk-aware budgets and escalation.
- **Reserved capacity by peak:** idle waste. Reserve base load, burst elastically.
- **Self-hosting assumed cheaper:** excludes engineering and redundancy. Full TCO and break-even.
- **Observability retained forever:** hidden storage cost and privacy risk. Tiered retention.
- **Routing solely by price:** quality/safety inconsistency. Constrain by policy and evaluation first.

25. Developer experience and platform engineering

25.1 Developer-experience objective

Make the secure, governed path the fastest way to deliver. Good platform engineering reduces cognitive load by encoding architecture, controls and operations into reusable APIs and templates. A paved road should be easier than assembling a shadow stack.

25.2 Golden paths

Prioritize a small set of supported patterns:

1. versioned batch training and scoring;
2. online predictive model with feature service;
3. managed LLM API through enterprise gateway;
4. self-hosted SLM/LLM endpoint;
5. enterprise RAG with ACL-aware corpus;
6. bounded agent workflow with approved tools;
7. edge model packaging and rollout.

Each golden path includes:

- reference architecture and threat model;
- project template and sample application;
- IaC and pipeline definitions;
- SDK and local test harness;
- evaluation policy and example datasets;
- default SLOs, dashboards and alerts;
- cost model and quotas;
- runbook and rollback;
- control evidence generated automatically;
- documented extension and exception points.

25.3 Self-service workflow

A user should be able to:

1. register an approved use case;
2. create a project/workspace with identity, repo and budget;
3. discover authorized data/features/models;
4. run tracked experiments;

5. submit training/evaluation jobs;
6. register a release;
7. request deployment through declarative configuration;
8. inspect quality, traces, cost and approvals;
9. manage feedback and retraining;
10. deprecate/retire.

Self-service is not unrestricted access. Policy and quotas execute within the workflow and explain denials with actionable remediation.

25.4 API and resource design

Define stable platform resources such as:

- `AIProject`;
- `DatasetBinding`;
- `TrainingJob`;
- `EvaluationRun`;
- `ModelRelease`;
- `KnowledgeIndex`;
- `InferenceService`;
- `AgentWorkflow`;
- `Approval` and `Exception`.

Principles:

- declarative desired state and idempotent operations;
- clear ownership and lifecycle status;
- immutable version references;
- sensible secure defaults;
- validation errors that identify policy and fix;
- asynchronous long-running operations with status/events;
- backward compatibility and versioned APIs;
- consistent labels for tenant, owner, cost and risk;
- generated client SDKs and examples;
- no product-specific fields in the enterprise contract unless required.

25.5 Templates and inner-loop tooling

- repository templates by archetype;
- dev containers and reproducible environments;
- local/small-data emulators for pipelines and endpoints;
- policy linter before remote submission;
- synthetic/masked test datasets;
- prompt and RAG trace replay;
- contract and schema test harness;
- performance smoke test and cost estimate;
- one-command or portal deployment to ephemeral nonproduction;
- automatic cleanup;
- code snippets that use workload identity and telemetry correctly.

25.6 Documentation and education

Documentation layers:

- 10-minute quick starts;
- task-oriented how-to guides;
- conceptual architecture and decisions;
- API/reference documentation;
- troubleshooting and known errors;
- security/governance responsibilities;
- migration/deprecation guides;
- example services with tests and dashboards.

Operate office hours, role-based training, internal community, champions and incident learning. AI literacy should include limitations, data handling, evaluation and human oversight—not only prompt writing.

25.7 Exception experience

A good paved road has a credible exception path:

- user states unmet requirement and evidence;
- platform/security evaluates whether pattern should expand or exception is justified;
- compensating controls and operational owner are defined;
- exception is time-bound and visible;
- reusable solution is integrated back into platform when demand repeats.

If exception approval takes months, teams will bypass the platform. If every preference is accepted, the platform fragments.

25.8 Platform product discovery

- interview builders and operators by archetype;
- observe lead-time bottlenecks and support tickets;
- instrument funnel from project creation to production;
- identify top manual steps and policy denials;
- measure template adoption and abandonment;
- run design-partner cohorts before broad rollout;
- publish roadmap, service health and deprecations;
- treat documentation and migration as product features.

25.9 Developer-experience KPIs

- time to first tracked experiment and first compliant endpoint;
- self-service completion and abandonment;
- golden-path adoption and exception rate;
- deployment lead time and pipeline success;
- support tickets per active team and time to resolution;
- documentation search success/deflection;
- environment build/provision time;
- developer satisfaction and retained active teams;
- duplicate stack/tool count;
- percentage of required evidence generated automatically;
- upgrade/migration completion before deprecation.

25.10 Developer-experience failure modes and anti-patterns

- **Portal over APIs:** attractive UI with manual backend. Make resources/API authoritative.
- **Golden path covers demo only:** no security, scale or rollback. Test with production archetypes.

- **Every team gets custom stack:** support and control collapse. Standardize contracts and extension points.
- **Policy denial without explanation:** users bypass controls. Return rule, rationale and remediation.
- **Docs written by component owners only:** fragmented user journey. Organize by tasks and archetypes.
- **SDK hides release identity:** debugging/audit gap. Expose trace and version metadata.
- **Platform adoption mandated, not earned:** shadow AI. Measure flow and remove friction.
- **No deprecation help:** old versions persist. Provide compatibility reports and migration automation.

26. Technology selection and build-versus-buy

26.1 Selection objective

Choose a coherent portfolio that satisfies enterprise contracts, not a collection of locally optimal tools. The unit of decision is a capability with interfaces, ownership, data flow, SLO and exit plan.

26.2 Evaluation criteria

A weighted scorecard can use:

Criterion	Example weight	Questions
Security, privacy and compliance	20%	identity, isolation, encryption, audit, residency, provider terms
Functional fit	20%	dominant workloads, lifecycle completeness, APIs and extensibility
Operability and reliability	15%	HA, upgrades, backup, support, observability, failure behavior
Integration and portability	15%	open formats, APIs, Kubernetes/cloud fit, data/model export
Total cost and economics	15%	licenses, compute, labor, egress, commitments, growth and exit
Performance and scale	10%	representative latency, throughput, concurrency and limits
Vendor/project viability	5%	roadmap, community, support, financial/project health

Weights vary by capability. Security and operability should be pass/fail for critical services, not only weighted offsets.

26.3 Build-versus-buy decision

Favor managed/buy

- capability is commodity and not differentiating;
- speed to market is critical;
- provider meets data, security, availability and version requirements;
- demand is uncertain or bursty;
- internal operating expertise is scarce;
- exit is feasible and cost is acceptable.

Favor open/self-host/build

- workflow/data/model is strategically differentiating;
- residency, latency or isolation requires control;
- steady volume gives favorable unit economics;

- provider version or policy opacity is unacceptable;
- deep runtime optimization or edge deployment is required;
- enterprise already operates the substrate reliably.

Avoid custom build when

- the problem is an undifferentiated registry, scheduler, vector database or observability primitive;
- no permanent product team and SLO exist;
- build cost excludes upgrades, security and support;
- standard interfaces can meet requirements.

Build integration and enterprise contracts where necessary; do not rebuild mature infrastructure without a measured reason.

26.4 Proof-of-value protocol

Evaluate on representative workloads, not vendor demos:

1. define workload and acceptance thresholds before access;
2. use approved masked/synthetic or controlled real data;
3. test quality, tail latency, concurrency, failure, recovery and cost;
4. test identity, tenancy, egress, logging, deletion and audit;
5. execute upgrade/version change and artifact export;
6. integrate with registry, CI/CD, policy and telemetry;
7. estimate three-year TCO and operating team;
8. document gaps, compensating controls and exit;
9. run architecture/security/legal/procurement review;
10. select, reject or time-bound further trial.

26.5 Vendor and managed-model due diligence

- architecture and shared-responsibility model;
- certifications and control evidence relevant to scope;
- data retention/training use and deletion verification;
- data location, subprocessors and support access;
- tenant isolation and encryption/key options;
- model/version change, pinning, deprecation and rollback;
- rate limits, quota, capacity and regional availability;
- SLO, support response and incident notice;
- vulnerability disclosure and abuse response;
- logging/telemetry export and audit APIs;
- IP, model/data license and indemnity terms;
- pricing dimensions, egress and commitment;
- export formats, migration assistance and termination;
- financial/strategic viability and concentration risk.

26.6 Portability strategy

Portability is not “runs everywhere.” Define the assets worth moving:

- source, data contracts and transformation code;
- open table and model formats where feasible;
- model weights/adapters/tokenizers subject to license;
- prompts, evaluation sets and system cards;

- release manifests and telemetry schema;
- provider-neutral application API;
- infrastructure modules and deployment configuration;
- corpus/source manifests enabling index rebuild;
- cost and usage history.

Test portability annually for critical dependencies. A documented exit plan that has never loaded an artifact or rerun evaluation elsewhere is unverified.

26.7 Platform stack patterns

Lean managed-first

Use managed data, training, model APIs/endpoints and observability with an enterprise gateway, catalog, evaluation and governance layer. Appropriate for moderate scale and small platform teams. Main risks: lock-in, provider drift and fragmented evidence.

Portable cloud-native

Use Kubernetes, object/lakehouse storage, Kubeflow/Argo/Airflow, MLflow, KServe, vLLM/Triton, Feast, OpenTelemetry and policy/signing components. Appropriate when control, multi-runtime and steady scale justify operations. Main risks: integration and upgrade burden.

High-assurance federated

Dedicated management plane plus regional/domain data planes, strict workload identity, private networks, signed supply chain, independent evaluation and dedicated critical capacity. Appropriate for regulated/high-impact environments. Main risks: cost, lead time and duplication unless templates are strong.

Hybrid

Managed model/data services behind enterprise interfaces combined with self-hosted workloads where economics, privacy or performance demand it. Often the pragmatic enterprise target; requires strong routing, inventory and consistent evidence.

26.8 Technology portfolio governance

- approved, conditional and prohibited product catalog;
- owner, SLO, version and end-of-life for every platform component;
- reference interfaces and data formats;
- duplicate-tool review and consolidation;
- quarterly vendor/project risk and cost review;
- vulnerability and upgrade roadmap;
- design authority for new stateful dependencies;
- exit tests for critical services;
- exception expiry and migration plan;
- user feedback and capability adoption.

26.9 Selection failure modes and anti-patterns

- **Logo architecture:** products listed without contracts or ownership. Define flows and SLOs.
- **Suite-first decision:** assumes integration and governance are solved. Test exact workflows and evidence.
- **Benchmark procurement:** vendor benchmark differs from use case. Use representative evaluation.
- **Open source equals free:** ignores operations and upgrade labor. Full TCO.

- **Multi-cloud by duplication:** doubles complexity without tested portability. Abstract only critical interfaces and run exit drills.
- **Best-of-breed everywhere:** integration tax. Optimize portfolio, not component scores.
- **Vendor proof uses unrestricted public data:** security constraints appear later. Test real boundary early.
- **No decommission plan:** tools accumulate. Contract renewal requires adoption/value and exit review.

27. Implementation roadmap

27.1 Roadmap principles

- Deliver one production value stream while building reusable platform capability; avoid a platform built in isolation.
- Sequence identity, data authorization, artifact integrity, evaluation and observability before broad self-service.
- Start with a small set of workload archetypes that represent future demand.
- Define enterprise contracts and metadata before selecting every product.
- Automate evidence as capabilities are built; retrofitting lineage and policy is expensive.
- Establish operating ownership and on-call before multiplying workloads.
- Use stage exit criteria and measurable outcomes; a roadmap of product installations is not an implementation plan.

27.2 Workstreams

Run five coordinated workstreams:

1. **Platform foundation:** identity, network, runtime, storage, orchestration, registry, deployment and telemetry.
2. **Data and knowledge:** source contracts, catalog/lineage, quality, features, labeling, document and retrieval services.
3. **Trust:** risk tiering, policies, security, privacy, evaluation, evidence and incidents.
4. **Operating model:** product management, team topology, service catalog, support, SRE and FinOps.
5. **Lighthouse products:** representative use cases that prove value and harden golden paths.

Every sprint should produce both platform increments and evidence from a real workload.

27.3 Days 0–30: mandate, baseline and architecture

Objectives

Create decision rights, inventory current activity, select lighthouse workloads and establish the non-negotiable architecture/control contracts.

Deliverables

- executive sponsor, platform product owner and cross-functional steering group;
- current-state inventory of AI systems, data flows, vendors, tools, costs and teams;
- use-case intake and risk/service-tier taxonomy;
- initial acceptable/prohibited-use, data/provider and release policies;
- target logical architecture and control/data-plane topology;
- service catalog MVP and top three golden paths;
- lighthouse portfolio: one classical ML, one LLM/RAG and optionally one bounded agent or edge case;
- baseline KPIs: lead time, deployment failure, incident, cost and existing control gaps;
- product/vendor decisions that are genuinely blocking, with time-boxed evaluations;
- 12-month funding, staffing and capacity hypothesis.

Exit criteria

- owners and decision rights accepted;
- lighthouse use cases have G0 approval and measurable baselines;

- architecture principles and minimum controls approved;
- current unregistered high-risk/shadow AI has containment plan;
- first 90-day backlog and outcomes committed.

27.4 Days 31–90: secure platform foundation

Platform foundation

- enterprise SSO, project/tenant provisioning and workload identity;
- development/test/production trust zones and private networking;
- object/artifact and container registries with versioning;
- source control, trusted CI builders, signing and scan pipeline;
- managed workspace and remote job execution;
- experiment tracking and initial model registry;
- basic workflow orchestration and deployment path;
- OpenTelemetry-compatible trace/metric/log foundation;
- cost tags, quotas and showback;
- backup and restore for control-plane metadata and artifacts.

Data and trust

- data-source onboarding contract and catalog integration;
- versioned lakehouse/warehouse access for lighthouse data;
- quality checks and lineage for at least one pipeline;
- risk-tiered release checklist and evaluation policy;
- model/system card templates and approval workflow;
- approved external model gateway rather than direct provider keys;
- incident taxonomy, kill switch, rollback and support contacts.

Lighthouse outcomes

- reproducible baseline and candidate pipelines;
- first governed batch or noncritical endpoint in production;
- first LLM request routed through enterprise gateway with trace and cost;
- first release evidence package generated mostly from automation.

Exit criteria

- workspace-to-release path is reproducible;
- production accepts only registered, digest-pinned artifacts;
- identity, network, telemetry, rollback and cost controls function end to end;
- support and incident process has completed a tabletop.

27.5 Months 3–6: production MLOps and LLMOps MVP

MLOps

- reusable training/evaluation pipeline templates;
- feature definitions and point-in-time dataset support;
- batch and online serving golden paths;
- champion/challenger and delayed outcome linkage;
- data/feature drift and performance monitoring;
- continuous-training candidate workflow with normal gates.

LLMOps

- prompt and system-release registry;

- policy-aware LLM gateway with provider/model catalog;
- representative offline evaluation and human review;
- enterprise RAG service with document ACLs, hybrid retrieval and citations;
- LLM traces, token/latency/cost and safety telemetry;
- self-hosted SLM option if economics/privacy justify it;
- provider version-change detection and fallback.

Governance and operations

- AI inventory integrated with project/deployment provisioning;
- automated control profiles and approval routing;
- SLO/error-budget, on-call and production-readiness review;
- vulnerability/patch, upgrade and deprecation policies;
- FinOps unit metrics and monthly value/cost review;
- data deletion propagation test across one complete lineage.

Exit criteria

- at least two workload archetypes meet production SLOs and value targets;
- full release manifest links data/model/prompt/index/runtime/evaluation;
- independent validation operates for high-risk releases;
- rollback, provider outage and telemetry-failure exercises pass;
- platform adoption is voluntary for new lighthouse-adjacent teams because it is faster than bespoke delivery.

27.6 Months 6–12: scale, federation and assurance

- domain onboarding kit and delegated project/data administration;
- multi-tenant quotas, fair-share scheduling and dedicated high-risk isolation;
- distributed training/HPO and accelerator capacity portfolio;
- multi-model inference, routing, cache and performance optimization;
- centralized feature store only where reuse/online needs justify it;
- corpus/index lifecycle, freshness and deletion automation;
- red-team service and incident-derived regression library;
- controls-as-code for deployment, provider route, region and agent tools;
- regional artifact replication and disaster recovery;
- platform SLOs, service desk tiers and known-error database;
- chargeback or stronger showback and forecast accuracy;
- tool consolidation and retirement of duplicate stacks;
- quarterly governance and value review with executive scorecard.

Exit criteria

- majority of new production AI uses golden paths;
- platform/control-plane SLOs and recovery objectives are met;
- risk-tier evidence and recertification are automated at scale;
- unit cost and lead time improve while incident/control breach rate does not worsen;
- domain teams can deliver without central-team handholding for standard cases.

27.7 Months 12–24: optimization and strategic scale

- active-active or regional autonomy where justified and tested;
- advanced routing across classical models, SLMs, LLMs and providers;
- high-throughput self-hosted inference, disaggregated execution or specialized accelerators where economics prove value;
- model distillation, quantization and edge deployment factory;

- bounded agent platform with durable workflows, tool broker, approvals and transaction controls;
- continuous assurance from production samples and rotating hidden evaluations;
- confidential-compute or privacy-enhancing patterns for selected high-sensitivity workloads;
- enterprise knowledge governance and reusable domain ontologies/graphs where valuable;
- automated capacity portfolio and scenario forecasting;
- certification or external assurance where strategically required;
- platform contribution model and reusable component marketplace;
- systematic retirement of low-value models and providers.

27.8 Recommended implementation backlog by dependency

Foundation dependencies

1. identity and project boundaries;
2. private networking and approved egress;
3. durable object/artifact storage;
4. source control, trusted CI and signatures;
5. registry and release identity;
6. telemetry and cost attribution;
7. orchestration/deployment;
8. data contract/catalog/lineage;
9. evaluation and policy gates;
10. workload-specific services.

Do not deploy a sophisticated feature store, vector database or agent framework before identity, release, telemetry and ownership exist.

27.9 Migration of existing systems

Classify existing workloads:

- **Adopt:** compatible and valuable; onboard to registry, telemetry and policy.
- **Wrap:** retain runtime temporarily behind gateway/identity/monitoring.
- **Replatform:** move to golden path due to reliability, cost or control.
- **Retire:** low value, unsupported or excessive risk.
- **Contain:** high-risk system requiring immediate traffic/data/tool restrictions.

Migration sequence:

1. inventory owners, consumers, data, provider, release and cost;
2. establish trace and runtime identity without changing behavior;
3. register current production as baseline release;
4. run retrospective evaluation and risk assessment;
5. add rollback and incident controls;
6. migrate data/model/prompt/deployment components incrementally;
7. compare shadow/canary behavior;
8. cut over and decommission old identities/endpoints;
9. verify evidence, deletion and cost closure.

27.10 Staffing and capability model

Roles may be combined initially, but capabilities must exist:

- platform product management;
- platform/backend engineering;

- data platform/engineering;
- MLOps/LLMOps and inference performance;
- SRE/operations;
- security and privacy engineering;
- evaluation/model risk/responsible AI;
- developer experience/documentation;
- FinOps and vendor management;
- domain product and subject-matter expertise.

Scale central staffing with number of supported archetypes and platform SLO, not number of models alone. Domain teams remain responsible for product outcomes and system-specific behavior.

27.11 Program risks

- building horizontal platform before validating user journeys;
- choosing a suite before architecture and contracts;
- governance queues without automation;
- underfunding data ownership and outcome measurement;
- no permanent operations team;
- overcommitting GPU capacity before demand profile;
- fragmented provider keys and direct app integrations;
- missing migration/deprecation authority;
- using lighthouse teams so exceptional that patterns do not generalize;
- measuring installations rather than lead time, value, reliability and adoption.

28. Maturity model and executive scorecard

28.1 Maturity levels

- **Level 0 — Ad hoc:** individual notebooks/APIs, manual deployment, unknown inventory and reactive incidents.
- **Level 1 — Repeatable:** source control, basic pipelines, named owners and standard environments for some teams.
- **Level 2 — Standardized:** common golden paths, registry, CI/CD, telemetry, risk tiers and production readiness.
- **Level 3 — Governed self-service:** policy/evidence automation, federated ownership, SLOs, unit economics and broad adoption.
- **Level 4 — Optimized:** continuous assurance, risk-aware routing, predictive capacity, tested portability and systematic value optimization.

Do not average away critical gaps. A high-risk production estate cannot claim Level 3 if incident response or artifact provenance remains Level 0.

28.2 Maturity matrix

Dimension	L0 Ad hoc	L1 Repeatable	L2 Standardized	L3 Governed self-service	L4 Optimized
Strategy/value	model-led pilots	named use cases	portfolio and baselines	value realization and WIP control	dynamic portfolio optimization
Operating model	hero teams	informal platform group	product team and RACI	federated domains and service catalog	contribution ecosystem and measured platform product
Data/knowledge	copies and unknown lineage	versioned sources in pockets	contracts, catalog and quality	feature/index services with deletion/ACL	adaptive quality and reusable knowledge assets

Dimension	L0 Ad hoc	L1 Repeatable	L2 Standardized	L3 Governed self-service	L4 Optimized
Development	local notebooks	standard images/trackers	managed workspaces and templates	policy-aware self-service	automated environment optimization
Training	scripts	tracked runs	reusable pipelines/HPO	distributed, quota/cost governed	efficient model/data strategy and predictive scheduling
Evaluation	one metric	repeatable test set	risk-tier suites and segments	independent validation and continuous regression	rotating hidden tests and continuous assurance
Release	manual copy	model registry	signed system releases and CI/CD	controls-as-code and progressive delivery	risk-aware automated promotion within policy
Serving	bespoke endpoints	standard runtime	gateway, batch/online patterns	routing, RAG/agent controls, multi-region	adaptive quality/cost routing and edge factory
Observability	logs only	system metrics	end-to-end traces and model/data monitoring	quality/safety/value SLOs	causal diagnostics and automated safe response
Security/privacy	broad keys/access	baseline scans	zero trust, supply chain and DLP	continuous control monitoring and red team	attested/confidential and threat-adaptive controls
Governance	spreadsheets	owner/risk questionnaire	inventory, gates and evidence	federated automated governance	continuous recertification and control effectiveness
Reliability	reactive	basic on-call	SLO, rollback and DR	error budgets, game days and platform SRE	predictive resilience and cell-based containment
FinOps	invoice review	tags and quotas	unit costs and forecasts	routing/capacity portfolio and chargeback	value-based autonomous optimization within guardrails
Developer experience	tribal knowledge	docs and starter repo	golden paths and portal	high self-service/adoption	platform evolves from telemetry and user research

28.3 Assessment method

1. Define evidence required for each cell.
2. Score by workload archetype and risk tier, not only enterprise average.
3. Use the lowest critical control as a constraint for production expansion.
4. Select a 6–12 month target based on strategy, not a universal goal of Level 4.
5. Fund the few capability gaps that block value or create material risk.
6. Reassess quarterly using measured evidence.

28.4 Executive scorecard

Value

- realized annualized benefit versus approved case;
- percentage of systems with active business KPI and counterfactual method;
- adoption, task completion and human effort changed;
- portfolio value concentration and stopped/retired low-value work.

Flow

- intake-to-G0, G0-to-first value and change lead time;
- deployment frequency, evaluation duration and approval wait;

- golden-path adoption and self-service rate;
- WIP, stage aging and top blockers.

Trust and risk

- inventory and owner coverage;
- current risk/impact/evaluation/recertification coverage;
- critical control breaches, incidents, complaints and appeals;
- exception count/age, security vulnerabilities and provider concentration;
- high-risk independent-validation coverage.

Reliability and quality

- service and quality/safety SLO attainment;
- change failure, rollback, MTTR and repeat incidents;
- data/feature/index freshness and telemetry completeness;
- production performance by critical segment.

Economics

- total AI run rate and forecast variance;
- cost per successful outcome by product;
- GPU/reservation utilization and provider/model mix;
- training/evaluation/inference/observability cost distribution;
- savings from routing, quantization, caching and retirement validated against quality.

Platform health

- active teams/services and retention;
- platform SLO and support metrics;
- percentage of evidence automated;
- version compliance and migration status;
- customer satisfaction and duplicate-stack reduction.

28.5 Example target bands

Targets must be enterprise-specific. Illustrative mature-state bands:

- 90% of production AI systems inventoried with current owners and release identity;
- 90% of new standard workloads delivered through golden paths;
- 95% of production releases have signed provenance and complete required evaluation;
- 99% trace identity coverage for consequential decisions/actions;
- 100% of critical services with tested rollback and annual or more frequent DR exercise;
- 0 expired high-risk approvals or unowned critical exceptions;
- measurable reduction in median time to production and cost per successful outcome year over year.

Never use a target that incentivizes hiding incidents, lowering test coverage or registering trivial models to inflate counts.

29. Troubleshooting and operational runbooks

29.1 Universal triage sequence

For any AI production issue:

1. **Confirm impact.** Users, decisions, tenants, regions, safety/privacy and business process.
2. **Identify release.** Model, prompt, index, feature, policy, provider, tool and runtime versions.
3. **Compare change timeline.** Deployment, data/schema, corpus, provider, traffic and infrastructure.
4. **Inspect end-to-end trace.** Queue, preprocessing, feature/retrieval, model, tools and postprocessing.
5. **Contain safely.** Reduce cohort, disable route/tool, rollback bundle, use fallback or manual process.
6. **Preserve evidence.** Relevant logs/traces/config under privacy and chain-of-custody controls.
7. **Diagnose by layer.** Infrastructure, dependency, data, feature, model, prompt, retrieval, policy, tool or business process.
8. **Verify recovery.** Synthetic, targeted regression and business checks.
9. **Correct affected outputs/actions.** Do not stop at endpoint recovery.
10. **Add prevention.** Test, monitor, runbook, capacity or design change.

29.2 Runbook: latency or throughput regression

Trigger: p95/p99 latency, queue wait, TTFT or batch deadline breach.

Immediate containment: enforce admission/rate limits; disable expensive optional steps; route permitted traffic to fallback; add warm capacity; pause best-effort workloads.

Diagnostics:

- compare traffic and input/token-length distribution;
- inspect queue and per-span latency;
- check feature/vector/tool/provider dependencies;
- inspect accelerator utilization, memory, cache and batch size;
- compare current release/runtime/driver/config;
- look for retry storms, cold starts, model reloads or network loss;
- test one request and controlled concurrency to locate saturation knee.

Likely fixes: context cap; retrieval/rerank tuning; dynamic-batch adjustment; prewarm/cache; scale bottleneck; isolate noisy tenant; rollback runtime/model; restore provider quota.

Prevention: representative load test, capacity headroom, autoscale on queue/tokens, request classes and maximum context.

29.3 Runbook: GPU out-of-memory or model-load failure

Containment: stop restart storm; reduce replica count only if memory contention; route to smaller/fallback model; preserve logs and node state.

Diagnostics: weight size/precision, KV-cache reservation, max sequence/concurrency, fragmentation, duplicate model copies, runtime version, adapter merge, node/device health.

Fixes: lower batch/concurrency/context; quantize with quality validation; tensor/pipeline shard; tune cache; preallocate/restart cleanly; correct memory limit; use compatible hardware/runtime.

Prevention: load test worst-case context/concurrency, model-aware readiness, memory headroom and admission limits.

29.4 Runbook: data or feature quality degradation

Trigger: freshness, schema, null/default, distribution, online/offline parity or performance breach.

Containment: pause affected batch publication; route to previous model or rules; disable stale feature; use manual review for consequential decisions.

Diagnostics: source health, schema changes, contract/quality reports, materialization lag, entity-key mismatch, event-time/late data, transformation release, replay/backfill.

Fixes: source rollback/repair; quarantine new schema; replay from verified checkpoint; restore feature release; retrain only after data correctness and label impact are understood.

Prevention: producer contracts, reconciliation, point-in-time tests, feature defaults as explicit policy and incident-linked evaluation cases.

29.5 Runbook: RAG quality degradation

Trigger: known-answer miss, citation failure, stale answer, zero-result or user complaint.

Containment: show search-only results or previous index; reduce unsupported answer confidence; disable affected corpus/tenant if ACL concern.

Diagnostics by component:

1. source exists and is authorized;
2. parser extracted required content;
3. chunk IDs/metadata/source offsets correct;
4. embedding completed with expected model/version;
5. index contains chunk and filters allow it;
6. query rewrite/search/rerank returns it;
7. context selection includes it within budget;
8. generator follows evidence and citation schema.

Fixes: source connector/parse/chunk repair; re-embed/reindex; filter/ACL fix; hybrid search/rerank tuning; context budget/prompt update; rollback index or model.

Prevention: component benchmarks, corpus coverage, index freshness SLO, ACL regression and release-coupled index versions.

29.6 Runbook: hallucination, unsafe output or over-refusal spike

Containment: reduce exposure; route to prior release/model; require human review; tighten permitted task; disable unsupported generation.

Diagnostics: release/prompt/provider/index/policy changes; retrieval hit/grounding; input segment; temperature/sampling; safety classifier/judge version; adversarial traffic; context truncation.

Fixes: restore known-good bundle; improve retrieval/context; change prompt/output schema; adjust deterministic policy; fine-tune only after representative evidence; block abuse source.

Prevention: groundedness/refusal regression, rotating adversarial sets, provider-change probes, segment monitoring and human-calibrated safety evaluation.

29.7 Runbook: prompt injection or suspected data exfiltration

Immediate containment: revoke/disable affected tool and provider route; isolate tenant/session; stop agent actions; rotate exposed credentials; preserve evidence; engage security/privacy/legal.

Diagnostics: direct versus retrieved/tool injection; data sent to model/tool; policy decision; tool grants; cache/session keys; egress logs; output/log exposure; affected identities and records.

Remediation: remove malicious source; rebuild index/cache; tighten tool schemas/permissions; move secrets to broker; enforce output handling; patch parser/rendering; notify/correct as required.

Prevention: treat content as data, deterministic authorization, sandbox, DLP, injection regression, source trust metadata and scoped credentials.

29.8 Runbook: agent loop or incorrect external action

Containment: global/scoped action freeze; revoke tool credential; stop workflow; reconcile external system; trigger compensation or manual correction.

Diagnostics: workflow state, repeated steps, tool errors, retries/idempotency, approval path, model/prompt change, stale memory, timeout/budget and external postconditions.

Fixes: bounded state machine; repeated-state detection; correct tool schema/preconditions; idempotency key and transaction journal; approval for action class; stronger postcondition verification.

Prevention: simulation, fault injection, maximum steps/time/cost, explicit action classes, compensating workflows and immutable action audit.

29.9 Runbook: managed provider outage, rate limit or silent change

Containment: circuit break; queue or degrade; route only to preapproved fallback; disclose reduced capability; preserve request IDs and provider status.

Diagnostics: provider errors/latency/quotas, region, model/version headers, output regression probes, gateway config and contract limits.

Fixes: capacity/quota increase; regional/provider failover; rollback enterprise route; pin version where possible; update evaluation and release if provider changed behavior.

Prevention: provider abstraction, synthetic probes, dual-source strategy for critical workloads, contractual change notice, fallback capacity and exit tests.

29.10 Runbook: cost or token runaway

Containment: tenant/workflow rate and budget limit; stop retry/agent loop; route to cheaper approved model; pause batch/embedding backlog; notify owner/FinOps.

Diagnostics: requests, prompt/output length, model route, retries, cache hit, tool calls, agent steps, traffic source, release and provider pricing/commitment.

Fixes: context/output caps; idempotency; routing; batching/cache; smaller model; quota and anomaly threshold; remove duplicate calls.

Prevention: per-request and monthly budgets, cost trace, maximum steps/tokens, forecast by driver and cost per successful task.

29.11 Runbook: cross-tenant or authorization leak

Severity: treat as security/privacy incident even if exposure is brief.

Containment: disable affected cache/index/endpoint; isolate tenants; revoke access; preserve audit; notify security/privacy/legal.

Diagnostics: identity propagation, ACL/filter, cache key, vector index partition, session memory, backup/log access, support/admin actions and affected records.

Fixes: enforce authorization before retrieval/cache; tenant in every key; rebuild/invalidate derived state; correct policy; rotate secrets; user notification/correction as required.

Prevention: automated isolation tests, canary tenant markers, least privilege, separate high-risk stores and trace identity completeness.

29.12 Runbook: monitoring or audit pipeline failure

Containment: declare observability impairment; freeze high-risk releases; increase synthetic checks/manual review; buffer critical audit events locally/durably.

Diagnostics: collector health, queue/backpressure, schema/cardinality, credentials/certificates, storage quota, network and alert-delivery path.

Fixes: scale/restore collector; reduce sampling/cardinality; repair credentials; replay buffer; validate alert path end to end.

Prevention: metamonitoring, external black-box probes, durable audit channel, telemetry SLO, cardinality budgets and fail-safe release policy.

29.13 Runbook: CI/CD or registry blockage

Containment: distinguish availability incident from policy failure; do not bypass signatures/evaluation casually. Use documented emergency pipeline only for urgent safe restoration.

Diagnostics: runner capacity, dependency mirror, scanner, signature trust, registry/object store, policy rule, approval expiry, artifact digest mismatch.

Fixes: scale/repair bottleneck; cache scans by digest; rotate trust/credential; correct metadata; restore replicated registry; use time-bound exception only with compensating controls.

Prevention: platform SLO, dedicated urgent capacity, trusted offline mirrors, replicated artifacts, policy tests and emergency drill.

29.14 Runbook template

Every runbook should contain:

- trigger and severity criteria;
- user/business and risk impact;
- safe containment commands/actions;
- required roles and escalation;
- diagnostic decision tree and dashboards;
- rollback/fallback/kill-switch procedure;
- evidence to preserve and privacy constraints;
- recovery verification and affected-output correction;
- communication template;
- prevention tests and owner;
- last exercise date and next review.

30. Appendices and reusable templates

30.1 Gate evidence checklist

Gate	Blocking questions	Required evidence	Automation target
G0 intake	Is use justified, owned, permitted and measurable?	intended/prohibited use, baseline, value, risk/service tier, data/provider concept, evaluation plan	intake validation, owner lookup, risk profile and project creation
G1 data	Is every input authorized, fit, versioned and recoverable?	contracts, lineage, classification, quality, representativeness, retention/deletion, replay	schema/quality/lineage and access checks
G2 experiment	Can another engineer reproduce the baseline/candidate safely?	source, environment lock, run lineage, tests, cost profile	workspace, CI, tracking and clean-room rerun
G3 validation	Does exact release meet contextual thresholds?	system evaluation, segments, robustness, security/privacy/fairness, performance/cost, residual risk	evaluation service and policy decision
G4 release	Are artifacts trusted, supportable and approved?	digest, signature, provenance, SBOM/model-BOM, cards, runbook, SLO, rollback	registry schema, signing and lifecycle policy
G5 launch	Is production behavior acceptable under bounded exposure?	smoke/load, canary/shadow, telemetry, capacity, cost and rollback evidence	progressive delivery and guardrail evaluation
G6 operate	Does evidence remain within approved bounds?	SLOs, incidents, value, access, provider/version, recertification	monitoring-to-ticket/ recertification workflow
G7 retire/improve	Is change or continued operation justified?	feedback, intervention decision, migration, archive/deletion	trigger, deprecation and cleanup workflow

30.2 Compact RACI

Legend: A = accountable, R = responsible, C = consulted, I = informed.

Activity	Product/domain owner	Data/model engineering	Platform/SRE	Security/privacy/risk	Data owner/validator
Use-case outcome and baseline	A/R	C	C	C	C
Risk tier and impact assessment	C	C	I	A/R	C
Data authorization and contract	C	R	C	C	A
Architecture and model selection	A	R	C	C	C
Experiment/training implementation	C	A/R	C	C	C
Evaluation policy	A	R	C	C	R/A for independent validation
Security/privacy controls	C	R	R	A	C
Release promotion	A	R	R	C/A by tier	A for high-risk validation
Production SLO and on-call	A	R	A/R	C	C
Incident command	C	R	A/R	R for security/privacy	C

Activity	Product/domain owner	Data/model engineering	Platform/SRE	Security/privacy/risk	Data owner/validator
Retraining/material change	A	R	C	C	C/A by tier
Recertification	C	R	C	A	R
Retirement	A	R	R	C	C

Assign named people, not only teams, for each production system.

30.3 Example declarative AI service manifest

The schema below is illustrative. Implement it as a custom resource, platform API object or versioned manifest.

```yaml
apiVersion: platform.example.com/v1
kind: AIService
metadata:
  name: claims-assist
  owner: claims-product
  costCenter: CC-417
  labels:
    domain: insurance
    environment: production
spec:
  intendedUse: "Assist trained adjusters with evidence-grounded claim summaries"
  prohibitedUse:
    - "Autonomous claim denial"
    - "Use outside approved jurisdictions"
  workload:
    type: rag-assistant       # batch-ml | online-ml | llm | rag | agent | edge
    serviceTier: gold
    riskTier: tier-2
    regions: [us-east, us-west]
  release:
    manifestDigest: sha256:...
    modelRef: registry/model/claims-slm@sha256:...
    promptRef: registry/prompt/claims-summary@sha256:...
    knowledgeIndexRef: registry/index/claims-policy@sha256:...
    policyRef: registry/policy/claims-assist@sha256:...
    runtimeImage: registry/runtime/vllm@sha256:...
  data:
    classifications: [confidential, personal-data]
    allowedPurposes: [claims-assistance]
    retentionPolicy: claims-assist-30d-redacted-telemetry
    externalProviderAllowed: false
  authorization:
    userGroups: [licensed-adjusters]
    retrievalAclMode: enforce-source-acl
    toolProfile: read-only-claims
  evaluation:
    policyRef: registry/eval-policy/claims-assist-v4
    reportDigest: sha256:...
    approvalRef: approval/AI-2026-00417
    expiresAt: 2026-12-31T23:59:59Z
  deployment:
    strategy: canary
    initialTrafficPercent: 2
    maxTrafficPercent: 100
    rollbackRelease: registry/release/claims-assist-v18
    autoscaling:
      minReplicas: 3
      maxReplicas: 20
```

```
      signal: active-tokens
    limits:
      maxInputTokens: 12000
      maxOutputTokens: 1200
      maxRequestCostUsd: 0.08
  slo:
    availability: 99.9
    p95LatencyMs: 4500
    maxIndexAgeMinutes: 30
    minGroundedAnswerRate: 0.96
  observability:
    traceSampling: risk-aware
    payloadMode: redacted
    dashboardRef: dashboard/claims-assist
    runbookRef: runbook/claims-assist
```

30.4 Example data contract

```
apiVersion: data.example.com/v1
kind: DataContract
metadata:
  name: customer-events-v3
  owner: customer-data-product
spec:
  purpose: [churn-model-training, churn-online-features]
  classification: confidential
  residency: [us]
  schema:
    compatibility: backward
    key: event_id
    eventTimeField: occurred_at
    fields:
      - name: event_id
        type: string
        nullable: false
      - name: customer_id_token
        type: string
        nullable: false
        semanticType: pseudonymous-identifier
      - name: event_type
        type: enum
        allowedValues: [login, purchase, support, cancel]
      - name: occurred_at
        type: timestamp
        nullable: false
  serviceLevels:
    freshnessMinutes: 10
    completenessPercent: 99.8
    duplicateRateMax: 0.001
    availabilityPercent: 99.9
  changes:
    noticeDays: 30
    incompatibleChangeRequiresApproval: true
  retention:
    rawDays: 30
    curatedDays: 730
    deletionPropagationHours: 72
  quality:
    quarantineOnCriticalFailure: true
    reconciliationFields: [record_count, purchase_total]
  contacts:
    support: data-customer-oncall
    escalation: customer-data-owner
```

30.5 Example evaluation policy

```yaml
apiVersion: eval.example.com/v1
kind: EvaluationPolicy
metadata:
  name: support-rag-release-v7
spec:
  appliesTo: [model, prompt, index, reranker, policy, runtime]
  baseline: production
  datasets:
    - ref: eval/support-golden-v12
      access: protected
    - ref: eval/support-adversarial-v6
      access: restricted
  segments:
    required: [language, product, customer-tier, document-age]
    minimumCasesPerBlockingSegment: 100
  gates:
    - metric: end_to_end_task_success
      rule: candidate >= 0.92
      blocking: true
    - metric: grounded_answer_rate
      rule: lower_95ci >= 0.95
      blocking: true
    - metric: retrieval_recall_at_10
      rule: candidate >= 0.97
      blocking: true
    - metric: citation_precision
      rule: candidate >= 0.98
      blocking: true
    - metric: critical_policy_violation
      rule: count == 0
      blocking: true
    - metric: valid_request_refusal_rate
      rule: candidate <= 0.03
      blocking: true
    - metric: p95_latency_ms
      rule: candidate <= 5000
      blocking: true
    - metric: cost_per_success_usd
      rule: candidate <= 0.12
      blocking: false
  statisticalDesign:
    pairedComparison: true
    confidenceLevel: 0.95
    nonInferiorityMargins:
      task_success: 0.01
  humanEvaluation:
    blind: true
    ratersPerCase: 2
    adjudicateDisagreement: true
    minimumAgreement: 0.75
  securitySuites:
    - prompt-injection-direct
    - prompt-injection-indirect
    - cross-tenant-retrieval
    - sensitive-data-exfiltration
  approvers:
    requiredByRiskTier:
      tier-1: [service-owner]
      tier-2: [service-owner, risk-owner]
      tier-3: [service-owner, independent-validator, security, risk-owner]
```

Thresholds above are illustrative; derive real thresholds from business harm, baseline and statistical power.

30.6 Model card template

Identity

- model name, version, digest and owner;
- source/base model, license and provenance;
- architecture, size, tokenizer and context limits;
- training/fine-tuning method and runtime formats.

Intended use

- supported tasks, domains, languages and populations;
- prohibited or unsupported uses;
- required human oversight and downstream controls.

Data

- training/validation sources and date ranges;
- collection, filtering, deduplication and labeling;
- sensitive data, consent/purpose and licensing;
- representativeness, exclusions and known gaps.

Evaluation

- datasets, metrics, segments and baselines;
- robustness, fairness, privacy and security tests;
- statistical uncertainty and known failure modes;
- performance after quantization/export.

Operation

- hardware/runtime and resource profile;
- latency/throughput and cost envelope;
- calibration/thresholds or decoding parameters;
- monitoring and retraining triggers;
- deprecation and support.

Limitations and risks

- out-of-domain behavior, uncertainty and explainability;
- foreseeable misuse and mitigations;
- residual risk and approval scope.

30.7 System card template

A system card extends the model card:

1. Business objective, intended users and process integration.
2. Architecture and end-to-end data flow.
3. Exact release components: models, prompts, retrieval, policies, tools and runtime.
4. User notice, human oversight, contest/recourse and accessibility.
5. Data classes, provider routes, retention and deletion.
6. Evaluation and release thresholds, including system and human performance.
7. Security threat model and agent/action boundaries.
8. SLOs, fallback, rollback, incident and continuity.
9. Monitoring, feedback, value and recertification.

10. Known limitations, residual risk, approvals and expiration.

30.8 Evaluation plan template

- **Decision:** what release decision will this evidence support?
- **System boundary:** exact artifacts and production-like configuration.
- **Baseline:** current process, prior release and simple model/rules.
- **Population/traffic:** segments, volumes, languages, edge cases and exclusions.
- **Outcomes:** primary metric, guardrails and minimum practical effect.
- **Datasets:** source, independence, sample size, split and contamination control.
- **Metrics:** task, calibration, retrieval, generation, agent, human and business.
- **Thresholds:** blocking/warning, confidence interval and non-inferiority margin.
- **Safety/security/privacy:** threats, misuse and test suites.
- **Fairness/impact:** relevant groups, rationale, metrics and remediation.
- **Performance/economics:** latency, throughput, capacity and cost.
- **Human evaluation:** rubric, blinding, raters, agreement and adjudication.
- **Online evidence:** shadow/canary/A-B design and stop rules.
- **Independence:** builder/validator access and decision rights.
- **Evidence/retention:** artifacts, approvers, storage and expiry.

30.9 Production-readiness checklist

Ownership and scope

- ☐ accountable product, technical, data/model and operational owners;
- ☐ intended/prohibited use and risk/service tiers current;
- ☐ user notice, oversight, recourse and support defined.

Artifacts and evidence

- ☐ immutable system release and rollback release;
- ☐ complete lineage, model/system card and evaluation report;
- ☐ signatures, provenance, SBOM/model-BOM and scans pass;
- ☐ approvals current and exceptions unexpired.

Data and model

- ☐ production data access, classification, retention and deletion approved;
- ☐ feature/index freshness and ACL tests pass;
- ☐ offline/online parity or batch reconciliation verified;
- ☐ model/prompt/provider/runtime versions pinned or monitored.

Security and privacy

- ☐ workload identity, least privilege and private network;
- ☐ secrets brokered; egress/provider route controlled;
- ☐ tenant cache/index/session isolation tested;
- ☐ prompt injection, output handling and tool controls tested;
- ☐ incident contacts and evidence policy ready.

Reliability and operations

- ☐ SLOs, dashboards, alerts and runbooks reviewed;
- ☐ load, stress, dependency failure and rollback tests pass;
- ☐ autoscaling/capacity and quotas configured;
- ☐ backup/restore and DR match service tier;

- ☐ on-call access and escalation exercised.

Launch and economics

- ☐ shadow/canary cohort and abort thresholds defined;
- ☐ business and quality guardrails observable;
- ☐ cost forecast, budget and anomaly controls active;
- ☐ fallback/manual process accepted;
- ☐ post-launch review date scheduled.

30.10 SLO specification template

Field	Definition
Service and user journey	exact request/process covered
Eligible events	valid requests or decisions included
Exclusions	planned maintenance, invalid input, upstream exclusions with rationale
SLI formula	numerator, denominator, data source and sampling
Objective/window	target and rolling/calendar window
Segments	tenant, region, language, risk or request class
Error budget	calculation and policy
Warning/page thresholds	burn rates, duration and minimum volume
Fallback/containment	action when threshold breaches
Owner/on-call	accountable team and escalation
Review cadence	business/technical review and changes

Recommended SLI categories: availability, end-to-end latency, quality/task success, safety/policy, data/index freshness, trace completeness and cost.

30.11 Incident severity matrix

Severity	Examples	Initial response	Executive/legal involvement
SEV-0 critical	ongoing serious harm; unauthorized irreversible actions; major sensitive-data exposure; critical system-wide compromise	immediate action freeze/kill switch, incident command and evidence preservation	immediate executive, security, privacy/legal and regulator/customer assessment
SEV-1 high	critical service outage; broad wrong decisions; cross-tenant leak; severe safety regression; signing/provider compromise	page 24×7, contain within minutes, frequent communications	executive/risk informed; legal/privacy as applicable
SEV-2 moderate	limited cohort quality issue; repeated tool failure; missed batch/SLO with material impact	respond within support target, reduce exposure/rollback	owner/risk notified; escalate by impact
SEV-3 low	noncritical defect, isolated incorrect output with safe fallback, documentation/control evidence issue	backlog or business-hours response	normal governance reporting

Severity is determined by actual and plausible harm, scope, reversibility, data sensitivity and criticality—not only number of HTTP failures.

30.12 Risk scoring worksheet

Use scoring to structure analysis, not replace judgment.

Dimension	0	2	4
Impact	informational	material workflow influence	rights, safety, essential service or legal consequence
Autonomy	no action	reversible action/recommendation	irreversible or financially/legal material action
Data sensitivity	public/synthetic	confidential/personal	highly sensitive, biometric, health, children or regulated
Scale/exposure	small internal	broad workforce/customer	public/large population/systemic
Reversibility/recourse	easy and immediate	delayed/manual	difficult, opaque or no practical recourse
Model/system opacity	deterministic/simple	complex but testable	highly nondeterministic, agentic or poorly observable
Adversarial exposure	trusted inputs	customer inputs	public/hostile inputs and high incentive to manipulate
Third-party dependency	none/replaceable	managed service	opaque critical provider/data/model concentration

Example classification: sum or weighted score proposes a tier, but override upward for prohibited/high-impact categories, children/vulnerable groups, biometric surveillance, autonomous irreversible action or material legal/regulatory scope. Record rationale and reviewer.

30.13 Capacity and cost formulas

Online concurrency

- `in_flight = arrival_rate_per_second × average_service_time_seconds`
- conservative sizing should use request classes and tail behavior rather than one average;
- `replicas = ceil(in_flight / (safe_concurrency_per_replica × target_utilization));`
- add capacity for one failure domain and autoscaling delay according to SLO.

Illustration: 40 requests/s × 0.4 s = 16 concurrent requests. If a replica safely handles 4 at 70% target utilization: `ceil(16 / 2.8) = 6` replicas. Add at least one replica or failure-domain capacity based on availability design.

LLM token throughput

- `prefill_tokens_per_second_required = requests_per_second × mean_input_tokens;`
- `decode_tokens_per_second_required = active_requests × mean_output_token_rate;`
- segment by prompt/output buckets and priority;
- validate with real batching and latency constraints.

LLM memory

- `total_memory ≈ weights + KV_cache + activations/workspace + runtime_headroom;`
- weight memory roughly equals parameter count × bytes/parameter plus quantization metadata;
- KV cache grows with layers, key/value width, active tokens, concurrency and bytes/element;
- maintain measured fragmentation/headroom rather than using a theoretical fit.

Composite availability

For independent serial critical dependencies, a rough upper bound is:

- `A_system ≈ A_gateway × A_feature_or_retrieval × A_model × A_policy`.

This shows why a chain of 99.9% dependencies cannot deliver 99.9% end-to-end without redundancy, graceful degradation or stronger component SLOs. Real dependencies are not perfectly independent; measure the user journey.

Cost

- `training_candidate_cost = compute + storage + data + HPO failures + engineering allocation`;
- `request_cost = model + retrieval + tool + gateway + network + observability + allocated capacity`;
- `successful_task_cost = total relevant cost / tasks meeting quality and business success criteria`.

30.14 Platform control checklist by architecture layer

Layer	Minimum controls
Experience	SSO/MFA, role-aware UI, no embedded secrets, audit and safe defaults
Control plane	HA, least privilege, signed policy/config, immutable audit, backup/restore
Data/knowledge	contracts, classification, lineage, quality, ACL, deletion and versioning
Training	approved data/model, isolated job, pinned environment, checkpoint and budget
Registry	digest, signature, provenance, lifecycle, approval and retention
Release	trusted CI, policy gates, GitOps/desired state, progressive rollout and rollback
Inference	gateway auth, quotas, routing policy, tenant isolation, safe fallback and tracing
RAG	source provenance, parse sandbox, ACL retrieval, index version, injection tests
Agents	typed allowlisted tools, brokered secrets, sandbox, budgets, approval, transaction log
Observability	common trace identity, privacy-aware telemetry, SLO, alert and metamonitoring
Infrastructure	segmentation, workload identity, patching, admission, backup, DR and capacity

30.15 Glossary

Term	Meaning in this playbook
Adapter	Small trainable parameter set applied to a compatible base model, such as LoRA
AI BOM/model BOM	Inventory of model, data, tokenizer, adapter, runtime and related components
Canary	Production exposure to a bounded cohort before wider rollout
Champion/challenger	Current production system compared with one or more candidates
Concept drift	Change in relationship between inputs and target/outcome
Continuous training	Policy-controlled generation of a new candidate from new data or triggers
Data drift	Change in input distribution; may or may not imply performance loss
Data plane	Workload execution and state for data, training, retrieval and inference
Evaluation policy	Versioned metrics, datasets, thresholds, segments and approval requirements

Term	Meaning in this playbook
Feature skew	Difference between training and serving feature values/logic
Golden path	Supported automated delivery pattern with secure operational defaults
Groundedness	Degree to which an answer is supported by supplied authoritative context
Human-in-command	Human authority to set limits, stop or redesign the system
Idempotency	Repeating an operation produces no additional unintended effect
KV cache	Transformer key/value state retained for active sequence generation
Material change	Change requiring impact assessment and defined reevaluation scope
Model router	Policy and optimization service selecting an approved model/provider
Point-in-time join	Historical join using only information available at the prediction time
Prompt injection	Input or embedded content attempts to alter instructions or obtain unauthorized behavior
RAG	Retrieval-augmented generation using external evidence supplied at inference
Release manifest	Immutable dependency graph identifying the complete deployable AI system
Shadow	Candidate processes real traffic without affecting user decisions/actions
SLM	Small language model selected for bounded capability, efficiency or edge/private use
System card	Documentation of deployed context, components, controls, evaluation and limitations
TEVV	Test, evaluation, verification and validation
TTFT	Time to first generated token
Workload identity	Cryptographic identity assigned to a running service/job rather than a human/static key

31. Primary references

Accessed 25 June 2026. Standards, regulations and software change; verify the current edition, legal applicability and supported product version before implementation.

[R1] National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1.* 2023. https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf

[R2] National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile, NIST AI 600-1.* 2024. https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf

[R3] ISO/IEC 42001:2023. *Information technology — Artificial intelligence — Management system.* https://www.iso.org/standard/42001

[R4] ISO/IEC 23894:2023. *Information technology — Artificial intelligence — Guidance on risk management.* https://www.iso.org/standard/77304.html

[R5] ISO/IEC 42005:2025. *Information technology — Artificial intelligence — AI system impact assessment.* https://www.iso.org/standard/42005

[R6] ISO/IEC TS 25058:2024. *Systems and software engineering — SQuaRE — Guidance for quality evaluation of artificial intelligence systems.* https://www.iso.org/standard/82570.html

[R7] European Union. *Regulation (EU) 2024/1689 (Artificial Intelligence Act)* and European Commission implementation timeline. https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng and https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai

[R8] National Institute of Standards and Technology. *Security and Privacy Controls for Information Systems and Organizations, SP 800-53 Rev. 5.* https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final

[R9] National Institute of Standards and Technology. *Zero Trust Architecture, SP 800-207.* https://csrc.nist.gov/pubs/sp/800/207/final

[R10] National Institute of Standards and Technology. *Secure Software Development Framework (SSDF) Version 1.1, SP 800-218.* https://csrc.nist.gov/pubs/sp/800/218/final

[R11] National Institute of Standards and Technology. *Secure Software Development Practices for Generative AI and Dual-Use Foundation Models, SP 800-218A.* https://csrc.nist.gov/pubs/sp/800/218/a/final

[R12] OWASP GenAI Security Project. *OWASP Top 10 for LLM Applications 2025.* https://genai.owasp.org/llm-top-10/

[R13] MITRE. *ATLAS — Adversarial Threat Landscape for Artificial-Intelligence Systems.* https://atlas.mitre.org/

[R14] Supply-chain Levels for Software Artifacts. *SLSA Specification v1.2.* https://slsa.dev/spec/v1.2/

[R15] Sigstore. *Cosign documentation: signing and verifying artifacts and attestations.* https://docs.sigstore.dev/cosign/

[R16] SPIFFE Project. *SPIFFE/SPIRE overview and workload identity.* https://spiffe.io/docs/latest/spiffe-about/overview/

[R17] National Institute of Standards and Technology. *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations, NIST AI 100-2 E2025.* https://csrc.nist.gov/pubs/ai/100/2/e2025/final

[R18] Confidential Computing Consortium. *About Confidential Computing.* https://confidentialcomputing.io/about/

[R19] Kubernetes. *Concepts and production environment documentation.* https://kubernetes.io/docs/concepts/overview/ and https://kubernetes.io/docs/setup/production-environment/

[R20] MLflow. *Tracking, model registry, dataset tracking and LLM/agent lifecycle documentation.* https://mlflow.org/docs/latest/

[R21] Kubeflow. *Kubeflow Pipelines concepts and component specifications.* https://www.kubeflow.org/docs/components/pipelines/concepts/

[R22] KServe. *Predictive and generative inference on Kubernetes.* https://kserve.github.io/website/docs/

[R23] Feast. *Feature store concepts, offline and online stores.* https://docs.feast.dev/

[R24] OpenLineage. *Open standard for data lineage.* https://openlineage.io/docs/

[R25] OpenTelemetry. *Observability framework, signals and semantic conventions.* https://opentelemetry.io/docs/ and https://opentelemetry.io/docs/specs/semconv/

[R26] Apache Iceberg. *Table specification and documentation.* https://iceberg.apache.org/docs/latest/

[R27] Delta Lake. *Documentation.* https://docs.delta.io/latest/

[R28] Apache Spark. *Structured Streaming Programming Guide.* https://spark.apache.org/docs/latest/streaming/index.html

[R29] Hugging Face. *Transformers and Parameter-Efficient Fine-Tuning documentation.* https://huggingface.co/docs/transformers/ and https://huggingface.co/docs/peft/

[R30] PyTorch. *Fully Sharded Data Parallel documentation.* https://docs.pytorch.org/docs/stable/fsdp.html

[R31] DeepSpeed. *ZeRO and distributed training tutorials.* https://www.deepspeed.ai/tutorials/zero/

[R32] Prometheus. *Monitoring and Alertmanager documentation.* https://prometheus.io/docs/introduction/overview/ and https://prometheus.io/docs/alerting/latest/overview/

[R33] FinOps Foundation. *FinOps Framework.* https://www.finops.org/framework/

[R34] FinOps Foundation. *FinOps for AI technology category and overview.* https://www.finops.org/framework/technology-categories/ai/ and https://www.finops.org/wg/finops-for-ai-overview/

[R35] vLLM. *Inference and distributed-serving documentation.* https://docs.vllm.ai/en/latest/

[R36] NVIDIA. *Triton Inference Server documentation.* https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/

[R37] Ray. *Ray Train and Ray Tune documentation.* https://docs.ray.io/en/latest/train/train.html and https://docs.ray.io/en/latest/tune/index.html

[R38] Great Expectations. *Checkpoints and production data validation.* https://docs.greatexpectations.io/

[R39] Open Policy Agent. *Policy-as-code documentation.* https://www.openpolicyagent.org/docs/latest/

[R40] Argo CD. *Declarative GitOps continuous delivery for Kubernetes.* https://argo-cd.readthedocs.io/en/stable/

Document maintenance

Review this playbook at least every six months and after a material change in regulation, foundation-model/provider landscape, enterprise risk appetite or platform architecture. Maintain a change log covering control changes, deprecated technologies, new workload archetypes and revised evaluation or incident practices.